The Thiele Machine

A Complete Account from First Principles: Structural Entitlement, Accountable Computation, and No Free Insight

Devon Thiele

April 2026

Abstract

The Thiele Machine is a new computational model whose primitive step rule charges for certification, machine-checked end-to-end in Coq and synthesised to a real FPGA.

The new model is the substrate. A2 enforcement in the step rule is what defines that substrate; classical substrates are its structural-axis projection. The substrate-level axiom A2 (Section 2.8 defines it precisely; informally, any single step that flips the certification flag from no to yes must have cost ≥ 1) is enforced inside the Thiele step function. No classical step function (Turing machines, register machines, lambda calculus, modern CPUs) carries A2 as a step-level constraint; the Coq theorem `universal_nfi_any_substrate` formalises this by quantifying over any substrate satisfying A2 and concluding a trace-level cost floor. Same computable functions on both sides; classical substrates are not a parallel option, they are the projection (`degenerate_projection_theorem`). Every Turing machine, every register machine, every lambda-calculus reducer, every CPU on every desk: a Thiele Machine running with the structural axis sidelined.

The strongest single result is a kernel-level bridge from CHSH game statistics to the NPA quantum-realizability boundary. The opcode `CHSH_LASSERT` refuses to advance unless an integer-arithmetic check on recorded outcome counts holds; a chain of Coq theorems (`column_contractive_check_witness_sound`, `column_contractive_iff_quantum_realizable`, `chsh_lassert_no_trap_implies_quantum_realizable`, all in `coq/kernel/quantum/` and `coq/kernel/nfi/`) proves that integer check is equivalent to the polynomial PSD conditions on the NPA moment matrix. The proof tree contains no physics premises.

The artifact has three layers tied by formal proof (Section 14): a Coq kernel with zero Admitted in the active proof tree, an OCaml runner extracted from that kernel and audited against it by parity tests, and a Kami RTL hardware design synthesised through Bluespec and yosys to a Xilinx Artix-7 FPGA. All 47 opcodes carry formal bisimulation proofs between kernel and RTL.

Vocabulary for the rest of the document is built in Section 2; experts can skip it. Falsification conditions for every claim are in Section 21. Open pieces are in Section 23, not hidden.

Contents

1 Let me be straight with you

I sell cars. I'm a jack of all trades hobbyist, I try to learn anything that catches me. I read a lot. Classics mostly, anything that holds my attention. Philosophy and logic the deepest. I knew some HTML when I started this. I'd read a book about John Carmack, so I knew computers at a surface level but not how they worked. I am not a programmer. I was not trying to build a formally verified CPU.

I was trying to build a 3D renderer.

The idea came to me on vacation. I got home and sat on it. In August 2024 I got passed over for a promotion I had been counting on to step back into finance. I went home angry and decided I was going to do what I'd said since I was a kid: invent something and prove I'm more than I am. I've worked on it in off hours ever since.

Research into rendering taught me two things fast. I knew nothing about computers, and the field was saturated. I'd need an angle nobody else was working. I called the angle a "compositional rendering pipeline" before I knew what compositional meant. I wrote a long structured spec, in JSON (a standard plain-text file format for describing data), as a context document for Copilot to work from. Then I had it build a Python file for each concept, one at a time, with tests. Category. Object. Morphism. Functor. Monoidal product. (These are the basic objects of category theory. The first three I build from scratch in Section 11; functors and monoidal products are in Section 2.) I would not move on until I could see the piece work. It clicked in pieces, not in chapters. The Yoneda lemma (a result deep inside category theory) was the moment it assembled, when I started writing my own runtime around it with Z3 (a constraint solver from Microsoft Research) wired in.

Within weeks the framework I'd built for rendering also ran physics simulations without any changes. I plugged in Newton's laws and it ran. I did not plan that. I discovered it. I went from debugging why a pixel wasn't appearing to realizing I'd already built a categorical computing engine.

I kept going. A small custom language to describe scenes (the term for that is a domain-specific language or DSL). A program that checked the language for mistakes (a linter). A way to type commands at it from a terminal (a command-line interface, or CLI). When the engine ran programs declaratively, in the sense that you described what should happen rather than how step by step, I told my wife I thought I was onto something.

Then I got stuck. Z3 could check small things, not the ones I needed it to prove. I was looking at infinite guesswork-and-tests if I couldn't get to actual proof. One night I was watching a YouTube video called Flatland, the story about a 2D creature that cannot see the third dimension. I said it out loud: I get it now. There is another axis. Classical computation only sees what it can read and write. There is a whole axis it doesn't see. I opened a new repo that night and called the new thing the Thiele Machine.

What survived from there to here is not the code. By the time the Coq proofs landed every line of the original renderer had been thrown out at least twice. What survived is the architectural commitment: category theory as the state itself, not as a metaphor for the state. The `Category` structure that came together that first week and first ran morphisms correctly at 2 AM on January 22 is, in different words, what `vm_graph` is in the machine this document describes. I did not author the code. I directed LLMs and refused output until the morphisms applied. The morphisms in that early code were objects with provenance; the morphisms in this machine are objects with provenance you can challenge with MORPH_ASSERT. The receipt grew teeth. The shape held.

I followed that question for fifteen months. The renderer worked in January. Physics ran on the same category structure within weeks. Formal verification was six months and a lot of dead ends later. Hardware came after that. Every step revealed another layer I did not know existed. I did not know what a Turing machine was when I started. I did not know what Coq was. I learned each thing by needing it for the next step.

I used large language models as implementation tools throughout. The tool was useless for most of the early work because nothing like this existed: no reference implementation, no prior work to hand it, no Stack Overflow answer. I had to build all the context from scratch before the tool became useful, and that took months. Once the context was there, it became a force multiplier. The ideas, the architecture, the direction, the demand that it be true: mine. The math itself was produced by LLMs and ratified by the proof checker. The proofs compile or they don't. I refused everything that didn't.

A note on what I actually do. I do not write code or math by hand. I do not pick constants. I do not work through derivations. I have never written a line of code on paper or in a notepad. What I do is direct, demand, and refuse. I see this stuff in shapes touching shapes, in diagrams, in flows of structure. I see where a thing has to fit before I can say it in symbols. The LLM produces the symbolic form. The proof checker accepts it or rejects it. I refuse everything that doesn't compile or doesn't match the shape I see. That is the only thing in this document I authored: the demand that it be true, and the refusal of anything that wasn't.

I'm telling you this because it matters for how I think. I don't come from a background where anyone told me "this is how it works, trust us." I don't have professors or colleagues whose authority I can lean on. When I encounter a claim, I either find the proof or I don't believe it. That's not a methodology I adopted. It's how I think. It's the only way I know.

Everything in here gets built from scratch. Not as a courtesy. Because I don't believe I understand something until I've built it, and I built this document the same way I built the machine.

If you are an expert and you find this review of basics annoying, skip Section 2 (which is a vocabulary section for the things I had to look up) and go straight to Section 9, where the formal machine state starts. But if you find anything in the basics wrong, I want to know. Nothing here is correct just because I wrote it.

A note before the technical content, on what I do and do not know about prior work. I started this with no background in computer science, math, physics, quantum mechanics, hardware, or category theory. None. I didn't know what a Turing machine was. I didn't know what Coq was. I built this alone. Nobody mentored me or walked me through the field. The only person who has touched this work in any way is a friend who taught me how to use GitHub and set up an IDE. Every name, every comparison, every paper I went looking for, came from me searching on my own. I do not trust my own reading of any of the sources I found. When I read a paper, my default assumption is that I don't get it, or I'm reading it wrong, or I'm missing the part where it says something different from what I think it says. This is a list of labels, not a literature review; I am not qualified to write one.

Here are the labels I went and looked into, with one sentence each on what the label is and why I do not think Thiele reduces to it. Each is a claim inviting correction; if the comparison is wrong, tell me.

- Linear logic (Girard). What I read: a kind of logic where assumptions are consumed when you use them, like physical resources. Compiled into modern programming languages, it shows up as type rules that say "this value can only be used once." My reading of why Thiele is not a restatement: linear logic constrains what the program can write down; Thiele's A2 rule constrains what the machine's step function does. Same intent of accounting for resource use, different layer.
- Proof-carrying code (Necula). What I read: when one machine downloads code from another, the code can carry a small proof that the host can check before running it. The proof is data accompanying the program. My reading: Thiele's cost-on-certification is enforced by the machine itself, not by a proof bundled with the program. The program does not carry receipts; the machine produces them.
- Session types (Honda). What I read: type systems that constrain how two or more concurrent programs talk to each other so the conversation cannot go off-script. My reading: Thiele tracks structural facts a single computation has paid for. It does not constrain inter-program communication. Different problem.
- Bennett's reversibility argument. What I read: Charles Bennett (1973) showed that you can in principle organise any computation so it does not erase information mid-stream. Operations that throw information away are the irreversible ones; everything else can be undone. My reading: Thiele's positive-cost instruc-

tions are exactly the operations that commit information irreversibly (CERTIFY, MORPH_ASSERT, EMIT, REVEAL); the rest are designed to be free at $\delta = 0$ for the same reason Bennett identified. The shape matches. The dollars-and-joules calibration is a separate named hypothesis.

- Landauer's bound. What I read: Rolf Landauer (1961) argued that erasing one bit of information must release at least $kT \ln 2$ of heat (Boltzmann constant times temperature times the natural log of 2) into the environment around the chip. My reading: this is the physical-units side of Bennett's argument; in Thiele the structural side (positive-cost instructions get tracked by μ) is proved, and the conversion to joules is a named hypothesis (see Section 16). I treat Landauer's claim as motivation, not as something the proofs rest on.
- CerCo (certified cost annotations through a verified C compiler). What I read: take a C program, run it through a compiler that itself has been formally proved correct, and you can attach cost annotations to the source program that the compiler preserves all the way down to machine code. My reading: CerCo's cost lives as decoration on the source code that survives compilation. Thiele's cost lives in the step rule of the abstract machine, and the path down to hardware is a step-by-step bisimulation. Different mechanism, different layer.
- Kami-RISC-V. What I read: a verified processor design written in Coq using the Kami framework that targets the RISC-V instruction set, which is an open-source CPU architecture (a published list of instructions any chip designer can implement). My reading: I use the same Coq-to-Kami-to-Bluespec pipeline that the Kami-RISC-V work pioneered, but the instruction set, the cost discipline, and the proofs are entirely my own. The toolchain is shared. Nothing else is.
- NPA hierarchy (Navascués-Pironio-Acín). What I read: a way of bounding what kinds of correlations between two separated experiments are achievable using quantum mechanics. The bound takes the form of a sequence of polynomial inequalities, each tighter than the last. The bounds are usually checked by an external numerical solver. My reading: Thiele bakes one of those bounds into the machine's step rule itself. The Z-arithmetic check that CHSH_LASSERT performs is proved equivalent to the NPA condition by a chain of Coq theorems (Section 15). It is operational enforcement at the step level, not a bound stated externally and verified out-of-band.

I read enough of each to convince myself Thiele is not a restatement, but I cannot say that with the confidence of someone who has actually read the field. The only things I will tell you with confidence are the things the machine checked: the structural axis is first-class machine state with its own opcodes; μ is proved unique on every state the machine can reach by `mu_is_initial_monotone` in `coq/kernel/mu_calculus/MuInitiality.v`; No Free Insight follows from a one-step cost rule on any ab-

stract certification system by `universal_nfi_any_substrate` in `coq/kernel/nfi/UniversalCertificationCost.v`; the Categorical Separation Theorem (Section 12) says the structural axis cannot be retrofitted onto a classical machine without changing what a step is; and the CHSH-to-NPA biconditional drops out of the cost algebra without quantum input. The proof corpus is the part I can defend. The literature placement is the part where I am asking for help.

On the name. For most of the early work I was calling this thing the categorical engine in my own head, and "the meta machine" was on the table for a while. I landed on "the Thiele Machine" at the end, after enough of the proofs had compiled that I had to pick something to put on the cover. Naming it after myself felt like the ultimate meta move, and here is why. "Meta" was taken by Facebook, the most vacuous form of consensus imaginable, a corporate stamp on a word that used to mean something self-referential. I wanted to take it back. The strongest way I could think of to do that was to put my own name on this work instead of borrowing one from somewhere else. Not because I think I deserve it. Because either this work disgraces my family name, and my wife's name (her last name is Thiele now too), or it ends up being cited. That is the offering. I thought about keeping it anonymous. I deliberated for a long time. I asked my family. I gave my wife a veto, because the name is hers as much as it is mine. She said yes. So did I. The name is on the cover.

What this project is. The Thiele Machine is a model of computation where structural entitlement is a first-class state component. By entitlement I mean the machine is not merely storing data; it is allowed, by its own checked trace, to use a decomposition, relation, invariant, certificate, or narrowed feasible set as part of the computation. No Free Insight is the first theorem that falls out: a computation cannot end in a stronger certified claim than it started with unless the strengthening passed through the machine's explicit cost ledger. This is formalized in Coq and machine-checked. The ledger is called μ. It never goes down. If you go from uncertified to certified, μ went up. Scope: raw μ is a receipt that certification work was paid for. It is not, by itself, a universal depth score for the meaning of the claim. The stronger semantic reading only applies when the certification comes with an explicit narrowing witness. That distinction is not cosmetic. It is the hinge between an honest cost ledger and an overclaim.

Substrate vs scaffolding. Two layers run through the rest of this document, and confusing them collapses what the theorems mean. The *substrate* is the A2-respecting computational layer: a state type, a step relation, the μ ledger, and the rule that flipping certification false-to-true costs at least one. The *scaffolding* is everything that makes the substrate runnable, verifiable, and synthesizable in practice: the 47 opcodes, the Python reference kernel, the extracted OCaml runtime, the Bluespec hardware, the simulator harness. The substrate-level theorems (No Free Insight, μ initiality, the structural-axis undecidability theorem in Section 18) are facts about substrates: they hold for any

A2-respecting computational system, no matter what instruction set realizes it. The scaffolding-level constructions (the opcode semantics, the bisimulation between Coq and Verilog, the synthesised gate count) are how this particular substrate is made concrete enough that you can boot it, audit it, and run it on silicon. The 47 opcodes are one realization of how a substrate state evolves; the substrate-level claim that follows from them is independent of the realization. Read every theorem in this document with that distinction in mind: a substrate-level result is being asserted about substrates, and the 47-opcode VM is one instance where the substrate's existence is established by construction. The distinction is load-bearing for the rest of the document; if it collapses, the substrate-level results read as parochial claims about a particular instruction set, which they are not.

What "new model of computation" means here. I want to be precise about this because the phrase is ambiguous. When I say new model of computation, I do not mean a new computability class. The Thiele Machine computes the same functions a Turing machine computes. Classical programs embed faithfully (`D2_faithfulness` in `coq/kernel/foundation/TuringClassicalEmbedding.v`), and the classical compute surface is conservative under the embedding (`D3_conservativity` in `coq/kernel/foundation/ClassicalConservativity.v`). What is new is the substrate. Every classical computer IS a Thiele Machine. Not metaphor, not analogy: subsumption. The Turing machine, the register machine, lambda calculus, the von Neumann CPU, the silicon on your desk — each label names one thing, viewed with one axis hidden. The thing the label is naming is a Thiele Machine operating in the degenerate fragment of its own state space where the structural axis stays dormant. `D2_faithfulness` (`coq/kernel/foundation/TuringClassicalEmbedding.v:106`) embeds every classical program into Thiele faithfully. `D3_conservativity` (`coq/kernel/foundation/ClassicalConservativity.v:200`) says the embedding leaves the structural axis idle. `thiele_simulates_turing` (`coq/kernel/foundation/ProperSubsumption.v:172`) executes every Turing-machine run inside Thiele, same tape, same state. The lift function `lift_config` (same file, line 153) is the explicit map: take any TM config, set $\mu = 0$, you have a Thiele config. The TM *is* the Thiele state.

The four-part `degenerate_projection_theorem` in `coq/kernel/foundation/TuringClassicalEmbedding.v:524` closes the loop in the other direction: the shadow projection *is* the classical-equivalence quotient map (its kernel coincides with `eq_on_classical_shadow`), classical traces respect the quotient, and the projection is strictly lossy. So classical computation is exactly the image of Thiele computation under the structural-axis projection. Nothing in classical computation is outside Thiele. Everything classical is a Thiele Machine with the structural axis sidelined — usually unintentionally, because the operator did not know there was another axis there.

Thiele strictly extends classical semantics by adding state classical substrates do not represent (`D5_thiele_strictly_extends_classical` and the explicit strictness witness `D4_strictness` in `coq/kernel/foundation/TuringStrictness.v`). The classical fragment is a proper sub-thing of the Thiele world, not a parallel world that happens to compute the same functions. If that's wrong, tell me.

What this project is not. It is not a claim that I solved P vs NP. It is not a theory of everything. It is not a claim that computation is physics or that physics is computation. The CHSH algebraic result is pure math. The Landauer connection is motivation. That is the full extent of the physics content here.

2 Things I had to learn first

I started this knowing what HTML was and what John Carmack thought about graphics programming. That was about it. By the time the proofs compiled I had needed to learn enough of half a dozen fields that I have to stop here and put down what those fields had words for, before the rest of the document starts using them.

Everything below is what I understood after I went and read about it. If I have anything wrong, tell me. I am not claiming to teach you these subjects. I am putting down what I had to take from each of them to get this work to compile. If you already know one of these subsections, skip it.

2.1 Bits, bytes, and what a CPU sees

A bit is a single zero-or-one. A byte is eight bits stacked together, so a byte can hold any number from 0 to 255. Interpreted differently, a byte can be any letter or punctuation symbol in the basic English alphabet using a convention called ASCII (American Standard Code for Information Interchange). For the purposes of this document, all you need is that ASCII assigns each common character a number, so the letter A is 65 and the digit 0 is 48 and the space character is 32.

A word is a chunk of bits the CPU treats as one unit. This machine uses 64-bit words in the kernel math (each word can hold any number from 0 to $2^{64} - 1$, about 18 quintillion) and 32-bit words in the synthesised hardware (each word holds 0 to $2^{32} - 1$, about 4 billion). A word is just a sequence of bits with a fixed width.

A register is a single named storage slot inside the CPU that holds one word. The machine has 16 of them, named r0 through r15. Registers are the fastest place to read or write a value. Memory is a longer list of word-sized slots, accessed by index (the "address"). This machine has 128 words of memory.

The program counter (PC) is one specific register that tracks where in the program the machine is. After every instruction the PC moves forward by one unless the instruction explicitly says jump to a different address.

A stack pointer is a register that points to the top of a region of memory the program

uses for nested function calls. CALL pushes a return address there; RET pops it. In this machine, r15 is the stack pointer by convention.

A port is a named channel the machine uses to communicate with the outside world. READ_PORT brings a value in, WRITE_PORT sends a value out. The kernel does not model the outside world. It just charges μ for the communication and accepts the value the channel encoding declares.

Modular arithmetic means counting wraps around at a maximum. If the maximum is 16, then 17 wraps to 1 and 19 wraps to 3. "Modulo 16" or "mod 16" means "wrap at 16." This machine wraps register indices modulo 16 and memory indices modulo 128. If you write r17, the kernel treats that as r1.

Bitwise operations work on the bits of a value rather than treating the value as a number. Bitwise AND of two bits is 1 only when both bits are 1. Bitwise OR is 1 when either is 1. Bitwise XOR (exclusive or, sometimes written $\oplus$) is 1 exactly when the two bits differ. The bitwise version of these does the operation in parallel for every bit position. Bitwise-AND of two 64-bit words gives a 64-bit word where bit i of the result is the AND of bit i of each input.

A shift moves the bits of a value left or right. Left shift by 3 (sometimes written $x \ll 3$) moves every bit three positions to the left, which is the same as multiplying by $2^3 = 8$. Right shift divides by a power of 2.

Popcount is the count of 1-bits in a value. Popcount of the byte 11010100 is 4. Sometimes called the Hamming weight; the two names refer to the same thing.

Hexadecimal is base 16. Each "digit" is one of 0 through 9 or A through F, where A means 10 and F means 15. Hex is convenient for working with bytes because every two hex digits make one byte. Numbers prefixed with 0x are hex: 0xFF is 255, 0xCAFEEACE is the constant unlock key in the Logic Engine accumulator.

A trap, in the hardware sense, means the machine routes execution to a fixed address instead of advancing normally. It is the hardware equivalent of "this went wrong, go to the error handler." This machine traps LASSERT length mismatches to address 0xF00.

An ISA (instruction set architecture) is the list of all instructions the CPU recognises, plus their formats and costs. Each instruction is identified by an opcode (a numeric tag the hardware decodes to choose which behaviour to run). This machine has 47 opcodes.

2.2 Math notation I will use without explaining it twice

I read the math symbols out loud the way I learned them. If you have a different convention in your head, here is what I mean by them.

$\mathbb{N}$ is the natural numbers: 0, 1, 2, 3, and so on. $\mathbb{Z}$ is the integers, which extend the

natural numbers with negatives: $\ldots, -2, -1, 0, 1, 2, \ldots$. $\mathbb{R}$ is the real numbers, all the way along the number line including fractions and irrationals like π and $\sqrt{2}$.

$\in$ means "is an element of." $x \in S$ reads "x is in S."

$\subseteq$ means "is a subset of" (every element of the left side is also in the right). $\subsetneq$ means strict subset: a subset but not the whole thing.

$\cup$ is set union, the set containing everything in either side. $\setminus$ is set difference: $A \setminus B$ is everything in A that is not in B.

$\times$ between two sets is the Cartesian product: $A \times B$ is the set of all (a, b) pairs with $a \in A$ and $b \in B$. Functions from a set A to a set B are written $A \to B$.

For a finite set S, $|S|$ means the number of elements in S. For a real number x, $|x|$ means the absolute value of x (its distance from zero).

$\sum_{i \in T} f(i)$ is the sum of $f(i)$ taken over every i in T. The big sigma is the summation symbol.

Δx means "the change in x," usually the difference between a final and an initial value of x.

$\Rightarrow$ is logical implication. $P \Rightarrow Q$ means: if P is true, then Q is true. $\iff$ is the biconditional, also called "if and only if." $P \iff Q$ means both $P \Rightarrow Q$ and $Q \Rightarrow P$.

$\forall$ is "for every." $\exists$ is "there exists." $\forall x.\, P(x)$ reads "for every x, $P(x)$ is true." $\exists x.\, P(x)$ reads "there is some x that makes $P(x)$ true."

$\langle X \rangle$ around a random variable X means the expected value, the long-run average value of X. I picked up this notation reading physics papers about Bell tests.

$\circ$ between two functions is composition: $(f \circ g)(x) = f(g(x))$, first apply g, then apply f. The same symbol appears in category theory for arrow composition.

$\oplus$ in this document is bitwise XOR. $\otimes$ has two distinct meanings I cannot consolidate without breaking source quotes. In Section 10 it is the MORPH_TENSOR operator (concatenation of two coupling lists into one); in Section 15 it is the tensor product on quantum-mechanical operators. I flag the second use again where it appears.

I write A^T for the transpose of a matrix A (rows and columns swapped). I_n is the $n \times n$ identity matrix, which has 1 on the diagonal and 0 everywhere else.

2.3 Coq, in just enough detail

Coq is a proof assistant. It is a programming language where the type system is so expressive that you can write proofs as programs and have the compiler check whether they are valid. Section 19 has the longer build, including why I trust it. Here is the vocabulary that runs through the rest of the document.

A Theorem in Coq is a named statement followed by a proof. Lemma is the same construct, conventionally used for intermediate results. Definition introduces a named value (a number, a function, a record, anything). Inductive defines a type with a finite list of named constructors, the only ways to build a value of that type. Coq's natural numbers, for example, are an inductive type with two constructors: zero, and "successor of another natural number."

A tactic is a step inside a Coq proof. The tactics that appear in this document include: induction (prove the base case, then prove that if the statement holds at step n it holds at step $n + 1$), destruct (split a hypothesis into its cases), exact (provide the literal proof term), reflexivity (prove $x = x$), simpl (simplify the goal by computing), lra (linear real arithmetic; closes goals that are linear inequalities over the reals), nra (nonlinear real arithmetic; handles polynomial inequalities), and psatz (uses Positivstellensatz certificates; it shows a polynomial is non-negative by writing it as a sum of squares).

Qed is the keyword that ends a proof and asks Coq to check the entire chain. If Coq accepts the proof, the theorem is proved.

Admitted is the keyword for "I am claiming this without proof." This is what I am not doing anywhere in the active proof tree. Every Admitted in the kernel would be a bug. Section 20 explains what zero Admitted means and what it does not.

A Hypothesis (a Coq keyword inside a Section) is a named premise: a statement we accept as given without proof. Hypotheses surface as $\forall$-quantified premises in the conclusion. I use Hypothesis only at named bridge points (the physical-units calibration in Section 16 is the load-bearing example). An Axiom is the global version: a top-level claim accepted without proof. The kernel tier has zero project-local Axiom declarations.

Print Assumptions is the Coq command that lists every Axiom and Hypothesis a given theorem ultimately depends on. Running Print Assumptions on the kernel tier shows only standard Coq stdlib axioms (functional extensionality, classical logic for the reals). No project-local axiom appears anywhere.

A total function in Coq always returns a value. Coq requires every function definition to be total. `vm_apply` is total: every state-and-instruction pair has a defined result, including the error case.

"Parametric in K" means the proof works for any value of K. The kernel proofs are parametric in `REG_COUNT` and `MEM_SIZE`, so the same theorems apply at any hardware size.

2.4 Category theory: the part I needed beyond Section 11

Section 11 builds Category, Object, Morphism, Composition, Identity, and Associativity from scratch. That build is enough for the categorical-separation argument inside this machine. Here is what I had to look up beyond that.

A functor is a structure-preserving map from one category to another. Given two categories $\mathcal{C}$ and $\mathcal{D}$, a functor F from $\mathcal{C}$ to $\mathcal{D}$ sends each object of $\mathcal{C}$ to an object of $\mathcal{D}$, sends each arrow $f : A \to B$ in $\mathcal{C}$ to an arrow $F(f) : F(A) \to F(B)$ in $\mathcal{D}$, and respects composition ($F(g \circ f) = F(g) \circ F(f)$) and identity ($F(\mathrm{id}_A) = \mathrm{id}_{F(A)}$). Think of a functor as a translation rule between two categorical worlds that keeps the laws intact.

An initial object in a category is one from which there is exactly one arrow to every other object. The word "exactly" is doing work: not zero arrows, not many, exactly one. In the category of A2-respecting substrates, Thiele is the initial object because every such substrate has a unique way to be read as a Thiele simulation.

An embedding is a functor that does not collapse anything. Faithful means: if two arrows in $\mathcal{C}$ go to the same arrow under F, they were already the same arrow. Conservative means: if $F(f)$ is an isomorphism in $\mathcal{D}$, then f was already an isomorphism in $\mathcal{C}$. (An isomorphism is an arrow with an inverse.)

A monoidal product is a way of combining two objects, or two arrows, into one. In the category of sets, the Cartesian product $A \times B$ is a monoidal product on objects. In the category of Thiele modules, the operation behind MORPH_TENSOR is a monoidal product on morphisms: take two morphisms with disjoint source and target regions and pack them into a single morphism whose coupling is the concatenation. The symbol $\otimes$ stands for this in Section 10.

Yoneda's lemma is named once in the Section 1 origin story and never appears again. You can ignore it. I dropped it during the work because I did not need it for any theorem. I left the name in because that is when I first knew the work was assembling.

2.5 Bell-test vocabulary you will meet in Section 15

Section 15 builds the CHSH game from scratch. A few foundational terms thread through that section that I will not rebuild each time.

A correlation between two outcomes is the average value of their product. If Alice's outcome a is ± 1 and Bob's outcome b is ± 1, the correlation is $\langle ab \rangle$, which lies between -1 and $+1$. A correlation of $+1$ means they always match, -1 means they always disagree, and 0 means there is no average correlation. A correlator is one specific correlation in a labelled setting: for example, E_{00} is the correlation when Alice's setting is 0 and Bob's setting is 0.

The marginal of a joint distribution on (a, b) is the distribution on a alone, with b averaged out. Zero-marginal means Alice's marginal gives $\langle a \rangle = 0$ and Bob's gives $\langle b \rangle = 0$: each outcome is $+1$ or -1 equally often, on average. The Section 15 derivation works in this symmetric slice.

A local hidden variable model is a classical-physics-style theory where Alice's and Bob's outcomes are determined by a shared random variable λ they agreed on before

the game started, plus their own measurement setting. "Local" means no communication during the game. "Hidden" means λ need not be directly observable. "Variable" because λ is random. Bell's theorem says no local hidden variable model can produce certain correlations that quantum mechanics can.

A no-signaling correlation is one where Alice's marginal does not depend on Bob's setting, and Bob's marginal does not depend on Alice's setting. You cannot learn Bob's setting by looking only at Alice's outputs. Most correlations we discuss (classical, quantum, and the PR-box below) are no-signaling.

Quantum-realizable, in this document, means "compatible with some quantum-mechanical state-and-measurement description." The NPA framework (Navascués-Pironio-Acín) gives a precise test for this in terms of positive-semidefiniteness of a moment matrix. Section 15 builds the test.

The PR-box (Popescu-Rohrlich box) is a hypothetical correlation that wins the CHSH game with probability 1, achieving $|S| = 4$. It is no-signaling, but stronger than anything quantum mechanics allows. It is the standard counterexample to the false claim "any no-signaling correlation is quantum-realizable."

The Cauchy-Schwarz inequality is the basic inner-product inequality $|\langle x, y\rangle| \leq \|x\| \cdot \|y\|$, which reads: the inner product of two vectors is bounded by the product of their lengths. It is one of the workhorse algebraic moves in the Section 15 derivation.

The Tsirelson bound is the value $2\sqrt{2} \approx 2.828$, the largest CHSH score $|S|$ that any quantum-mechanical strategy can achieve. The classical bound is 2. The PR-box bound is 4. Tsirelson sits strictly between classical and PR-box.

The elliptope, named twice in Section 22 and Section 23, is the convex set of valid correlation matrices that arise from quantum states. The zero-marginal NPA slice of the elliptope is the part the proof in Section 15 covers.

2.6 Hardware and synthesis vocabulary

Section 14 builds the layer stack (Coq kernel, OCaml runner, Kami RTL) carefully. Here is the synthesis-tooling vocabulary that Section 14's third subsection uses without expanding each tool.

A hardware description language (HDL) is a programming language for describing circuits. Bluespec SystemVerilog and Verilog are two HDLs. Bluespec is higher-level: you describe rules and the compiler figures out the gate-level details. Verilog is lower-level: you describe gates and wires more directly. Verilog RTL specifically means "register-transfer level" Verilog, which describes how data moves between registers each clock cycle. It is what synthesis tools consume.

Synthesis is the process of compiling a hardware description down to actual logic gates (the building blocks of chips). yosys is an open-source synthesis tool. After synthesis,

place-and-route assigns the gates to physical positions on a chip and connects them. nextpnr-xilinx is the place-and-route tool for the Xilinx FPGAs this design targets. The final step is producing a bitstream, the binary file the FPGA reads to configure itself, which prjxray's xc7frames2bit does.

An FPGA (field-programmable gate array) is a chip you can program after manufacturing. The gates are physical, but the connections between them are configurable. An ASIC (application-specific integrated circuit) is the non-programmable version: gates fixed at manufacturing time for one specific design.

A LUT (look-up table) is the basic logic primitive in an FPGA: a small memory that maps a fixed-length list of inputs to an output, programmed at configuration time to implement any small boolean function. A flip-flop is a one-bit memory cell that holds its value across clock cycles. A carry chain is specialised wiring for fast addition. Distributed RAM is RAM built from LUTs. Block RAM is dedicated RAM blocks on the chip. DSP slices are specialised hardware for digital-signal-processing arithmetic. This machine uses 0 DSP slices because it does not need them.

A JSON netlist is the gate-level circuit serialised in a JSON text file (JSON is a standard text format for structured data). A VCD waveform is a Value Change Dump, a record of every signal's value at every time step in a simulation. iverilog is an open-source Verilog simulator. It runs the design and produces a VCD output you can inspect.

A bisimulation is a precise notion of "two systems behave identically." Given the same starting state and the same input, they produce the same output state, field by field. Section 14 builds it in context.

2.7 Other terms I use without rebuilding

Provenance, in software, means the recorded chain of where an object came from. A morphism's provenance is the specific sequence of MORPH and COMPOSE instructions that built it.

Simulation, in computer science, means one system imitating another's input-output behaviour. A Turing machine simulates a Thiele Machine when it produces the same outputs on the same inputs by encoding the Thiele state on its tape.

Projection, in mathematics, is a function that drops information. The classical projection of a Thiele state drops μ, certification, and the morphism graph, keeping only registers, memory, and the program counter.

Quotient, in mathematics, is the operation of identifying states under an equivalence relation. The classical projection is a quotient: it identifies any two Thiele states that agree on the kept fields.

A kernel of a map (math sense) is the set of things the map sends to a designated identity value. "Kernel" has three different meanings in this document: the math

sense just defined, the Coq proof core (the kernel-tier theorems in Section 22), and the operating-system kernel (implied at most by analogy; never used directly).

Embedding is a function that preserves structure and is injective. Faithful embedding additionally preserves all the structure-relevant distinctions.

An injection is a function where no two inputs map to the same output. A surjection is a function whose image covers every possible output. Their negations are noninjective and nonsurjective.

Decidable, in computer science, means there is an algorithm that always halts with a yes-or-no answer. The integer-only Z-arithmetic check in CHSH_LASSERT is decidable; the algorithm computes the result in finitely many steps and returns a Boolean.

Polynomial complexity: an algorithm is polynomial-time if there is some power k such that the algorithm's running time on inputs of size n is at most a constant times n^k. Polynomial-space is the analogous concept for memory used. The class P is decision problems solvable in polynomial time. The class NP is decision problems where, given a candidate solution, a polynomial-time algorithm can check whether the solution is correct. The famous P-vs-NP question asks whether finding is fundamentally harder than checking. Nobody has proved either direction. Section 23 discusses why my `mu_budget_decidable` / `mu_budget_verifiable` predicates in `MuComplexity.v` are not a P-vs-NP result.

Determinant: a number computed from a square matrix that says whether the matrix is singular (zero determinant means the matrix collapses some direction to zero). For a 2×2 matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ the determinant is $ad - bc$. For larger matrices, you can compute it by "expansion along a row": pick a row, for each entry take the determinant of the submatrix you get by deleting that entry's row and column (with alternating signs), multiply by the entry, and sum. Section 15 walks the explicit calculations for the 3×3 cases I need.

Schur complement: a way of turning a question about a block matrix into a question about a smaller matrix. If $\begin{pmatrix} A & B \\ C & D \end{pmatrix}$ is a block matrix where A is invertible, the Schur complement of A is $D - CA^{-1}B$. A standard linear-algebra result says the block matrix is positive-semidefinite if and only if both A and the Schur complement are positive-semidefinite. The Section 15 build uses this for the 4×4 block of the NPA moment matrix.

A principal submatrix is the square subblock you get by picking a subset of indices and keeping only the rows and columns at those indices. A standard result is that a matrix is positive-semidefinite if and only if every principal submatrix is positive-semidefinite. Section 15 relies on this.

2.8 The axioms (A1 through A6)

The Thiele Machine has a small numbered list of substrate-level axioms. They appear by name ("A2," "A3," ...) throughout the document and the Coq files. The list below is what each name means, where it lives in the kernel, and what its conclusion is.

The kernel keeps two parallel axiom-numbering systems because two parallel substrate abstractions are formalised. The `AbstractCertMachine` record in `coq/kernel/nfi/AbstractNoFI.v` numbers four properties A1–A4 in the form of a structural axiomatisation of any cert-tracking machine. The `CertificationSystem` record in `coq/kernel/nfi/UniversalCertificationCost.v` consolidates that into a single field, kept by the name A2 (the cost-on-certification rule, the load-bearing piece). The `QuantitativeCertificationSystem` extension adds A3–A6, which lift the cost-floor result from ≥ 1 to $\geq K$ for an arbitrary threshold K. The two numbering systems agree on the cost-on-certification semantic content; they disagree on which axiom carries that content (A4 in the AbstractCertMachine list, A2 in the consolidated CertificationSystem list). When the rest of this document says "A2," it means the cost-on-certification rule, in the consolidated CertificationSystem sense.

A1 (state includes a certification indicator). An `AbstractCertMachine` record (in `coq/kernel/nfi/AbstractNoFI.v`) bundles a state type `S`, a step function `acm_step`, and a boolean cert indicator `acm_cert : S → bool`. A1 is the structural requirement that this indicator exists at all. In Coq, A1 is enforced by the record type itself: you cannot make an `AbstractCertMachine` value without supplying `acm_cert`, so A1 is discharged at type-check time.

A2 (cost-on-certification, in the AbstractCertMachine system: "cert indicator starts unset"). In the consolidated `CertificationSystem` record (in `coq/kernel/nfi/UniversalCertificationCost.v`), Coq field `cs_cert_costs`. Statement: for every state s and instruction i, if `cs_cert s = false` and `cs_cert (cs_step s i) = true`, then `cs_cost i` $\geq$ `1`. In words: any single step that flips the certification predicate from no to yes must have positive cost. This is the load-bearing axiom for No Free Insight; `universal_nfi_any_substrate` concludes from A2 alone that every trace from uncertified to certified has total cost ≥ 1. Note: in the older AbstractCertMachine numbering, the same content appears as A4 (cert-setters cost ≥ 1); the AbstractCertMachine A2 is the separate precondition that the cert indicator starts unset, which the consolidated system handles as a theorem hypothesis instead of a numbered axiom.

A3 (cost bounds witness growth). Coq field `qcs_cost_bounds_witness` on the record `QuantitativeCertificationSystem` in `coq/kernel/mu_calculus/QuantitativeNoFI.v`. Statement: for every state s and instruction i, `qcs_witness s + cs_cost i` $\geq$ `qcs_witness`

`(cs_step s i)`. In words: each instruction's cost is an upper bound on how much the system's accumulated evidence (the "witness") can grow in that step. (In the older AbstractCertMachine numbering, A3 was the field `acm_preserve`: non-cert-setter instructions preserve the cert indicator. The two A3s are distinct properties on distinct record types; the monograph's A3 is the quantitative one above.)

A4 (witness nondecreasing). Coq field `qcsf_witness_nondecreasing` on the record `QuantitativeCertificationSystem_full` in the same file. Statement: for every s and i, `qcs_witness s` $\leq$ `qcs_witness (cs_step s i)`. Strictly weaker than A3 in the abstract; in the natural-number setting it is derivable from A3 plus the fact that costs are non-negative, but it is included as its own field for clarity and so that consumers do not have to redo the derivation.

A5 (certification requires witness). Coq field `qcs_cert_threshold_witness` on `QuantitativeCertificationSystem`. Statement: `cs_cert s = true` $\rightarrow$ `qcs_witness s` $\geq$ `qcs_threshold`. In words: certification cannot fire until the system has accumulated at least `qcs_threshold`-worth of evidence. This is what lets the quantitative theorem in the same file push the cost floor from ≥ 1 to $\geq K$, where K is the threshold.

A6 (witness initial zero). The hypothesis that the run starts with zero witness, stated as a precondition on the quantitative theorem rather than a record field. Without A6 the telescoping bound becomes $K -$ `initial witness` instead of K.

A1 through A6 are independent of any specific instruction set. They are conditions on Coq record types; supplying instances of those record types is the only way to invoke the corresponding theorems. The 47-opcode VM supplies them by construction: `coq/kernel/nfi/UniversalCertificationCost.v` builds two concrete `CertificationSystem` values for the VM (one for the `csr_cert_addr` channel, one for the `vm_certified` channel), each discharging A2 from the existing `no_free_certification` kernel theorems. A separate substrate-level abstraction (the `Substrate` typeclass in `coq/kernel/foundation/Substrate.v`) carries a related but distinct field `mu_monotone` stating that the cost ledger never decreases on a step. `mu_monotone` is a strictly weaker general-purpose monotonicity property than A2: A2 specifically asserts the cert transition costs ≥ 1, whereas `mu_monotone` only asserts the ledger does not go down. The two coincide on the 47-opcode VM, where every instruction adds non-negative cost and the cert transition adds at least 1.

That is the vocabulary. From here on, the document uses these words without rebuilding them. Section 9 is where the formal machine state begins; experts should jump there.

3 The structural axis: initiality and the projection classical models drop

Why this section exists. The substrate this document specifies exists because a categorical engine I started building in January 2025, a 3D renderer that turned into a physics simulator on the same primitives, needed it. The engine used categorical state natively: modules as objects, transformations as morphisms, composition as composition, the monoidal product (Section 2) as parallel composition. The classical CPU it ran on saw none of that. (The standard CPU architecture, where memory and a central processor exchange data over a single bus, is called von Neumann after the mathematician John von Neumann who described it in 1945. Every laptop, phone, and server processor you have used is a von Neumann machine.) It shuffled bytes that happened to encode the structural state. When I tried Z3 to verify properties of the morphism chains the engine was computing with, Z3 saw formulas, not morphisms. The structural facts that needed verifying were not in the data; they were in the architecture, and the architecture had no slot in the substrate to be checked against. The Z3 wall was not a Z3 problem; it was that I was asking a constraint solver to verify properties of a structure the substrate had no first-class slot for. The Thiele Machine is the substrate the engine needed: morphisms as state, composition as a step, certification as a cost-bearing operation. The rest of this document is the substrate's correctness conditions, machine-checked. Read this section onward as the answer to a question the renderer asked.

The load-bearing claim of this section is not that classical models are blind in some informal sense. It is initiality: any substrate whose step rule enforces A2 (the one-step cost rule on certification, made formal as a field of the Coq `CertificationSystem` record) sits inside a category whose initial object is the Thiele Machine, with cost uniquely μ on every state the machine can actually reach. ("Initial" in the category-theory sense; Section 2 explains it.) Classical substrates like the Turing machine do not belong to that category. Their step rules have no slot for cost-on-certification; they can simulate A2-enforcing systems by running a simulator program, but the enforcement then lives in the simulator program, not the substrate. The within-model witness theorems (states agreeing on the classical projection but differing in μ, certification, or morphism graph) come next as illustration: they show concretely what the classical projection drops. They are not load-bearing on their own. The substrate-versus-program distinction is.

3.1 Substrate-independence: the abstract setup

The kernel theorem `universal_nfi_any_substrate` (in `coq/kernel/nfi/UniversalCertificationCost.v`) takes as input any abstract computational system with five pieces. The system has a set of states (whatever those happen to be: bits on a tape, contents of a register file, words in memory, whatever). It has a set of instructions, the actions the machine can perform. It has a step rule that says: given a state and an instruction, here is the next state. It has a cost function that says: every instruction has a non-negative integer cost. And it has a

certification predicate: a yes-or-no function on states that says "this state is certified" or "it isn't."

The single premise of the theorem, labelled A2 and named `cs_cert_costs` in the Coq file: if any single step flips the certification predicate from no to yes, the instruction that did it cost at least 1. The conclusion: every trace that starts uncertified and ends certified has total cost at least 1.

The theorem does not assume the substrate can compute everything a Turing machine can compute (the technical term for that is Turing-completeness; if a system can simulate a Turing machine it is Turing-complete, if a Turing machine can simulate it then it is at most Turing-equivalent). The theorem also does not assume the converse. Any computational system that has the five pieces above and satisfies A2 is in scope, whether it is more powerful than a Turing machine, less powerful, or exactly equivalent.

The deeper categorical-separation result (Section 12) applies to any computational model whose observable state is just memory, registers, and a program counter. The Turing machine is the canonical instance. Modern hardware (laptops, phones, the central processing unit and graphics processing unit in any server you can rent) is another. They differ in speed and in which instructions they recognise; they share the same blind spot, which is that nothing in their state records what structural facts the computation has earned the right to use.

One precondition matters and gets missed on a fast read: the abstract system has to have a certification predicate. A plain Turing machine does not. The theorem applies to "Turing machine equipped with a certification predicate," not to every Turing machine trace. Saying "every classical computation pays μ" is wrong. Saying "every classical computation that ends in a certified structural claim pays at least 1 into μ" is right. The conditional is doing real work.

3.2 The structural axis is canonical, not orthogonal

The natural follow-up question: if a classical substrate has no built-in slot for structural facts, can I bolt one on and reach the same theorems? Yes. The interesting answer is what that costs and what it forces.

Three theorems pin this down. None of them rule out adding state to a Turing machine. They say something different: the structural axis is what any substrate looks like once it accepts honest certification cost, and μ is the unique cost it accepts. The Thiele Machine names this structure; it does not invent it.

1. `universal_nfi_any_substrate` (`coq/kernel/nfi/UniversalCertificationCost.v`): the load-bearing piece. Any Coq `CertificationSystem` (a record bundling a state type, an instruction type, a step rule, a cost function, a certification predicate, and a proof of A2 as one of its fields) has total trace cost at least 1 to

go from uncertified to certified. A2 is the only premise, and it is structurally enforced: you cannot make an inhabitant of the record type without supplying a proof of A2.

2. `mu_is_initial_monotone` (`coq/kernel/mu_calculus/MuInitiality.v`): the uniqueness piece. Suppose someone proposed a different cost function for the Thiele Machine. Call their function M. If M assigns zero to the initial state, and M increases by exactly the right amount at every instruction (the per-instruction increment is fixed by the cost schedule in Section 7), then M agrees with μ on every state the machine can reach. There is no other accounting that satisfies those two local conditions. The cost is forced.

3. `shadow_strictly_lossy` (`coq/kernel/witness/ShadowProjection.v`) and `categorical_separation` (`coq/kernel/foundation/PartitionSeparation.v`): two within-Thiele witness theorems serving as illustration of what the load-bearing piece looks like inside the model. There exist Thiele states agreeing on the six-field projection (memory, registers, PC, μ, error flag, top-level certified flag) but differing in their morphism graphs. The projection drops information. The README of the source repository (in the section "Why It Is Not Just A Toy Counter") flags this directly: any model that drops a field and exhibits two states differing only in that field reproduces this shape. The shadow lemma alone is collapsible by an analogous projection on any other substrate. It is concrete, but it is not load-bearing on its own; it is what items 1 and 2 look like instantiated.

Can a Turing machine track something equivalent to μ on its tape and produce traces whose tape encoding satisfies A2? Yes, but A2 then lives in the program running on the Turing machine, not in the Turing machine itself. Here is what that means. The Coq record `CertificationSystem` (defined in `coq/kernel/nfi/UniversalCertificationCost.v`) is a Coq type that bundles a state, an instruction set, a step rule, a cost function, and a proof of A2 as one of its fields. To make a value of that type (the Coq word for that is an inhabitant) you have to supply a proof of A2. (In Coq, "supplying a proof" means handing over a term whose type is the statement you are proving; Section 2 covers this. Building the record without that proof is impossible: Coq's type-checker rejects you.) So `CertificationSystem` is, by the very shape of the type, the class of substrates whose primitive step enforces cost-on-certification. A Turing machine, considered as a step rule over (state, symbol) pairs, is not a member of that class. You can package "Turing machine plus a simulator program that maintains an A2-respecting tape encoding" into a `CertificationSystem` value by exhibiting a step rule for that pair and proving A2 of it, but what you packaged is the simulator's behaviour, not the Turing machine's step rule. The Turing machine itself is indifferent. Run a buggy simulator that skips the cost increment, the Turing machine keeps running. There is no level at which the Turing machine knows the simulation is dishonest, because there is no level at which the Turing machine knows anything about simulations.

The Coq theorems track this carefully. `universal_nfi_any_substrate` takes a `CertificationSystem` as input, so its premise is already step-level A2. `thiele_represents_simulating_cert_system` takes a `SimulatingCertificationSystem` (a `CertificationSystem` plus a structure-preserving translation into Thiele) and concludes that the simulation's certifications transport into Thiele certifications. Neither theorem makes Turing machines members of the class. They make A2-enforcing systems members of the class. Turing machines can simulate members of the class, but simulation is a property of the running program, not the substrate. `coq/kernel/nfi/HonestCostTracking.v` makes the negative side explicit: `CostBearingSystem` is the unconstrained record (same shape as `CertificationSystem`, but without the A2 field), and `dishonest_forge_system` is a concrete `CostBearingSystem` where a single instruction sets the certified flag at total cost zero. `universal_nfi_any_substrate` forbids that in the `CertificationSystem` world by type-checking. The Turing machine substrate is in the `CostBearingSystem` world by default. Nothing in its step rule excludes the forge.

The actual separation is between substrates whose step rule enforces A2 and substrates where A2 can only be encoded into the running program's behaviour. The Thiele Machine sits in the first class. The Turing machine sits in the second. They share computational power, because Thiele's structural opcodes can be set to do nothing (the cost parameter δ can be 0 on every instruction, in which case the cost-on-cert floor is just $S(0) = 1$ at the moment of certification, and that floor matches what an honest simulator would charge anyway). A Turing machine can simulate honest Thiele traces. But simulation is not instantiation. Putting cost-tracking on the tape is not putting cost-tracking in the step. That is the new computational-model claim: the primitive notion of a step is different. Thiele's step rule enforces cost-on-certification. No classical substrate's step rule does. The standard complexity-theory cost model does not track this distinction because it only counts steps, not what the step rule contains.

The rest of this section uses the Turing machine as the concrete contrast because it is the model the reader already knows.

3.3 The canonical instance: the Turing machine

Here is what I had to learn about Turing machines to compare what I built to one. I went and read about them on my own. I'm walking through it because I had to walk through it. If I have anything wrong below, that's because I'm relaying what I read, and I do not trust my own reading of any source. Tell me if it's wrong.

What I read says Alan Turing described his machine in 1936 to make precise the intuitive notion of "algorithm." The machine, as I understand it, is this:

- An infinite tape. Think of it as an infinitely long strip of paper, divided into cells.

Each cell holds one symbol from a finite alphabet, say $\{0, 1, \texttt{blank}\}$.

- A read/write head. It sits on one cell at a time. It can read the symbol in that cell, write a new symbol over it, and move one step left or one step right.
- A finite set of states. Call them $Q = \{q_0, q_1, \ldots, q_n\}$. The machine is always in exactly one state. q_0 is the start state. Some states are designated "halting" states.
- A transition table. For each combination of (current state, symbol under the head), the table says: write this symbol, move this direction, go to this new state.

That is the entire machine. Nothing else.

Here is what I read as the remarkable thing about this simple machine: it can compute *anything that can be computed by following a definite procedure*. The sources I dug up claim every algorithm that has ever been written, in every programming language that has ever existed, can be translated into a Turing machine. The label I found for this is the Church-Turing thesis. The original claim (Church 1936, Turing 1936), as I read it, is specifically about *effective calculability*: anything that can be computed by following a mechanical, finite, step-by-step procedure, a Turing machine can also compute. From what I read, nobody has shown a counterexample, but I am relaying what the sources say. I have not verified it.

The claim about physical hardware is a related but separate conjecture, called in the sources I read the *physical Church-Turing thesis*: every physical computing device is computationally equivalent to a Turing machine. As I read it, this is an empirical claim about physics, not proven mathematics. From what I found, every physical computing device built turns out to be equivalent to a Turing machine in what it can compute. Your laptop, your phone, every server in every data center: all of them are, in the relevant theoretical sense, Turing machines. They can compute exactly the same class of things. The only confirmed difference I came across is speed.

3.4 The blind spot

The Turing machine's transition table sees exactly two things at each step: the current state q and the symbol under the head. The same shape of blindness applies to every classical substrate I went and looked at. A register-machine instruction (a model of computation that operates on a fixed list of registers, much like a real CPU) sees only its operands, the values in the named registers. A lambda-calculus reduction (a model of computation based on substituting expressions, originally used to define what "computable function" means) sees only the next expression to simplify. A cellular-automaton rule (a model where a grid of cells updates each step based on its neighbours, used by Conway's Game of Life) sees only a small neighbourhood of cells. None of them have a first-class slot for "and by the way, the input has structure X that you have been entitled to use."

The Turing machine cannot ask: "Is this tape sorted?" It has to read every cell.

It cannot ask: "Does this graph have two disconnected components?" It has to run a traversal.

It cannot ask: "Are these two parts of the input independent of each other?" It has to check.

The view of the tape is LOCAL. One cell at a time. There is no primitive way to see global structure. The machine can COMPUTE global structure, but computing it and seeing it are different things. To compute that a list is sorted, the machine has to execute a sorting-detection algorithm that touches every element. It pays in time and space for every structural fact it learns.

The same story holds for any model whose state space projects to (`mem`, `regs`, `pc`). The transition function sees the projection. Whatever structural commitment the program is exploiting, a known decomposition, a checked invariant, an asserted morphism, has to be either re-derived from scratch on every step or carried in the data implicitly, with no auditing. There is no place in the state where the entitlement to use that structure lives.

I spent months staring at this before it clicked. The Turing machine is not broken. It is blind by design, and so is everything that compiles down to its observable state. It is like trying to find your way through a maze by only ever looking at the floor tile you are standing on. You *might* do it, if the maze has an exit and if your search strategy eventually reaches it. Some mazes have no exit, and some search strategies loop forever. That is what the halting problem says, in the version of it I had to learn: there is no general algorithm that can look at an arbitrary program and decide whether it eventually stops or runs forever. Turing proved this in 1936. The maze-tile reader has the same disability: it cannot in general tell from the floor tile whether it is making progress or going in circles. But if the maze does terminate, you are paying for every wrong turn and every part already explored. Someone with a map does not pay that cost.

The Thiele Machine does not magically give the map. "The map" here is a concrete thing: a certified partition graph with proven structural properties, a checked decomposition into modules, a validated formula, an asserted morphism with provenance. The Thiele Machine represents the specific moment a computation becomes entitled to use one of these, and it makes that entitlement auditable: there is always a μ-ledger entry showing when the certification was paid for. The map is not free.

3.5 Classical complexity ignores structural cost

Standard complexity theory (the part of computer science that classifies problems by how hard they are to solve) measures two things: time (number of steps the machine takes) and space (number of cells or registers it uses). It says nothing about the cost of knowing that the input has structure. This is true whether the underlying model is a Turing machine, a RAM machine (a model with random-access numbered memory

slots, closer to a real computer than a tape), a lambda-calculus reducer, or any other classical substrate. The cost model only counts steps.

Consider the satisfiability problem (SAT): given a logical formula ϕ over n boolean variables, is there an assignment that makes it true? In the worst case, a blind machine has to check all 2^n assignments. (Here 2^n means 2 raised to the power n: 2 times itself n times. For 20 variables it is about a million, for 30 it is a billion, for 60 the universe runs out of time before a normal computer finishes.) That is what makes SAT hard.

Now suppose ϕ decomposes cleanly: $\phi = \phi_1 \wedge \phi_2$, where ϕ_1 and ϕ_2 share no variables. You can solve ϕ_1 on its own (2^{n_1} work, where n_1 is the number of variables in ϕ_1) and ϕ_2 on its own (2^{n_2} work, where n_2 is the number in ϕ_2). Total work: $2^{n_1} + 2^{n_2}$ instead of $2^{n_1+n_2}$. The first sum stays manageable as n_1 and n_2 grow; the product in the exponent grows much faster. That is an exponential improvement: a small change in the input gets a wildly large change in the cost.

But that improvement only applies if you *know* the decomposition exists. And no classical computational model assigns a cost to knowing it. The structure is either there or it isn't, and you either see it or you spend time finding it, but the act of certifying "I know this decomposition is valid" is not tracked, not paid for, not audited, not in the Turing model, not in any of its computational equivalents.

The Thiele Machine says: that act belongs inside the computation. Here is the structural object. Here is the checked transition that makes the object admissible. Here is where the cost shows up in the trace. Here is how to verify that the cost was actually paid. This is the structural-entitlement reading of No Free Insight, and via `universal_nfi_any_substrate` it lifts to any computational system that can be packaged as a `CertificationSystem`.

3.6 The subsumption theorems, in full

The Section 1 framing claimed that every classical computer IS a Thiele Machine, operating in the degenerate fragment where the structural axis stays dormant. The proofs of that claim are four theorems in the Coq kernel. Their statements and proof sketches are below, in math notation, so they can be checked from this document without leaving it.

Theorem 3.1 (D2 faithfulness, `coq/kernel/foundation/TuringClassicalEmbedding.v:106`)**.** *For every classical program* prog *(a sequence of* `vm_instruction` *values each satisfying* `is_classical_opcode`*) and every starting state s_0, let s_n be the state after running* prog *on s_0. Then (a)* `shadow_proj`(s_n) *extracts exactly the six classical fields of s_n (*`vm_regs`, `vm_mem`, `vm_pc`, `vm_mu`, `vm_err`, `vm_certified`*); and (b) the structural layer is preserved: s_n.`vm_graph` $= s_0$.`vm_graph`, s_n.`vm_csrs.csr_cert_addr` $= s_0$.`vm_csrs.csr_cert_addr`, and s_n.`vm_certified` $= s_0$.`vm_certified`.*

Proof. Part (a) is unfolding `shadow_proj`: by its definition it extracts exactly those six fields; reflexivity closes it. Part (b) is D3 conservativity (next theorem) iterated over

the program. □ □

Plain reading. Classical programs leave the structural axis alone. The shadow you see equals the full state's classical fields, and the graph and cert channels stay at whatever they were at the start.

Theorem 3.2 (D3 conservativity, `coq/kernel/foundation/ClassicalConservativity.v:200`)**.** *If* `is_classical_opcode`(i) = true *and* $s' =$ `vm_apply`(s, i)*, then* s'.`vm_graph` = s.`vm_graph`*,* s'.`vm_csrs.csr_cert_addr` = s.`vm_csrs.csr_cert_addr`*, and* s'.`vm_certified` = s.`vm_certified`*. By induction over the program list, classical programs preserve all three.*

Proof. Case analysis on the instruction i. The predicate `is_classical_opcode`(i) = true holds exactly for the compute, memory, control-flow, ordinary I/O, heap, and `MORPH_GET` / `TENSOR_GET` opcodes — the ones whose `vm_apply` rule mutates only `vm_regs`, `vm_mem`, `vm_pc`, `vm_err`, `vm_mu`, `vm_logic_acc`, `vm_mu_tensor`, or `vm_mstatus`. The graph-mutating opcodes (`PNEW`, `PSPLIT`, `PMERGE`, `MORPH`, `COMPOSE`, `MORPH_ID`, `MORPH_DELETE`, `MORPH_TENSOR`), the cert-channel opcodes (`LASSERT`, `LJOIN`, `EMIT`, `REVEAL`, `MORPH_ASSERT`, `CERTIFY`, `CHSH_LASSERT`), and the witness-channel opcode (`CHSH_TRIAL`, `PDISCOVER`) return `false` from `is_classical_opcode`, so they are excluded by the premise. For each of the included opcodes, the apply rule by direct inspection leaves `vm_graph`, `csr_cert_addr`, and `vm_certified` fixed. Iterating over the program list by induction closes the trace-level preservation. □ □

Plain reading. The classical opcodes are defined by what they don't touch. They mutate the classical-shadow part of the state and nothing else. A program built only out of them cannot disturb the structural axis, even if you wanted it to. The structural axis is dormant under classical traces by construction, not by accident.

Theorem 3.3 (Thiele simulates Turing, `coq/kernel/foundation/ProperSubsumption.v:172`)**.** *Define the lift function* `lift_config`$(c) := \{$ `th_tm_config` $= c$; `th_mu` $= 0\}$*, mapping any Turing-machine configuration to a Thiele configuration with* $\mu = 0$ *and no accumulated cost. For every fuel* $n \in \mathbb{N}$*, every transition table* δ*, and every TM configuration* c*:*

$$(\texttt{thiele\_run}\ n\ \delta\ (\texttt{lift\_config}\ c)).\texttt{th\_tm\_config} = \texttt{tm\_run}\ n\ \delta\ c.$$

Proof. Induction on the fuel n.

Base ($n = 0$)*.* Both `thiele_run` 0 and `tm_run` 0 return their inputs unchanged. Reflexivity.

Step ($n + 1$)*.* Unfold one step on both sides. Let $c' =$ `tm_step` δ c when defined. By the definition of `thiele_step` on a lifted configuration, the Thiele step's TM-portion

is exactly `tm_step` δ c (the structural fields are not consulted, and the lifted μ stays at 0 because TM steps are classical). Apply the induction hypothesis to the next n steps starting from `lift_config`(c'). □ □

Plain reading. Take any Turing machine. Lift it to Thiele (set $\mu = 0$, empty the structural axis). Run `thiele_run` on the lifted state. At every step, the underlying TM portion of the Thiele state matches what the standalone TM would have done one step further. By induction on fuel, the whole trace matches. The Turing machine inside the lifted Thiele state IS doing the TM's computation. Same tape, same state, same answer.

Theorem 3.4 (Degenerate projection theorem (four-part), `coq/kernel/foundation/TuringClassicalEmbedding.v:524`)**.** *The following four propositions hold simultaneously:*

1. Faithful embedding. *For every fuel n, transition table δ, and TM configuration c:*

$$(\mathit{thiele\_run}\ n\ \delta\ (\mathit{lift\_config}\ c)).\mathit{th\_tm\_config} = \mathit{tm\_run}\ n\ \delta\ c.$$

2. shadow_proj is the quotient map. *For all Thiele states s_1, s_2:*

$$\mathit{shadow\_proj}(s_1) = \mathit{shadow\_proj}(s_2) \iff \mathit{eq\_on\_classical\_shadow}(s_1, s_2).$$

3. Classical traces respect the quotient. *For every classical program* prog *and every two Thiele states s_1, s_2 that agree on the classical shadow:*

$$\mathit{shadow\_proj}(\mathit{run}(\text{prog}, s_1)) = \mathit{shadow\_proj}(\mathit{run}(\text{prog}, s_2)).$$

4. The projection is strictly lossy. *There exist Thiele states s_1, s_2 with $\mathit{shadow\_proj}(s_1) = \mathit{shadow\_proj}(s_2)$ and $s_1 \neq s_2$.*

Proof. Each part is a separately-established lemma stitched into one conjunction.

- Part (1) is `thiele_simulates_turing` (preceding theorem).
- Part (2) is by definition: `shadow_proj` extracts a six-tuple of classical fields, and two Thiele states have equal projections precisely when those six fields agree. The Coq witness is `shadow_proj_kernel_is_eq_on_classical_shadow`.
- Part (3) follows from D2 and D3: the classical-shadow fields after running the program depend only on the classical-shadow fields before, so equal shadows in produce equal shadows out. The Coq witness is `D2_classical_shadow_preserved`.
- Part (4) is `shadow_strictly_lossy`, witnessed by the Categorical Separation construction in Section 12: s_1 has one identity morphism in `pg_morphisms`, s_2 has none; six classical fields agree; the morphism lists do not.

The conjunction of the four pieces is the theorem. □ □

Plain reading. Four parts about the same projection. Part 1: every TM lifts faithfully into Thiele — same computation, in a Thiele state with $\mu = 0$ and no structural goods. Part 2: the projection from Thiele back to classical IS the classical-equivalence quotient — two Thiele states have the same shadow exactly when they agree on the six classical fields. Part 3: classical programs commute with the projection — you can project before running or after, you get the same answer. Part 4: the projection genuinely loses information — there are Thiele states that look identical from the classical side but differ on the structural axis. Put together, classical computation is exactly the image of Thiele computation under the structural-axis projection, and the projection drops real information.

4 What structure is

"Structure" is one of those words everyone uses and nobody defines. The whole machine rests on what it means, so here is the definition.

4.1 Partitions

Start with the simplest case. Take any collection of things (a set) and divide it into labeled buckets that do not overlap and cover everything. That is a partition.

Formally: a partition of a set S is a collection of non-empty subsets $S_1, S_2, \ldots, S_k$ that are disjoint (no two of them share an element) and whose union is the whole of S (every element of S is in exactly one subset). I will write the union $S_1 \cup S_2 \cup \cdots \cup S_k = S$, using the $\cup$ symbol from Section 2 for set union. Each S_i is called a module or block or cell, depending on the context.

Now here is the key thing: the partition IS information. Knowing that element x belongs to block S_1 and element y belongs to block S_2 tells you something real. It tells you x and y are in different groups. If you did not know the partition, you did not know that. If you know the partition, you can look it up immediately, in constant time (a complexity-theory phrase meaning the lookup cost does not grow with the input size).

In a computational context, partitioning the memory or register state means saying: "These addresses belong to module A, those addresses belong to module B, and these two modules are independent." That is structural knowledge. A Turing machine has no way to represent this directly. It has to recompute independence from scratch every time it needs to know it. The Thiele Machine stores the partition as part of its state, audits the cost of building it, and enforces that building it was paid for.

4.2 Axioms on modules

A partition alone is just a labeling. What makes it structural knowledge is when you attach *constraints* to the modules.

First, two definitions I need.

A **predicate** is a statement about a thing that is either true or false. "$x > 5$" is a predicate about a number x. "This memory region contains a sorted list" is a predicate about a memory region. Predicates are the basic unit of making a claim about something.

A **logical formula** over boolean variables is an expression built from variables (each either true or false) connected by AND ($\wedge$), OR ($\vee$), and NOT ($\neg$). Example: $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3)$: this formula is true when either x_1 or x_2 is true AND when either x_1 is false or x_3 is true. A satisfying assignment is a specific choice of true/false for each variable that makes the whole formula evaluate to true. An unsatisfying assignment makes it false.

With those in hand: you attach constraints to the modules. For example: "Module A's memory region satisfies the predicate Φ_A," where capital Greek Φ_A is the name I am giving to the predicate I want to attach (mathematicians often use Greek letters for predicates and formulas), and where Φ_A might say something concrete like "all values in this region are between 0 and 100," or "this region encodes a satisfying assignment to the formula stored in module B," or "this region is sorted in increasing order."

These constraints are called axioms in this document. "Axiom" has two meanings: the everyday math sense (a starting truth you accept without proof) and the sense I'm using here (a checked constraint a Thiele module carries). I mean the second sense throughout this document. A Thiele axiom is not asserted; it is checked, and the check has a price.

I will come back to the exact encoding in Section 10, but the rough picture is: the machine reads a formula from memory, checks it against a certificate (a witness that the formula is satisfiable, and also a witness that it is not a tautology), and either accepts or traps. The piece of hardware that does that checking is called the Logic Engine. It is a small finite-state machine built into the chip (a circuit that always sits in one of a fixed list of internal states, transitioning between them based on inputs): a dedicated unit that walks through the formula word by word and verifies the witnesses without calling out to any external constraint solver. If validation succeeds, the checked constraint package can be used by the later certification path. If validation fails, the machine traps (Section 2: routes execution to a fixed error address) and sets an error flag. The cost is charged either way.

The crucial point: you cannot add an axiom to a module for free. Adding an axiom means the machine paid to certify it. The axiom is evidence, not assertion.

4.3 Structural gain

A trace has "structural gain" if it starts in a state where some structural fact is not certified and ends in a state where it is. The classic example: a trace that starts with an uncertified partition and ends with a certified satisfying-assignment witness has

undergone structural gain.

The Thiele Machine's core question is: where, in the trace, did the structural gain enter? Not merely "did the gain happen." That is visible from inspecting the start and end states. The question is which instruction, at which step, introduced the certified structural knowledge.

The answer is always: an instruction that costs $\mu \geq 1$. That is No Free Insight.

5 Structural entitlement

The center of the project is not just "certification has a price." That is true, but it is the first conservation law, not the whole picture.

The deeper object is *structural entitlement*. A computation has structural entitlement when the state does not merely contain data, but contains a structural fact that later steps are allowed to use as if it were already established. The word I use for "allowed to use" is *admissible*: the machine has earned the right to use the fact, in the sense that the trace contains the cost-positive instruction that paid for the right, and the proof object that establishes the right is on hand. Admissible is not just "true." Admissible is "tracked and paid for."

Several kinds of object can be admissible in this sense: a partition (Section 4), a decomposition (a split of a problem into independent parts), a morphism (Section 11), a constraint (Section 4), a witness (a value that proves the existence of something), or a narrowed feasible set (defined below). The difference between an admissible fact and raw data matters. A raw bit pattern can be copied. A certified decomposition can justify solving two smaller problems instead of one large one. A raw morphism ID can be forged in a register: you can write 42 into a register and call it "the ID of some morphism." An asserted morphism in the graph has provenance (Section 2), meaning the machine's execution history is the chain of evidence: a specific sequence of MORPH and COMPOSE instructions built the morphism, and MORPH_ASSERT then committed it. A formula sitting in memory is text. A checked formula with a satisfying assignment and a falsifying assignment is admissible narrowing: the two witnesses prove the formula is satisfiable and not a tautology, which rules out at least one possible machine state.

Definition 5.1 (Structural entitlement)**.** A state has structural entitlement to a claim P when three things are present:

- a structural object in the machine state, such as a partition, morphism, witness counter, or formula package;
- a checked relation between that object and the claim P;
- a trace-visible certification or assertion event that makes later use of P admissible.

Without the third part, the object may exist as raw data, but the computation has not

earned the right to use it as certified structure.

The pieces connect as follows.

5.1 The precise vocabulary

Every term below is an exact Coq definition, not a metaphor. Every theorem that follows has been machine-verified. I'm not stamping each one individually with "machine-checked", assume all of them are.

Feasible set. A feasible set Ω is a concrete, finite list of Thiele Machine states. In the Coq code, the type is literally `list VMState`. The interpretation: Ω is the set of machine states the computation might be in, given everything that has been checked and certified so far. Before any structural knowledge is certified, Ω can contain many states. After certifying a constraint, Ω shrinks to only the states consistent with that constraint. A feasible set is not an abstract notion of "possible worlds." It is a list of actual machine states.

Prior and posterior. The *prior* Ω is the feasible set before an observation or structural certification event. The *posterior* Ω' is the feasible set after. The posterior is always a subset of the prior: $\Omega' \subseteq \Omega$. If the posterior is a *strict* subset ($|\Omega'| < |\Omega|$), some states have been ruled out. The observation eliminated them. The picture is: you start with a universe of possibilities. You run an experiment. You learn which outcomes are consistent with the result. The states that are no longer consistent get thrown out. What remains is the posterior. Here, "learning something" means the machine certified a structural claim that is false in those eliminated states, so they can no longer be the actual machine state.

Distinguishing observation. A distinguishing observation is a function that returns different values on the eliminated states (in Ω but not in Ω') versus the remaining states (in Ω'). Formally: there exists at least one eliminated state $s_w \in \Omega \setminus \Omega'$ such that the observation function outputs a value on s_w that no state in Ω' produces. The observation is the "test" that reveals the structure. If you cannot name such a function, you cannot show that the posterior genuinely excludes something the prior allowed.

Strictly stronger predicate. A predicate P_1 is strictly stronger than P_2 if P_1 accepts a strict subset of what P_2 accepts. Formally in Coq: $P_1 \leq P_2$ (whenever P_1 is true, P_2 is true too) AND there exists an observation where P_1 is false but P_2 is true. The word "stronger" means more restrictive: P_1 makes a harder claim, rules out more states as invalid. If P_2 is "the state is in Ω" and P_1 is "the state is in $\Omega' \subsetneq \Omega$", then P_1 is strictly stronger, it demands more, and anything that satisfies P_1 also satisfies P_2 but not vice versa. This is the opposite of everyday use of "stronger": a stronger claim in this logical sense is harder to satisfy, not more impressive-sounding.

Structure-addition event. A structure-addition event is a single execu-

tion step where the assertion register `csr_cert_addr` transitions from 0 to nonzero. Precisely: s.`vm_csrs.csr_cert_addr` $= 0$ before the step and (`vm_apply` $s\ i$).`vm_csrs.csr_cert_addr` $\neq 0$ after. This only happens via a successful MORPH_ASSERT with a non-empty property string, which stores the ASCII checksum of that property in `csr_cert_addr`. Not "structural knowledge was acquired" in some vague sense. Precisely: this specific field flipped from zero, at this step, by MORPH_ASSERT asserting a non-empty property against an existing morphism.

Supra-certification (`has_supra_cert`). The machine has reached supra-certification when `csr_cert_addr` $\neq 0$. The name distinguishes it from `vm_certified` (the top-level flag set by the CERTIFY opcode). Supra-certification specifically tracks the MORPH_ASSERT channel: has a morphism property been asserted and its checksum registered? A machine can have `vm_certified = true` without `has_supra_cert` (if it used only CERTIFY and not MORPH_ASSERT, since CERTIFY never touches `csr_cert_addr`). Both are positive-cost certification channels; they record different things.

Decision tree. A decision tree is an explicit witness to *how* the trace's observations branch over the state space. At each node, the tree records: here is an observation, these states passed it, these states failed it. The leaves record which states ended up in the posterior. The tree is a proof object, not a runtime structure: it exists to show that the transition from prior to posterior was the result of a definite sequence of binary tests rather than a magic jump. The Coq requires you to construct it explicitly as a witness; you cannot claim a shortcut is sound without handing over the actual branching structure that produced the posterior.

Posterior-representative reduction. Given a prior state $s \in \Omega$, a posterior-representative reduction provides a function mapping s to a representative $s' \in \Omega'$. This proves that every prior state corresponds to something in the posterior. The purpose: it shows the posterior was not reached by arbitrarily discarding states, but by a coherent mapping where each prior state is "accounted for" by a posterior representative. Without this, the narrowing could in principle be fake: you could claim the posterior is small by just deleting states without any principled connection to the prior.
Theorem 5.1 (Classical blindness as quotient)**.** *The four-field projection from Thiele states to (PC, μ, registers, memory) is a many-to-one map: two different Thiele states can land on the same projection. (The Section 2 word for this is a quotient: the map identifies any two Thiele states that agree on the four kept fields.) Specifically, the projection collapses states that differ in certification, witness counters, or morphism graph into the same image. The slightly richer six-field projection* `shadow_proj`*, which keeps* `vm_certified` *as a fifth field, still forgets the morphism graph. No function of either projection alone can separate the morphism-witness states.*

Proof. Many-to-one is exhibited by the Categorical-Separation witness pair in Sec-

tion 12: s_1 with `pg_morphisms` $= [(0, \mathrm{id})]$, s_2 with `pg_morphisms` $= []$, and both s_i having `vm_regs` $= []$, `vm_mem` $= []$, `vm_pc` $= 0$, `vm_mu` $= 0$, `vm_err` $=$ false, `vm_certified` $=$ false. Both four-field projection and six-field `shadow_proj` map s_1 and s_2 to the same image. So $s_1 \neq s_2$ with equal projections under either map. "No function of the projection alone can separate them" is the contrapositive: any classification depending only on the projection-image agrees on equal-image inputs, hence agrees on s_1 and s_2. □ □

Plain reading. Same witness as Categorical Separation, used in the other direction. Two Thiele states with identical projections (under any projection that drops the morphism graph), but they are demonstrably different states. So the projection is provably many-to-one. A function defined on the projection output can't tell them apart, because the projection has already collapsed them.

If your machine state only records registers, memory, PC, error bit, and maybe μ, you have already thrown away part of what matters. You have collapsed states that differ on the inside into a single raw state that looks the same from the outside. That is not a philosophical complaint. It is a theorem about a projection map.

Theorem 5.2 (μ-ledger necessity, complete independence classification)**.** *For two pieces of state X and Y, I will write $X \perp Y$ to mean: knowing Y tells you nothing about X. (I am redefining the $\perp$ symbol here for this section. It is sometimes used for orthogonality in other math contexts; here it means independence.) Concretely, $X \perp Y$ says there exist two reachable machine states with identical Y-values but different X-values, and this holds starting from every state the machine can reach (not just from one chosen starting point). Five such independence results hold simultaneously: (1) $\mu \perp (\mathtt{mem}, \mathtt{regs}, \mathtt{pc})$; (2) $\mathtt{certified} \perp (\mathtt{mem}, \mathtt{regs}, \mathtt{pc})$; (3) $\mathtt{certified} \perp (\mathtt{mem}, \mathtt{regs}, \mathtt{pc}, \mu)$; (4) $\mu \perp (\mathtt{mem}, \mathtt{regs}, \mathtt{pc}, \mathtt{certified})$; (5) $\mathtt{vm\_graph} \perp (\mathtt{mem}, \mathtt{regs}, \mathtt{pc}, \mu, \mathtt{certified})$. The three non-classical components (μ, `vm_certified`, `vm_graph`) are each impossible to recover from any combination of the others plus classical state. The projection $(\mathtt{mem}, \mathtt{regs}, \mathtt{pc}, \mu, \mathtt{certified})$ is the minimal complete extension for the ledger semantics: dropping either μ or `vm_certified` loses the ability to determine the other. The separation holds starting from every reachable machine state, and no program prefix of any length collapses it.*

Proof. Each independence claim is established by exhibiting two reachable Thiele states that agree on the named projection-tuple but differ on the named coordinate.

(5) $\mathtt{vm\_graph} \perp (\mathtt{mem}, \mathtt{regs}, \mathtt{pc}, \mu, \mathtt{certified})$. The Categorical-Separation witness pair: s_1 has `pg_morphisms` $= [(0, \mathrm{id})]$, s_2 has `pg_morphisms` $= []$, and the five classical fields agree across both. Both are reachable from `init_state`: s_2 by the empty program, s_1 by a single `MORPH_ID` instruction with $\delta = 0$ (which leaves μ, regs, mem, pc, certified untouched but adds an identity morphism). The construction is parametric in the starting state: from any reachable s_0 you can append a $\delta = 0$

`MORPH_ID` to get a graph-different but classical-shadow-equal sibling.

(1) $\mu \perp (\mathit{mem}, \mathit{regs}, \mathit{pc})$. Run a $\delta = 0$ no-op like `ADD 0 0 0 0` versus an `EMIT` event with $\delta = 0$ producing matching register, memory, pc effects but $\mu = 9$ versus $\mu = 0$. (The Coq witness in `coq/kernel/nfi/NecessityAbstract.v` produces this construction explicitly.)

(2) $\mathit{certified} \perp (\mathit{mem}, \mathit{regs}, \mathit{pc})$. Run `CERTIFY 0` in one branch, omit it in the other; tune the other branch with a $\mu = 1$ no-op (a single-bit `EMIT` then pad to match pc), giving two states with the same $(\mathtt{mem}, \mathtt{regs}, \mathtt{pc})$ but `vm_certified` differing.

(3) $\mathit{certified} \perp (\mathit{mem}, \mathit{regs}, \mathit{pc}, \mu)$. Strengthening of (2): the matching branch must also match μ. Achieved by selecting cost-equivalent instructions that produce no other distinguishable effect on the classical fields.

(4) $\mu \perp (\mathit{mem}, \mathit{regs}, \mathit{pc}, \mathit{certified})$. Dual of (3): two reachable states with identical $(\mathtt{mem}, \mathtt{regs}, \mathtt{pc}, \mathtt{certified})$ but different μ. Run distinct positive-cost instructions whose classical-shadow effect is identical.

The universal quantifier "from every reachable starting state" is structural: each construction is parametric in the starting state because the no-op and cost-positive instructions used as padding are universally available; no program prefix locks the machine out of building the required pair.

Minimality. Dropping μ from the projection collapses witnesses (1), (4) into equal classes (the pair witnessing μ independence is forgotten); dropping `certified` collapses witnesses (2), (3) similarly. Each dropped field corresponds to a forgotten independence axis. The full five-field projection is the smallest one for which all three structural components (μ, `certified`, `vm_graph`) are each independent of the rest. □ □

Plain reading. Five independence claims, one witness per claim, all parametric in the starting state. The headline case (5) is the categorical-separation pair: same six classical fields, different morphism graphs. The other four are pairs of states reachable by short programs that match on the named tuple but disagree on the named coordinate. Minimality says: drop any one field and you've broken one of the five witnesses. So the five-field projection $(\mathtt{mem}, \mathtt{regs}, \mathtt{pc}, \mu, \mathtt{certified})$ is the smallest one that keeps all three structural axes (μ, certified, vm_graph) independent of the rest. None of those three is a function of the others plus the classical state.

The ledger is not just a counter someone forgot to derive. There are explicit witness states with identical Turing/RAM-visible state and incompatible receipts. Not just from one starting point, but from every state the machine can reach. Cost, certification, and graph structure are three independent bookkeeping dimensions. A model that drops any of them cannot determine the full structural semantics internally.

Theorem 5.3 (Latent μ: the cost is intrinsic to computation)**.** *The total μ accumulated by running any instruction sequence equals the sum of the per-instruction costs, independent of every other aspect of machine state (the partition graph, the registers, the memory, the program counter, the certified flag, even the starting value of μ). Formally:*

$$\mu_{final} = \mu_{initial} + \sum_{i \in prog} \texttt{instruction\_cost}(i).$$

The right side uses the summation symbol from Section 2. It reads: add up `instruction_cost(i)` *for every instruction i in the program. The result does not depend on the starting state at all. Two machines running the same program from any two starting states accumulate the same additional μ; the change in μ depends only on the program, not on the starting state. Any accounting system that assigns zero to the starting state and increases by exactly* `instruction_cost` *at each step ends up with exactly* `trace_total_cost` *on every reachable state; there is no alternative honest accounting. Any program containing a positive-cost instruction accumulates strictly positive μ; there is no free computation.*

Proof. Induction on *prog*, applying μ-Conservation at each step.

Base (prog $= []$). $\mu_{\text{final}} = \mu_{\text{initial}}$ and $\sum_{i \in []} \texttt{instruction\_cost}(i) = 0$. The equation holds trivially.

Step (prog $= i ::$ rest). Let $s_1 = \texttt{vm\_apply}(s_0, i)$. By μ-Conservation,

$$s_1.\mu = s_0.\mu + \texttt{instruction\_cost}(i).$$

Apply the induction hypothesis to rest starting from s_1:

$$\mu_{\text{final}} = s_1.\mu + \sum_{j \in \text{rest}} \texttt{instruction\_cost}(j).$$

Substituting $s_1.\mu$,

$$\mu_{\text{final}} = s_0.\mu + \texttt{instruction\_cost}(i) + \sum_{j \in \text{rest}} \texttt{instruction\_cost}(j) = \mu_{\text{initial}} + \sum_{i \in prog} \texttt{instructio}$$

The expression on the right depends only on *prog* and μ_{initial}, not on any other field of the starting state. "Any positive-cost instruction $\Rightarrow$ strictly positive $\Delta\mu$" follows because each term is non-negative and the named term is ≥ 1. □ □

Plain reading. The change in μ across a run is just the sum of the per-instruction costs. μ-Conservation does each step; induction stitches them together. Two different starting states with the same starting μ end up at the same final μ on the same program. The starting graph, registers, memory, none of it matters for the μ count. Only the program does. The cost is a property of the program, not of the machine running it.

The μ-cost is not a feature of the Thiele Machine. It is a property of the instruction sequence itself. Replace the Thiele Machine's μ-counter with any other honest account-

ing and you get the same number. A Turing machine running the same program paid exactly this cost on every execution. It simply had no register to store the receipt. The ledger does not create the cost. It reveals what was always there.

Theorem 5.4 (Feasible narrowing becomes predicate strengthening)**.** *If a posterior feasible set Ω' is a strict subset of a prior feasible set Ω, and there is an observation that distinguishes a removed state from all posterior states, then the membership predicate for Ω' is strictly stronger than the membership predicate for Ω. (The membership predicate for Ω is the function that takes a state and returns true exactly when the state is in Ω.)*

Proof. Define $P_\Omega(s) := (s \in \Omega)$ and $P_{\Omega'}(s) := (s \in \Omega')$. "Strictly stronger" has two parts.

(a) $P_{\Omega'}$ implies P_Ω. If $P_{\Omega'}(s)$ holds, then $s \in \Omega'$, and since $\Omega' \subseteq \Omega$, $s \in \Omega$, so $P_\Omega(s)$ holds.

(b) P_Ω does not imply $P_{\Omega'}$. Since $\Omega' \subsetneq \Omega$, there exists $s_w \in \Omega \setminus \Omega'$. So $P_\Omega(s_w)$ is true and $P_{\Omega'}(s_w)$ is false. The distinguishing observation hypothesis gives an explicit Boolean function that witnesses s_w, rather than merely asserting its existence; the bare strictly-stronger conclusion above only requires the existence. □ □

Plain reading. The membership predicate of a smaller set is logically stronger than the membership predicate of a bigger set, by set inclusion. The distinguishing observation gives you a concrete witness state for the strictness; without it you'd only have an existential.

The search space getting smaller and the computation accepting a stronger claim are the same event seen from two angles. The stronger predicate is not invented for the theorem; it falls out of set membership in the smaller set through the distinguishing observation. Same event. Different side of the same equation.

Theorem 5.5 (Strengthening requires structure addition)**.** *If a trace starts with no certification address, ends certified for a strictly stronger predicate, and the certification is tied to the machine's supra-certification channel, then the run contains a structure-addition event.*

Proof. "No certification address at start" is $s_0.\texttt{vm\_csrs.csr\_cert\_addr} = 0$. "Certified for the supra-cert channel at end" is $s_n.\texttt{vm\_csrs.csr\_cert\_addr} \neq 0$ (the definition of `has_supra_cert`). The trace runs from s_0 to s_n, executing some sequence of instructions. Since the field `csr_cert_addr` starts at 0 and ends nonzero, there exists some step k in the trace where it flipped from 0 to nonzero. That step is, by the definition of a structure-addition event in Section 5, a structure-addition event. The strictly-stronger-predicate hypothesis names what the certification is *for*; it does not affect the existence of the flip. □ □

Plain reading. Cert address starts at 0, ends nonzero. So somewhere along the run, it flipped. That flip IS what we defined "structure-addition event" to be in Section 5.

The proof is just observing: if the value went from 0 to nonzero across a finite trace, the change happened on some specific step.

This is the formal answer to "where did the new admissible structure enter?" It did not appear from arithmetic alone. It entered through a trace-visible structure-addition event.

Theorem 5.6 (Universal certification cost)**.** *For any abstract computational system with a set of states, a set of instructions, a step rule, a cost function, and a certification predicate, if every single-step transition from uncertified to certified costs at least 1, then every trace that starts uncertified and ends certified has total cost at least 1.*

Proof. This is the same theorem as the Universal No Free Certification result in Section 8, restated here at the structural-entitlement layer. The proof is the trace-induction argument given there: empty trace cannot flip the cert predicate; non-empty trace either flips it at the first step (single-step A2 gives ≥ 1) or flips it later (induction hypothesis gives ≥ 1 on the rest). Either case yields total cost ≥ 1. □ □

Plain reading. Same theorem as the NoFI section, named at this layer because structural entitlement needs it. The certification cost is forced wherever A2 is accepted.

This theorem is important because it is not about the Thiele ISA. It says the conservation law is forced for any system once the system has a certification predicate and a cost rule that charges for certification transitions. If a model lets the certification predicate flip at zero cost, that model has free certification by construction. If a model refuses to represent certification at all, that model cannot talk about structural entitlement internally; it can only smuggle that entitlement in from outside.

Theorem 5.7 (Structural entitlement representation)**.** *For any explicit witness of structural entitlement consisting of:*

- *a strict feasible-set narrowing $\Omega' \subsetneq \Omega$;*
- *an observation function that distinguishes at least one eliminated witness state from all posterior states;*
- *a certified posterior predicate built from membership in Ω';*
- *a decision tree* realized by the trace*: meaning the trace's actual sequence of observations matches the tree's branching structure, each branch in the tree corresponds to a test the execution actually performed; and*
- *a posterior-representative reduction tying the prior states to posterior representatives,*

the posterior predicate is strictly stronger than the prior predicate, the trace contains a structure-addition event, and the size of the narrowing is bounded by the paid ledger:

$$\log_2^{\uparrow} |\Omega| - \log_2^{\uparrow} |\Omega'| \leq \Delta\mu.$$

Here $|\Omega|$ is the size of the prior feasible set (the number of states in it; the same $|\cdot|$ notation from Section 2), and $|\Omega'|$ is the size of the posterior. $\log_2^{\uparrow}$ is the ceiling of the base-2 logarithm: take $\log_2$ of the argument, round up to the nearest integer. It counts how many yes-or-no questions you need to identify one state in the set. (For a 1000-state set you need at least 10 yes-or-no questions, because 10 binary questions distinguish $2^{10} = 1024$ items. The ceiling rounds up so the bound is conservative; rounding up never makes the inequality easier to satisfy.) $\Delta\mu$ is the change in μ across the trace: final μ minus initial μ.

Proof. Three conclusions, chained.

(i) Strictly stronger predicate. The strict-narrowing $\Omega' \subsetneq \Omega$ and the distinguishing observation are the hypotheses of the Feasible-narrowing-becomes-predicate-strengthening theorem above. Apply it to conclude that the membership predicate of Ω' is strictly stronger than that of Ω.

(ii) Structure-addition event. The certified posterior predicate is tied (by hypothesis) to the supra-cert channel, so the run ends with `csr_cert_addr` $\neq 0$. The initial-uncertified hypothesis on the witness gives `csr_cert_addr` $= 0$ at start. Apply the Strengthening-requires-structure-addition theorem to conclude the run contains a structure-addition event at some step k^*.

(iii) The $\log_2^{\uparrow}$ bound. The realized decision tree T is a binary tree whose leaves are tagged with posterior states. Each prior state in Ω is mapped (by the realized-by-the-trace hypothesis) to a leaf via the trace's actual sequence of observations. For all of Ω to land in at most $|\Omega'|$ leaves, T must have at least $|\Omega|/|\Omega'|$ effective leaf-classes accessible at the root level, which forces tree depth

$$d \;\geq\; \log_2^{\uparrow}|\Omega| - \log_2^{\uparrow}|\Omega'|.$$

Each internal node of T corresponds to one observation in the trace that is certified (the realized-by-the-trace condition pins each node to a Boolean observation function executed by the run, and the certified-posterior hypothesis ties those observations to cert-channel activations). Each certified observation is a cert-setter step in the Thiele Machine, so each contributes at least 1 to μ by the cost-discipline rule for cert-setters (Section 7). The total μ-cost of the cert-setter steps along the trace is therefore $\geq d$, and is bounded above by $\Delta\mu$ (the total trace cost). Chain the inequalities:

$$\log_2^{\uparrow}|\Omega| - \log_2^{\uparrow}|\Omega'| \;\leq\; d \;\leq\; (\text{number of cert-setter steps}) \;\leq\; \Delta\mu. \quad \square$$

$\square$

Plain reading. Three conclusions, three steps. Conclusion 1 is just the predicate-strengthening theorem applied to the strict-narrowing and distinguishing-observation hypotheses. Conclusion 2 is just the structure-addition theorem applied to the trace's

start-uncertified, end-certified condition. Conclusion 3 is the entropy bound: a binary tree that compresses $|\Omega|$ states into $|\Omega'|$ leaf-classes has depth at least $\lceil \log_2(|\Omega|/|\Omega'|) \rceil$, which is bounded by $\log_2^{\uparrow} |\Omega| - \log_2^{\uparrow} |\Omega'|$ on the underestimate side. Each tree node is a certified observation, each certified observation pays at least 1 in μ, so the tree depth is bounded by $\Delta\mu$. You don't get to compress for less than you paid.

This theorem does not quantify over the English word "structure." It defines the witness class: strict narrowing, distinguishability, certification, and an explicit tree/fiber representative for the reduction. For every member of that class, structural entitlement is not metadata and not a slogan. It is predicate strengthening, it requires structure addition, and its quantified narrowing is charged to μ.

Theorem 5.8 (Every sound structural shortcut lands here)**.** *A* `SoundStructuralShortcut` *is a package containing exactly the receipts a shortcut must provide to count as sound: a prior feasible set, a posterior feasible set, strict narrowing, a distinguishing observation, a certified posterior predicate, a realized decision tree, and a posterior-representative reduction. For every such package, the structural-entitlement representation theorem applies. The shortcut lands in strictly stronger posterior knowledge, a structure-addition event, and the $\Delta\mu$ lower bound.*

Proof. A `SoundStructuralShortcut` Coq record (defined at `coq/kernel/nfi/HonestNoFI_TheoremsWithoutAssumptions.v:501-546`) is, by construction, a tuple of: Ω, Ω' with $\Omega' \subsetneq \Omega$, an observation function f_{obs} with a distinguishing-witness proof, a decision tree T with a tree-realized-by-the-trace proof, a posterior-representative reduction with a representatives proof, and an initial-uncertified proof on the starting state. These fields and obligations are exactly the hypotheses of the Structural-Entitlement-Representation theorem. Project the seven data fields and eight proof obligations from the record into the theorem's hypotheses; apply the theorem; the three conclusions (strictly stronger predicate, structure-addition event, $\log_2^{\uparrow}$-narrowing $\leq \Delta\mu$) follow. □ □

Plain reading. The record is exactly what the representation theorem needs as input. Unpack the record, plug the fields into the theorem, read off the three conclusions. The class is non-empty because the concrete witness `factored_n1_shortcut` in `coq/kernel/nfi/StructuralAdvantageCertifiedShortcut.v` constructs a member of the class explicitly.

This is the formal version of "it has to." Once a shortcut is no longer just a human story but a sound object the machine may use, it has to provide the data that makes the shortcut admissible. In this framework, that data is exactly the structural-entitlement witness. If someone says symmetry, factorization, modularity, independence, or decomposition gave them the shortcut, the next question is no longer mystical. Show me the prior set, the posterior set, the observation that rules something out, and the representative/tree witness. If they can show that, the theorem applies. If they cannot,

then the structure is still outside the computation.

5.2 What narrowing buys

The machine also has a concrete structural-advantage demonstration. `StructuralAdvantage.v` studies a factored search problem. The blind program has all instruction costs set to 0 and searches the flat space. The sighted program pays two receipt-visible events, each `EMIT "." 0`, for total cost 18, and searches two factors separately. The file proves the exact cost formulas and the arithmetic fact that the N^2 versus $2N$ gap eventually dwarfs the constant μ cost. The executable tests check the measured VM behavior on representative sizes.

The point of this example is not the N^2 versus $2N$ speedup. Divide-and-conquer with that speedup has been known since the 1960s. The point is the auditable trace: the receipt-visible `EMIT` events plus the certified suffix make the structural shortcut admissible to the machine, not just asserted by the programmer. The 18 μ is the cost of producing the receipt. The receipt is the evidence that the structural shortcut is honest.

There are two nearby claims here and it matters to keep them separate. The original `EMIT`-only $N = 1$ witness closes the observation layer: it proves posterior admissibility as `CertifiedObs` (a Coq type for "a certified observation has been recorded," meaning the machine has observed and recorded a result but has not yet activated a cert channel). It does not close the stronger bridge. The run does not end in `has_supra_cert`, and in the semantic CSR-transition sense it does not realize structure addition. A minimally extended witness does close the full theorem honestly: keep the factorized search exactly as it is, then append `PNEW`, `MORPH_ID`, and `MORPH_ASSERT`. That suffix is enough to fire the concrete certification channel, so the extended trace lands in the full structure-addition theorem.

That example is not a P vs NP result. It is a small, honest witness for the thing this machine is meant to track: certified structure can change which computation is admissible, and the cost of acquiring that admissibility is not the same as the time saved by using it.

What the package looks like in the concrete `n=1` witness. The kernel files `coq/kernel/nfi/StructuralAdvantageObservedShortcut.v` and `coq/kernel/nfi/StructuralAdvantageCertifiedShortcut.v` instantiate the components for the smallest factored-search case. The record `SoundStructuralShortcut` (`coq/kernel/nfi/HonestNoFI_TheoremsWithoutAssumptions.v:501-546`) carries fifteen fields: seven data fields (decoder, observation fn, representative observation fn, observation equality, prior set, posterior set, decision tree) and eight proof obligations on those data. The nine semantic ingredients below cover all seven data fields plus two of the proof obligations (strict narrowing and initial-uncertified state); the remaining six obligations are discharged inline by the wiring in

`StructuralAdvantageCertifiedShortcut.v`.

1. **Prior feasible set Ω.** `sighted_n1_supra_prior := [init_state; sighted_n1_supra_final]`. A literal two-element list of VMStates.
2. **Posterior feasible set Ω'.** `sighted_n1_supra_posterior := [sighted_n1_supra_final]`. A literal one-element list.
3. **Strict narrowing.** $|\Omega'| = 1 < 2 = |\Omega|$ by direct inspection of the lists.
4. **Distinguishing observation.** `sighted_n1_supra_obs_fn`. Returns `sighted_n1_supra_trace` when `vm_mu = 19`, else `[]`. The post-MORPH_ASSERT state has `vm_mu = 19`. The initial state has `vm_mu = 0`. The observation separates them.
5. **Decision tree.** `sighted_n1_tree := dt_branch dt_leaf dt_leaf` (defined in `coq/kernel/nfi/StructuralAdvantageObservedShortcut.v:53-54` and reused by the certified-shortcut file). A two-leaf branching tree, one branch per factor.
6. **Tree realized by the trace.** The trace's actual EMIT-then-search-then-MORPH_ASSERT sequence matches a root-to-leaf path of the tree.
7. **Posterior-representative reduction.** `sighted_n1_supra_repr_obs_fn` (`coq/kernel/nfi/StructuralAdvantageCertifiedShortcut.v:41-42`). Defined as `if Nat.eqb s.(vm_mu) s.(vm_mu) then [] else [instr_halt 0]`; the tautology `Nat.eqb mu mu = true` always picks the empty-list branch, so this is the empty-observation representative dressed up as a conditional.
8. **Certified posterior predicate.** Built from `omega_predicate receipt_list_eqb sighted_n1_supra_posterior sighted_n1_supra_obs_fn`. The MORPH_ASSERT in the supra-suffix fires the certification.
9. **Initial uncertified state.** `init_state.(vm_csrs).(csr_cert_addr) = 0` by definition of `init_state`.

The Coq lemma `sound_shortcut_from_components` (`coq/kernel/nfi/HonestNoFI_TheoremsWithoutAssumptions.v`, lines 637 to 679) constructively builds a `SoundStructuralShortcut` record from seven data parameters (the seven data fields above) plus eight proof-hypothesis arrows discharging the eight obligations. The closed term `factored_n1_shortcut : SoundStructuralShortcut 33 sighted_n1_supra_trace init_state` in `coq/kernel/nfi/StructuralAdvantageCertifiedShortcut.v` wires through `sound_shortcut_from_components`, providing the seven data parameters above and discharging each of the eight hypotheses by a named lemma. The downstream theorem `factored_n1_shortcut_lands_in_representation`

in the same file applies `every_sound_structural_shortcut_lands_here` to the closed term and reads off the conclusion: strictly stronger posterior predicate, structure-addition event, and the $\log_2^{\uparrow}$ narrowing bounded by $\Delta\mu$. The class is constructively non-empty.

5.3 The boundary

The hard internal theorem for the explicit class is proven, and so is the boundary in both directions on a concrete witness. An `EMIT`-only factorized-search trace closes only the observation layer: the posterior predicate is certified observationally, but the run does not reach `has_supra_cert` and does not realize semantic structure addition. A trace with the factorized search unchanged and a post-search suffix `PNEW ; MORPH_ID ; MORPH_ASSERT` closes the whole chain: stronger predicate, structure addition, and the $\Delta\mu$ lower bound. An unconditional Shannon-style bound on every deterministic single trace is something the code refuses to claim. A single execution path is not enough information; the bound needs the whole tree, or a per-branch preimage, or a distribution over traces. The proof stops where the witness stops. That is the honest place to stop, not a hole to be patched later.

Once structural entitlement is internalized with explicit witnesses, free certified strengthening is impossible; classical shadows lose entitlement-relevant distinctions; and feasible-set narrowing has a formal path into predicate strengthening and a ledger lower bound. The substrate-level result in Section 18 closes the question of whether a uniform internally representable translation procedure exists from informal structural arguments into `SoundStructuralShortcut` witnesses: it does not. The structural axis has its own undecidable predicate.

6 What insight is, and why it cannot be free

In this machine, insight has a precise meaning. Three kinds of observation:

6.1 The three tiers of observation

Not all observations are the same. The taxonomy is proven in Coq. File: `WitnessInsightGeneral.v`.

Tier 0: Raw observation (always free). A raw observation records data without committing to any structural interpretation. The clearest example in the Thiele Machine is a CHSH trial, an experimental record from the two-player game built from scratch in Section 15. You run a trial, you record the outcome in a counter, and nothing structural changes. The counter goes up. The certification channels are not touched. This is free. Cost: 0.

Tier 1: Certified structural insight (always costs ≥ 1). The machine has two recorded certification channels. One is the formula/certificate address channel, the control-status register `csr_cert_addr` (Section 9 explains the term). The other is

the top-level flag, `vm_certified`. Any instruction that turns either channel from off to on must have cost at least 1. This is proven in Coq (the theorem named `certification_requires_positive_mu`).

One thing to be precise about: in the current step rule, LASSERT (the formula-check opcode) reads a constraint package and charges for it, but LASSERT does not directly flip the certification channels. The two concrete state-changing instructions are CERTIFY (sets `vm_certified = true`) and MORPH_ASSERT (sets `csr_cert_addr` to a nonzero value when the assertion succeeds). LASSERT performs the validation that might precede MORPH_ASSERT or CERTIFY. The Coq file `AbstractNoFI.v` reasons about a broader cost-positive class that includes LASSERT, EMIT, REVEAL, LJOIN, READ_PORT, CERTIFY, and MORPH_ASSERT: each of them costs at least 1 at the step level. The distinction matters because LASSERT checking a formula is part of the chain of work that leads to certification, even if the formal cert-channel flip happens via the subsequent CERTIFY or MORPH_ASSERT.

Tier 2: Certified binary-game witness insight (always costs ≥ 1). The sharpest case: a machine that is certified AND whose CHSH statistics show that the CHSH score S exceeds 2, which is the classical bound (Section 15 builds the game and the bound). Getting certified is already a Tier 1 event (cost at least 1). Getting the statistics to show a violation means the data is incompatible with any local hidden variable model (Section 2), which is proven in `CHSHStatisticalBridge.v`. Together: certified game-violation evidence costs at least 1 to acquire, over any trace of any length.

6.2 Why insight cannot be free

The proof:

Suppose a machine starts uncertified and ends certified. Somewhere in that trace, a specific opcode fired that flipped one of the certification registers from off to on. The two concrete cert-channel instructions writing `vm_certified` or `csr_cert_addr` are: CERTIFY (which sets `vm_certified = true`) and MORPH_ASSERT when the morphism exists (which sets `csr_cert_addr` to the ASCII checksum of the property string: nonzero for any non-empty property string). These are the only instructions that change those fields. CHSH_LASSERT is a cert-setter in the cost-discipline sense ($S(\delta) \geq 1$) but does not write to `vm_certified` or `csr_cert_addr`; its success is observable via PC advance and `vm_err` staying false (the bridge theorem operates on that signature). MORPH_ASSERT with an empty property string or a missing morphism does not activate the cert channel, though it still costs $S(\delta) \geq 1$. This is proven by case analysis over all 47 opcodes, every other instruction leaves those fields unchanged. So: the cert flip happened at a CERTIFY or MORPH_ASSERT step. Look at that exact step. That step, by the semantics of the machine, has cost $S(\delta) \geq 1$, because both CERTIFY and MORPH_ASSERT are in the mandatory-floor class. So at that step: $\mu_{\text{after}} = \mu_{\text{before}} + S(\delta) \geq \mu_{\text{before}} + 1$. And because μ never decreases (each instruction adds a non-negative cost), $\mu_{\text{before}} \geq \mu_{\text{initial}}$. Therefore

$\mu_{\text{final}} \geq \mu_{\text{after}} \geq \mu_{\text{before}} + 1 \geq \mu_{\text{initial}} + 1.$

This is not a philosophical argument. It is a structural property of the step rule. The Coq proof checks it mechanically. The proof does not say "intuitively, certification should cost something." It says: here is the step-rule definition, here is the cost rule, here is the lemma that says μ never decreases (the monotonicity lemma; the Section 2 sense of monotonicity is the right one), here is the induction over the trace (Section 2 for what induction means). End of proof. The deeper reason: if you could certify structure for free, the certification would mean nothing. A receipt that any machine can produce at zero cost proves nothing. The whole point of the μ ledger is that it cannot be faked. You cannot accumulate certified structural knowledge without a trace of cost.

But cost alone is not yet semantic depth. A machine can spend resources checking a claim that turns out to be shallow. LASSERT therefore does not count as structural certification merely because a formula parses and one satisfying assignment exists. It must also carry a non-triviality witness: one satisfying assignment and one falsifying assignment. That proves the asserted constraint excludes at least one state and therefore genuinely narrows the feasible set. The receipt proves spend; the witness proves the spend bought an actual narrowing.

7 The μ ledger

μ (mu) is the global structural cost ledger of the Thiele Machine.

7.1 What mu is

μ is a natural number attached to every machine state. (A natural number is a non-negative integer: 0, 1, 2, ... The mathematical-notation cheat sheet is in Section 2.) It starts at 0. It never decreases.

Every instruction carries a cost parameter (δ) in its encoding. The machine adds that cost to μ when the instruction executes. Here is how δ is used in practice:

For **pure compute programs**, meaning programs that do arithmetic, load and store values, and jump, every instruction carries $\delta = 0$. The program I use as the concrete comparison case for the Turing Strictness theorem (`blind_program` in `StructuralAdvantage.v`) has this property: all instructions at $\delta = 0$, and the proof that its total μ-cost is zero is a direct calculation. For these programs, μ stays at 0 unless a positive-cost instruction fires.

For **certification and revelation instructions** (CERTIFY, LASSERT, MORPH_ASSERT, EMIT, REVEAL, LJOIN, READ_PORT): the cost is at least 1, regardless of δ. Even $\delta = 0$ still costs 1. Here is why that floor is forced, not chosen.

When CERTIFY fires, it commits the machine to a certified state: `vm_certified` flips from false to true. That commitment cannot be undone, because μ never goes down

and the certificate cannot be taken back. Committing to a structural fact is the same shape of event as erasing the "uncertified" possibility from the machine's history: once the commitment is recorded, the prior uncertainty is gone. Landauer's argument (covered in Section 16) is the physics-side motivation for charging at least 1 in this case, though the Coq proof does not depend on it. The minimum honest cost of an irreversible certification step, as a function only of the programmer-supplied δ, is $\delta + 1$: the δ the programmer declares plus the mandatory 1 for the commitment. I write that as $S(\delta)$ throughout.

Why $+1$ and not $+2$ or $+0.5$? Because $+1$ is the minimum that makes certification impossible for free. $\delta + 0$ would allow $\delta = 0$ to certify for free. $\delta + 2$ would be overcharging. $\delta + 1$ is the unique honest minimum: you can always choose $\delta = 0$ and pay exactly 1; you can choose $\delta > 0$ and pay more. The proof that $S(\delta)$ is the only consistent cost floor is in `MuCostDerivation.v` (`cost_necessity` + `cost_uniqueness`).

Throughout this document, $S(n)$ means $n + 1$. The S stands for "successor" in the basic math sense: the successor of n is the next natural number after n. So $S(0) = 1$, $S(1) = 2$, and so on. $S(\delta) = \delta + 1$ is always at least 1. EMIT, REVEAL, and READ_PORT go further: their cost also grows with the information content they carry. An EMIT of one ASCII character (which is 8 bits, see Section 2) with $\delta = 0$ costs $8 + 1 = 9$. Ten characters costs $80 + 1 = 81$. You cannot emit more for the same price as less.

For **everything outside that positive-cost class**, including ADD, LOAD, STORE, JUMP, PNEW, PSPLIT, PMERGE, MORPH, COMPOSE, and the rest, the machine charges exactly δ, which can be 0. Here is why these are allowed to be free.

PNEW creates a module. PSPLIT splits one module into two. PMERGE merges two modules into one. These are *reversible*: PNEW can be undone by removing the module; PSPLIT and PMERGE undo each other. Landauer's principle says reversible operations carry zero information-erasure cost. So PNEW at $\delta = 0$ is free, not as a special exemption, but because there is genuinely no information being destroyed.

The same logic applies to MORPH, COMPOSE, and the other graph-building operations. Building the morphism chain is the organisational work; the result is structure you have not yet committed to. The cost arrives when you assert the chain via MORPH_ASSERT. That is the moment of commitment, and the moment of commitment costs $S(\delta) \geq 1$. You can build for free. You cannot certify for free.

μ grows whenever `instruction_cost` is positive. The certification/revelation class is forced to be positive even when its encoded δ is 0. Other instructions can remain free by carrying $\delta = 0$.

Here is the complete cost schedule:

Instruction class	Cost = `instruction_cost`
Ordinary compute (ADD, LOAD, JUMP, PNEW, MORPH, ...)	δ (can be 0)
Certification setters (CERTIFY, MORPH_ASSERT, LJOIN)	$S(\delta) = \delta + 1$
EMIT (emit payload as certified trace event)	$\lvert\text{payload}\rvert_{\text{bits}} + S(\delta)$
REVEAL (disclose bits from a module)	$\text{bits} + S(\delta)$
READ_PORT (read bits from input port)	$\text{bits} + S(\delta)$
LASSERT (certify a logical formula)	$\mathit{flen} \times 8 + S(\delta)$

The $\lvert\text{payload}\rvert_{\text{bits}}$ notation means "the number of bits in the payload." A one-byte ASCII character contains 8 bits. A ten-byte string contains 80 bits.

For LASSERT: *flen* is the number of formula units in the encoded formula. Each unit is one byte (8 bits) in the binary encoding used by the machine. This is a design choice in the binary format: the formula header declares how many bytes follow, and the machine charges 8 bits per byte. The multiplication by 8 simply converts byte-count to bit-count, matching the information content you're claiming to have certified.

The proof that μ is always exactly $\mu_{\text{before}} + \texttt{instruction\_cost}$ for each step:
Theorem 7.1 (μ-Conservation)**.** *For any state s and instruction i:* $(\texttt{vm\_apply}\ s\ i).\mu = s.\mu + \texttt{instruction\_cost}\ i$.

Proof. Case analysis over the 47 opcode constructors of `vm_instruction`. For each constructor i, the apply rule has the form

$$(\texttt{vm\_apply}\ s\ i).\mu \ = \ s.\mu + c_i,$$

where c_i is the cost term written in the apply rule. Compare to `instruction_cost`: for ordinary compute opcodes $c_i = \delta$; for cert-setters (CERTIFY, MORPH_ASSERT, LJOIN, CHSH_LASSERT) $c_i = S(\delta)$; for EMIT $c_i = \lvert\text{payload}\rvert_{\text{bits}} + S(\delta)$; for REVEAL and READ_PORT $c_i = \text{bits} + S(\delta)$; for LASSERT $c_i = \mathit{flen} \times 8 + S(\delta)$. In every case c_i matches the corresponding case of `instruction_cost` by definition. The error sub-cases (e.g., MORPH_ASSERT on a missing morphism, CHSH_TRIAL with bad bits, LASSERT length mismatch) follow the same cost rule: the trap or err-latch only changes the PC and the error flag; the μ update is the same. So $(\texttt{vm\_apply}\ s\ i).\mu = s.\mu + c_i = s.\mu + \texttt{instruction\_cost}(i)$ for every i. □ □

Plain reading. Look at each of the 47 opcodes one at a time. Every apply rule writes μ as the old μ plus the cost. The cost term is exactly what `instruction_cost` returns. Including the error paths: failing an instruction costs the same as succeeding at it, because the cost is on the attempt, not the outcome. There is no opcode that escapes this. There is no backdoor.

This is not a monotonicity claim. It is an equality. μ goes up by exactly the cost of the instruction, always, for every instruction, in every case including error cases. There is no backdoor.

That equality should not be misunderstood. It means the ledger is exact about spend. It does not, by itself, mean every paid unit corresponds to deep semantic gain. In this machine, semantic gain is stricter than spend: it requires a narrowing witness in addition to the ledger entry.

7.2 Why mu is the unique canonical cost measure

Here is the stronger claim, which took a while before the proof compiled. μ is not just a valid cost measure for this machine. It is the unique one. Any other cost function that starts at zero and tracks the per-instruction cost schedule exactly must equal μ on every state the machine can reach. Not "isomorphic" in some abstract sense. Literally equal, value by value, on every reachable state.

Theorem 7.2 (μ-Initiality)**.** *Let M be any function from machine states to natural numbers satisfying: (a) M assigns zero to the initial state of the machine, $M(\texttt{init\_state}) = 0$; and (b) M tracks the cost schedule exactly: for every state s and every instruction i, $M(\texttt{vm\_apply}\, s\, i) = M(s) + \texttt{instruction\_cost}(i)$. Then M equals μ on every reachable state. (Monotonicity, the property that $M(s') \geq M(s)$ when s' is reachable from s, follows from (b) because `instruction_cost` is a non-negative natural number. You do not need to state monotonicity separately.)*

Proof. Induction on the reachability witness for s. A reachable state is either `init_state` itself or `vm_apply` s' i for some reachable s' and some instruction i.

Base case ($s =$ `init_state`). By hypothesis (a), $M(\texttt{init\_state}) = 0$. By definition of the initial state in the kernel, $\texttt{init\_state}.\mu = 0$. So $M(\texttt{init\_state}) = \texttt{init\_state}.\mu$.

Step case ($s =$ `vm_apply` s' i for reachable s'). By hypothesis (b),

$$M(s) \;=\; M(s') + \texttt{instruction\_cost}(i).$$

By the induction hypothesis on s', $M(s') = s'.\mu$. By μ-Conservation,

$$s.\mu \;=\; (\texttt{vm\_apply}\, s'\, i).\mu \;=\; s'.\mu + \texttt{instruction\_cost}(i).$$

Substituting: $M(s) = s'.\mu + \texttt{instruction\_cost}(i) = s.\mu$. □ □

Plain reading. M agrees with μ at the start (both zero). The step rule says M and μ both update by the same amount on every step (M by hypothesis (b), μ by μ-Conservation). Induction closes it. There is no alternative accounting that agrees with the cost schedule at every step, starts at zero, and disagrees with μ on a reachable state. The cost is forced.

In plain language: given the cost schedule, there is no alternative accounting. If you charge $S(\delta)$ for cert-setters, $|\text{payload}|_{\text{bits}} + S(\delta)$ for EMIT, and δ for everything else, then μ is the only function on states that starts at zero and tracks the schedule. What was a design choice is the schedule itself: the decision that CERTIFY costs $S(\delta)$ and ADD costs δ. Once those choices are in place, the initiality theorem closes the question of whether μ is the right thing to be tracking. You cannot swap in a different count that agrees on the schedule and reaches a different total. That freedom is closed.

7.3 The LASSERT cost formula

The most important cost formula in the machine is for LASSERT, the opcode that certifies a logical formula:

$$\Delta\mu_{\text{LASSERT}} = \mathit{flen} \times 8 + S(\mathit{mu\_delta})$$

flen is the encoded formula-unit count, read from the formula's own header in memory. Each unit contributes eight concrete bits, and the per-step certification charge is always at least 1.

The machine reads how many encoded formula units are actually present, charges eight bits for each unit, and adds at least 1 for the certification step. A longer formula costs more. A two-unit formula costs at least 16 + 1 = 17. The programmer does not get to smuggle a larger formula through a smaller bit count.

This is forced, not chosen. You cannot certify a 1000-bit formula by paying for a 10-bit one; the cost counts the formula you actually present. The proof that this is the unique minimum cost satisfying that constraint is in `MuCostDerivation.v` (`cost_necessity` + `cost_uniqueness`).

What this buys you: the formula pays for presenting and checking a constraint package. It does not, by itself, guarantee the constraint is meaningful. You could present a formula that is always true, a tautology, and the ledger would still charge you. The machine closes this separately via the dual-witness requirement: see the LASSERT opcode description in Section 10.

The natural attack, and why it doesn't work. The programmer declares *flen* in the instruction encoding. What stops them from writing $\mathit{flen} = 0$ for a huge formula and paying only $S(0) = 1$?

The hardware catches it. When LASSERT runs, the machine reads the formula's actual

encoded-unit count from the formula header in memory and compares it to the *flen* in the instruction. If they do not match, the machine traps (Section 2: routes execution to a fixed error address): the program counter jumps to `LASSERT_TRAP_PC`, which is 0xF00 in hexadecimal, decimal 3840. The error flag is set. The check does not succeed. This trap-to-fixed-address behaviour is unique to LASSERT. Every other error in the machine simply sets `vm_err` to true and continues advancing the program counter by 1. LASSERT is special because a mis-length formula is a serious integrity violation, not a recoverable error. The μ cost is still charged: that detail matters because failed checks are not free. The Coq lemmas `lassert_honest_cost` and `lassert_honest_mu_cost` in `StateSpaceCounting.v` prove that if LASSERT completes without trapping, the declared count equals the in-memory count, and the μ charge uses the real formula bits.

8 No Free Insight: the first conservation law

8.1 The abstract versions

Theorem 8.1 (Universal No Free Certification)**.** *Let $\mathcal{C}$ be any abstract computational system with:*

- *a set of states;*
- *a set of instructions;*
- *a step rule (given a state and an instruction, the rule produces the next state);*
- *a cost function (each instruction has a non-negative integer cost);*
- *a certification predicate (a yes/no test on states, with at least one "no" state and at least one "yes" state).*

Premise (A2, the single-step cost rule): for every state s and instruction i, if $\mathrm{cert}(s) = \mathrm{false}$ *and* $\mathrm{cert}(\mathrm{step}(s, i)) = \mathrm{true}$*, then* $\mathrm{cost}(i) \geq 1$.

Conclusion: any trace $t = [i_1, i_2, \ldots, i_n]$ that starts at state s_0 with $\mathrm{cert}(s_0) = \mathrm{false}$ *and ends at state s_n with* $\mathrm{cert}(s_n) = \mathrm{true}$ *has total cost* $\sum_{k=1}^{n} \mathrm{cost}(i_k) \geq 1$.

Proof. Induction on the trace t.

Base case ($t = []$). Then $s_n = s_0$, so $\mathrm{cert}(s_n) = \mathrm{cert}(s_0) = \mathrm{false}$. But the hypothesis says $\mathrm{cert}(s_n) = \mathrm{true}$. Contradiction; the empty-trace case is vacuous.

Step case ($t = i :: \mathrm{rest}$). Let $s_1 = \mathrm{step}(s_0, i)$ be the state after the first instruction. Case-split on $\mathrm{cert}(s_1)$.

- *Subcase A:* $\mathrm{cert}(s_1) = \mathrm{true}$. The first instruction flipped the certification predicate from false to true in one step. By the A2 premise, $\mathrm{cost}(i) \geq 1$. The total cost is

$\text{cost}(i) + \sum_{j \in \text{rest}} \text{cost}(j) \geq 1 + 0 = 1$, since every individual cost is a non-negative natural.

- *Subcase B:* $\text{cert}(s_1) = \text{false}$. Then the rest of the trace must do the flipping: $\text{cert}(\text{run}(\text{rest}, s_1)) = \text{true}$ with $\text{cert}(s_1) = \text{false}$. Apply the induction hypothesis at (rest, s_1) to get $\sum_{j \in \text{rest}} \text{cost}(j) \geq 1$. The total trace cost is $\text{cost}(i) + \sum_{j \in \text{rest}} \text{cost}(j) \geq 0 + 1 = 1$.

Either subcase gives total trace cost ≥ 1. □ □

Plain reading. The cert flag is off at the start, on at the end. So somewhere a step flipped it. Whichever step did the flipping costs at least 1 by the single-step rule. Every other step costs at least 0 because costs are non-negative naturals. Sum them up: at least 1. Empty trace can't do the flipping at all, so it's vacuous. That's the whole argument. The induction just makes precise the idea that the flip has to happen *somewhere*, and wherever it happens it pays.

This theorem does not know about registers, memory, partitions, morphisms, CHSH trials, or the Thiele ISA. It says: once a model has a yes/no certification test and the one-step certification transition is priced, trace-level non-free certification follows by induction over the trace. If that one-step premise fails, the model permits free certification by definition. If the model has no certification predicate, it cannot talk about structural entitlement internally.

Theorem 8.2 (Thiele No Free Insight)**.** *For the Thiele Machine specifically, cert-channel activation and certified predicate strengthening require structure-addition events, and those events are in the positive-cost certification/revelation class.*

Proof. Two parts.

Cert-channel activation requires positive cost. The Both-channels theorem below establishes that any trace activating either certification channel (`csr_cert_addr` flip or `vm_certified` flip) has $\Delta\mu \geq 1$. The flipping step in each case is a `CERTIFY`, `MORPH_ASSERT`, `EMIT`, `REVEAL`, `LJOIN`, `LASSERT`, or `CHSH_LASSERT` instruction — exactly the cert/revelation cost class with `instruction_cost` $\geq S(\delta) \geq 1$.

Certified predicate strengthening requires a structure-addition event. By hypothesis the run ends certified for a stronger predicate via the supra-cert channel (`csr_cert_addr` $\neq 0$). The Strengthening-requires-structure-addition theorem (Section 5) gives the structure-addition event directly. □ □

Plain reading. This is just the abstract universal NoFI theorem instantiated for the Thiele VM and stated at the Thiele-Machine layer. The two "ways to get certified" (cert_addr channel and vm_certified flag) are both cost-positive at the step level,

and the predicate-strengthening reading of NoFI follows from the structure-addition theorem.

This is the machine-level version. `AbstractNoFI.v` proves the two concrete cert channels cannot be activated for free. `NoFreeInsight.v` adds the predicate-strengthening layer: once a stronger accepted predicate is tied to `has_supra_cert`, the run must contain a structure-addition event.

8.2 The specific versions

For the Thiele Machine concretely, there are several specific versions. The two certification channels I mentioned in Section 6 are the formula-certification register (`csr_cert_addr`) and the top-level flag (`vm_certified`). Both are defined precisely in Section 9.

Theorem 8.3 (No Free Certification, single step)**.** *If* $s.\texttt{vm\_csrs.csr\_cert\_addr} = 0$ *and* $(\texttt{vm\_apply}\ s\ i).\texttt{vm\_csrs.csr\_cert\_addr} \neq 0$*, then* $\texttt{instruction\_cost}(i) \geq 1$.

Proof. Case analysis over the 47 opcode constructors. The `vm_apply` function changes `csr_cert_addr` from 0 to nonzero only when i is one of: `instr_morph_assert` (writes the ASCII checksum of a non-empty property string), `instr_emit`, `instr_reveal`, `instr_ljoin`, or `instr_lassert`. Every other opcode either leaves `csr_cert_addr` alone or fails (in which case it also leaves it alone). For each of the five flipping opcodes, the cost schedule at Section 7 gives $\texttt{instruction\_cost}(i) = S(\delta) = \delta + 1 \geq 1$ (cert-setter discipline) or higher (EMIT and REVEAL pay payload-bits$+S(\delta)$). □ □

Plain reading. Only five opcodes can ever set `csr_cert_addr` from zero to nonzero, and every one of them carries an $S(\delta) \geq 1$ cost floor. So if you see the field flip, the instruction that did it paid at least 1.

Theorem 8.4 (No Free Certification, trace level)**.** *If a trace starts at a state with* $\texttt{csr\_cert\_addr} = 0$ *and ends at a state with* $\texttt{csr\_cert\_addr} \neq 0$*, then the trace's total* μ *cost is* ≥ 1.

Proof. Instantiate the universal theorem above with $\mathcal{C}$ = the Thiele VM; $\text{cert}(s) := (s.\texttt{vm\_csrs.csr\_cert\_addr} \neq 0)$; step = `vm_apply`; cost = `instruction_cost`. The A2 premise is exactly the single-step theorem above. □ □

Plain reading. Single-step lifts to trace-level. The universal theorem does the work.

Theorem 8.5 (No Free Certification via CERTIFY)**.** *If a trace starts at a state with* $\texttt{vm\_certified} =$ false *and ends at a state with* $\texttt{vm\_certified} =$ true*, then the trace's total* μ *cost is* ≥ 1.

Proof. Case analysis over the 47 opcodes shows that `vm_certified` flips from false to true only when $i =$ `instr_certify`; every other opcode leaves it unchanged. The CERTIFY apply rule charges $S(\delta) \geq 1$. So the single-step instance of A2 holds for this channel. Instantiate the universal theorem with cert$(s) :=$ s.`vm_certified`, otherwise as above. □ □

Plain reading. Same shape as the other channel. Only one opcode (CERTIFY) can flip the top-level flag. It charges ≥ 1. The universal theorem lifts that to the trace.

Theorem 8.6 (Both channels)**.** *For ANY transition of either certification channel from absent to present, whether the formula register* `csr_cert_addr` *or the top-level flag* `vm_certified`*, μ grows by at least 1 over the trace.*

Proof. Suppose the trace either flips `csr_cert_addr` from 0 to nonzero, or flips `vm_certified` from false to true (or both). In the first case the trace-level theorem (csr_cert_addr) applies. In the second case the trace-level theorem (vm_certified) applies. The disjunction is exhaustive over "cert-channel activation," so μ grows by ≥ 1 in either case. □ □

Plain reading. Two channels, one master claim. Whichever channel got activated, the corresponding single-step rule fired and the universal theorem lifted it to the trace. There is no third way.

Theorem 8.6 is the master NoFI result. It covers both ways to get certified. There is no third way.

8.3 What this means in practice

Run any program on the Thiele Machine. Watch the μ counter. If the program ends with `vm_certified = true` or with `csr_cert_addr` nonzero, then somewhere in the execution, μ went up by at least 1. You can audit the trace and find exactly where. There is no program that can reach a certified state from scratch without paying for it.

This is what I mean by unforgeable. The μ ledger is the receipt. If you are certified, the receipt exists.

9 The formal machine state

Definition 9.1 (Thiele Machine State)**.** A Thiele Machine state s is a record with 12 fields:

- s.`vm_graph`: the partition graph. It contains module records, morphism records, and fresh-ID counters.
- s.`vm_csrs`: control-status registers. In hardware design, there are two kinds of registers. Data registers (like s.`vm_regs`) hold computation values. Control-

status registers hold machine configuration and status. This record has four fields:

  - `csr_cert_addr`: 0 means nothing has been asserted yet. Set to the ASCII checksum of the property string when MORPH_ASSERT succeeds (nonzero for any non-empty property string). This is not a memory address, it is a checksum fingerprint of the property that was asserted. Having a nonzero value here means a structural property was certified and paid for.
  - `csr_status`: general status code. Informational; not currently used to gate opcodes.
  - `csr_err`: error code. Nonzero means an instruction failed (bad argument, missing morphism, locality violation, etc.).
  - `csr_heap_base`: base address for HEAP_LOAD and HEAP_STORE. Heap-relative memory access adds this base to the register-specified offset.

- s.`vm_regs`: a list of register values. The kernel fixes `REG_COUNT = 16`; register reads and writes wrap indices modulo `REG_COUNT`. The kernel proofs are parametric in `REG_COUNT`.
- s.`vm_mem`: a list of data-memory words. The kernel fixes `MEM_SIZE = 128`; memory reads and writes wrap indices modulo `MEM_SIZE`. The kernel proofs are parametric in `MEM_SIZE`.
- s.`vm_pc` $\in \mathbb{N}$: the program counter.
- s.`vm_mu` $\in \mathbb{N}$: the structural cost ledger. Starts at 0. Never decreases.
- s.`vm_mu_tensor`: a 4×4 array of cost values (16 entries), indexed by direction pairs (i, j). TENSOR_SET writes an entry, TENSOR_GET reads one. REVEAL records disclosure costs at the direction index encoded in the instruction, so the machine can track *where* a cost was paid across directions, not just how much. In the kernel model these are natural numbers; in the hardware, 32-bit words. If you don't use the tensor tracking, ignore all 16 entries.
- s.`vm_err` $\in \{\text{false}, \text{true}\}$: the error flag. Set when an instruction fails.
- s.`vm_logic_acc`: a natural number serving two roles. First: the running accumulator for the Logic Engine during LASSERT: the on-chip unit that reads formula words and certificate bytes and tracks validation state. Second: a magic key the Logic Engine FSM matches against during LASSERT validation. When `vm_logic_acc` equals the constant `0xCAFEEACE` (hexadecimal; 3,405,703,886 in decimal), the formula-check path proceeds; mismatches the FSM rejects. The field exists so the Coq kernel and the physical hardware agree on its value after every step.
- s.`vm_mstatus`: machine-status word, currently either 0 or 1. 0 means Turing mode: the machine is executing ordinary computation, the partition graph and morphism map are idle. 1 means Thiele mode: structural state is live. In the

current kernel, this field is a status indicator that records which operational context the machine is in, it does not gate the structural opcodes at the step level; the opcodes are always available. The field exists so the hardware and the Coq model agree on machine status across the bridge proofs.

- s.vm_witness: the CHSH witness counters. This is a record of 8 natural-number counters, one for each combination of (measurement setting pair) $\times$ (same/different outcome). The four setting pairs are $(x, y) \in \{(0,0),(0,1),(1,0),(1,1)\}$, where x is Alice's setting and y is Bob's setting — the modern Bell-test convention used by the NPA hierarchy and the Brunner et al. 2014 review, and the same one the Coq constructor `instr_chsh_trial (x y a b : nat)` uses. The outcome variables are (a, b). For each setting pair, there are two counters: same-outcome and different-outcome. Eight counters total: `wc_same_00`, `wc_diff_00`, `wc_same_01`, `wc_diff_01`, `wc_same_10`, `wc_diff_10`, `wc_same_11`, `wc_diff_11`, where the numeric subscript is the value of (x, y) for that bucket. Each CHSH_TRIAL instruction increments exactly one counter. (Older Bell-test literature sometimes uses the reverse convention with (a, b) for settings; the numeric subscripts 00, 01, 10, 11 are unambiguous either way.) The CHSH score S is computed from the aggregate sums of these counters. For each setting pair (x, y), define $N_{xy} = \texttt{wc\_same}_{xy} + \texttt{wc\_diff}_{xy}$ (total trials with that setting) and $E_{xy} = (\texttt{wc\_same}_{xy} - \texttt{wc\_diff}_{xy})/N_{xy}$ (the correlation: fraction same minus fraction different, ranging from -1 to $+1$). Then $S = E_{00} + E_{01} + E_{10} - E_{11}$. If no trials have been run for a setting ($N_{xy} = 0$), the correlator for that setting defaults to 0 by convention. Classically $|S| \leq 2$; the NPA algebraic model allows $|S| \leq 2\sqrt{2}$ (see Section 15). The concrete Coq definition is `chsh_stat_from_wc` in `CHSHStatisticalBridge.v`.
- s.vm_certified $\in \{\text{false}, \text{true}\}$: the top-level certification flag. Set by the CERTIFY opcode.

The Coq kernel (the mathematical ground truth) and the physical hardware are intentionally not the same shape at every bit. The Coq kernel stores register and memory values as natural numbers, but truncates writes through 64-bit word helpers to keep the math clean. The hardware layer (explained in Section 14) is finite: 16 general-purpose registers, 128 memory words, bounded tables. The bridge proofs in the hardware section state exactly where those finite hardware representations match the kernel step; the size constants are parametric so the same proofs apply at any bound.

Definition 9.2 (Partition Graph)**.** The partition graph s.vm_graph is a record:

- `pg_next_id`: the next fresh module identifier.
- `pg_modules`: a list of module-ID/module-record pairs. Each `ModuleState` holds region data, axiom data, and the module's μ-tensor.
- `pg_next_morph_id`: the next fresh morphism identifier.

- `pg_morphisms`: a list of morphism-ID/morphism-record pairs. Each `MorphismState` holds a source module, target module, coupling data, and an identity flag.

Remark 9.1. There is no separate `vm_heap`, `vm_output`, or `vm_imem` field in the Coq kernel's `VMState`. Heap operations are heap-relative accesses into `vm_mem`. Output and instruction transport live at the runner, trace, or RTL harness layer, not as kernel state fields. This distinction matters because NoFI is a theorem about the state record above.

Remark 9.2. The partition graph has two distinct layers. The *module layer* records how the state is decomposed into named pieces. The *morphism layer* records typed categorical relationships between those pieces. These layers are independent. Two machine states can agree on every module while differing in their morphism maps. This independence is what makes the Categorical Separation Theorem possible (Section 12).

10 The instruction set: all 47 opcodes

The Thiele Machine has 47 opcodes (the term "opcode" is in Section 2: the numeric tag the hardware decodes to choose which behaviour to run). Every one carries an explicit `mu_delta` parameter. Cost is part of the instruction. The machine reads it, the machine charges it.

10.1 Group 1: Partition operations (4 opcodes)

These opcodes modify the partition graph. They are how the machine builds structural knowledge.

PNEW(R, δ): allocate a new module.
Creates a fresh module and charges δ to μ. The instruction carries a region list R describing which memory addresses belong to this module. The current step rule normalises this to a standard form (the math sense of "canonical": a unique representative for a class of equivalent inputs): it keeps the size (how many addresses) and stores it as the range `0, 1, ..., sz-1`. So the checked step preserves the module size, not arbitrary input geometry. The bounded RTL has its own partition-table and active-module machinery for locality enforcement (locality, in this machine, is the constraint that an instruction may only access memory inside its own active module's region; Section 14 covers the table). The `PNEW` opcode itself is the graph allocation step.

Why does this exist? Because structural knowledge needs a home. Creating a module is how you tell the machine "this region of computation is a named unit I'm tracking." The cost δ is programmer-declared and can be 0; the machine does not force you to pay for allocation itself. If building this partition cost something because you had to figure out the right boundary, run a search, or derive the structure, you record that here. If it was bookkeeping, $\delta = 0$ is fine.

PSPLIT(*m*, *L*, *R*, *δ*): split one module into two.
Splits module m and charges δ to μ. The constructor still carries left and right region payloads, but the current hardware-aligned step ignores those payloads and calls `graph_hw_psplit`: the left side gets half the module size, and the right side gets the remainder. This is the refinement operation in the finite hardware model. If discovering the split deserves a cost, encode a positive δ; the canonical reversible-accounting policy keeps it at 0.

PMERGE(m_1, m_2, δ): merge two modules into one.
Merges modules m_1 and m_2 and charges δ. The hardware-aligned step concatenates the module sizes through `graph_hw_pmerge`; module IDs wrap modulo 64. It does not perform a rich invariant-compatibility check in this step. Like PNEW and PSPLIT, the machine does not enforce a minimum cost here. The mandatory cost comes later if you try to certify a claim about the structure.

PDISCOVER(*m*, *evidence*, *δ*): query partition structure.
The evidence payload is carried by the instruction, but the hardware-aligned relation does not attach axioms or mutate the graph. It advances the PC and charges δ. This is a placeholder for discovery accounting, not a register query in the current kernel step.

10.2 Group 2: Logic and certification (3 opcodes)

These opcodes interact with the Logic Engine, the on-chip finite-state machine that verifies formulas and certificates.

LASSERT(*freg*, *creg*, *kind*, *flen*, *cost*): assert and certify a formula.
Reads a formula whose declared encoded-word count is *flen* from memory starting at the address in register *freg*. The *kind* boolean flag selects which certificate path the Logic Engine should follow. "SAT" (satisfiable) means: I'm claiming this formula has at least one satisfying assignment, some combination of variable values that makes it true. "UNSAT" (unsatisfiable) means: I'm claiming it has no satisfying assignment, it's always false. In the on-chip semantics, *kind* = *true* is the SAT path: it requires a dual-witness certificate (one satisfying assignment and one falsifying assignment) so the validation actually proves the formula is non-trivially satisfiable. *kind* = *false* is the UNSAT-tag path: `lassert_check_ok` returns false unconditionally when *kind* = *false*, so any LASSERT issued with *kind* = *false* fails the check, traps to `LASSERT_TRAP_PC`, and sets the error flag, the μ-cost is still charged either way. The SAT-only certification path is the kernel's structural-proof shape; the UNSAT tag is a reserved opcode bit that the kernel routes through the trap. This matches the RTL FSM, which has phases 0 (idle), 1 (header read), and 2 (scan). The kernel and hardware both validate that the declared count matches the in-memory formula header before the check is allowed to succeed. A mismatch traps and sets the error flag. Reads a certificate block from memory starting at the address in register *creg*. That block contains two witnesses: one satisfying assignment and one falsifying assignment. The on-chip Logic Engine

FSM processes the formula and both witnesses in-place without external solver calls. Validation succeeds only if the declared count is honest, the first witness satisfies the formula, and the second witness falsifies it. That means the formula is not a tautology and really excludes at least one state. LASSERT itself is a checked validation step; certification channels are activated by the concrete state-changing paths such as CERTIFY and MORPH_ASSERT, while the abstract cert-address theorem treats LASSERT as part of the certification/revelation class. Charges $\mathit{flen} \times 8 + S(\mathit{cost})$ to μ, even when the check fails.

This is the most important logic opcode in the machine. It is where a constraint package is checked before any later certification step can lean on it. You cannot fake the pricing anymore by declaring a tiny *flen* for a large in-memory formula: if the header and the instruction disagree, the machine traps. You also cannot fake semantic narrowing with a tautology: the dual-witness requirement closes the tautology-inflation hole because there is no falsifying witness to show the formula excluded any state. The cost formula is still a spend ledger. The honest-length check and the non-triviality witness are what tie that spend to the real checked object.

LJOIN(*c1reg*, *c2reg*, δ): join two certificates.
Combines two memory-resident certificates into a single composite certificate. The addresses of the input certificates are in *c1reg* and *c2reg*. Charges $S(\delta) \geq 1$. This supports constructing complex multi-part proofs from simpler pieces.

MDLACC(m, δ): MDL cost accounting.
Charges δ as a module-discovery/model-description cost for module m. It does not read a register, and it does not mutate the graph.

Suppose you have two models that both fit the same data. One model is a ten-thousand-parameter neural network. The other is a five-line formula. Both fit the training set. Which one are you buying? The minimum-description-length principle says: the total bill is the model description itself plus the cost of encoding the data once you have the model. The description length of a model is how many bits you need to write it down, a simple formula takes very few bits; a massive network takes many. The short formula costs more to encode data with, but costs almost nothing to describe. The massive network achieves perfect in-sample compression, but its own description is enormous. The correct choice minimizes the sum of (bits to describe the model) + (bits to encode the data given the model).

Translated to the Thiele Machine vocabulary: a module structure that you built has a description-length cost. Not because someone told you it does, but because constructing a specific partition of state space may require information that should be billed. MDLACC is the opcode that lets a program explicitly enter that description cost into the ledger after the structure has been built. It performs no graph change. Its sole effect is to advance the program counter and charge the encoded δ. The connection to MDL

Opcode	What it does	Typical cost
`LOAD_IMM`(r, v, δ)	Write immediate value v into register r	0
`XFER`(r_{dst}, r_{src}, δ)	Copy register r_{src} to r_{dst}	0
`ADD`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a + r_b$ (modular 64-bit)	0
`SUB`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a - r_b$ (modular 64-bit)	0
`MUL`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a \times r_b$ (modular 64-bit)	0
`AND`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ AND r_b (bitwise)	0
`OR`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ OR r_b (bitwise)	0
`SHL`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ shifted left by r_b bits	0
`SHR`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ shifted right by r_b bits	0
`LUI`(r_d, v, δ)	Load upper immediate: $r_d \leftarrow v \ll 8$	0
`XOR_LOAD`(r_d, *addr*, δ)	Load from absolute memory address *addr* into r_d	0
`XOR_ADD`(r_d, r_s, δ)	$r_d \leftarrow r_d$ XOR r_s (XOR accumulate)	0
`XOR_SWAP`(r_a, r_b, δ)	Swap registers r_a and r_b (reversible, direct two-write exchange)	0 by convention
`XOR_RANK`(r_d, r_s, δ)	$r_d \leftarrow \text{popcount}(r_s)$ (Hamming weight)	0

is not a citation to a 1978 paper; it is the claim that the machine's natural μ-accounting can track what information-theoretic compression theory says a model costs, when the program chooses to enter that cost.

10.3 Group 3: Compute and data transfer (14 opcodes)

Standard arithmetic and data movement. These are the operations you find in every ISA. They mostly carry $\delta = 0$ because they do not introduce new structural knowledge. They just compute.

Why does the ISA have all these? Because the Thiele Machine is a complete computing machine, not just a structural accounting system. Programs need to compute intermediate values, manage data, and control flow. The compute group provides all of that. The key property: none of these opcodes touch the certification channels. They cannot create certified structural knowledge.

XOR_SWAP is a reversible register swap. The kernel implements it as a direct two-write exchange. The name suggests the classic XOR-trick sequence that some assembly programmers use to swap two values without a temporary register (it works by writing one register with the XOR of both, then the other, then the first again), but the kernel does not actually do the trick. It just writes both registers at once. It carries $\delta = 0$ by convention because a reversible operation, in the Bennett/Landauer sense (Section 16), does not destroy any information.

10.4 Group 4: Memory and control flow (8 opcodes)

Memory access and program flow control.

Opcode	What it does	Note
`LOAD`(r_d, r_a, δ)	Load `vm_mem`$[r_a]$ into r_d	Kernel register-indirect load
`STORE`(r_a, r_s, δ)	Store register r_s into `vm_mem`$[r_a]$	Kernel register-indirect store
`JUMP`(addr, δ)	Set PC to addr unconditionally	none
`JNEZ`(r, addr, δ)	Jump to addr if $r \neq 0$	none
`CALL`(addr, δ)	Push return address via stack pointer r15, jump to addr	Qed coverage
`RET`(δ)	Pop return address via stack pointer r15, jump there	Qed coverage
`HALT`(δ)	Advances PC and charges δ, like any other instruction. The runner stops when PC goes past the program; HALT inserted before the end is just a no-op.	runner-level
`CHECKPOINT`(ℓ, δ)	Accept label ℓ, advance PC	label outside kernel

At the kernel level, LOAD and STORE are register-indirect memory operations over `vm_mem`. At the RTL/cosimulation layer, the partition wall adds active-module locality checks. `INIT_PT` and `INIT_ACTIVE_MODULE` are harness setup commands for that finite hardware table, not VM opcodes and not fields of `VMState`. This is the honest split: the abstract ISA defines memory semantics; the bounded hardware layer enforces locality through its own table state.

10.5 Group 5: Revelation, I/O, heap, and tensor (7 opcodes)

This group mixes two fundamentally different kinds of operations. EMIT and READ_PORT are in the *certification/revelation cost class*, they cost ≥ 1 unconditionally and are subject to the same mandatory-floor rule as CERTIFY and MORPH_ASSERT. WRITE_PORT, HEAP_LOAD, and HEAP_STORE are ordinary I/O and storage, no mandatory floor. TENSOR_SET and TENSOR_GET manage per-module cost tensors. They share this group only by physical-channel adjacency; their cost semantics are distinct.

Opcode	What it does	Cost
`READ_PORT`(r, c, v, *bits*, δ)	Write value v into register r. The value v is embedded in the instruction encoding at assembly time (the kernel step is deterministic: it writes the declared v, not a runtime port read). Channel c and the *bits* count are used by the runner/environment layer for I/O orchestration and cost accountability.	$bits + S(\delta) \geq 1$
`WRITE_PORT`(c, r_s, δ)	Send register r_s to channel c outside kernel state	δ
`EMIT`(m, *payload*, δ)	Emit payload as a certified trace event	$\lvert payload\rvert_{\text{bits}} + S(\delta) \geq 1$
`HEAP_LOAD`(r_d, r_a, δ)	Load from `csr_heap_base + regs[`r_a`]` into r_d	δ
`HEAP_STORE`(r_a, r_s, δ)	Store r_s to `csr_heap_base + regs[`r_a`]`	δ
`TENSOR_SET`(m, i, j, v, δ)	Set per-module tensor entry (i, j)	δ
`TENSOR_GET`(r_d, m, i, j, δ)	Read per-module tensor entry (i, j) into r_d	δ

EMIT and READ_PORT live in the cert/revelation class but pay more than the $S(\delta)$ floor: they pay for every bit they touch. EMIT unfolds the payload into actual bits, charges those bits directly, then adds at least 1 for the certification step. A one-byte ASCII payload (8 bits, $\delta = 0$) costs 9. Ten bytes (80 bits, $\delta = 0$) costs 81. You cannot emit more for the same price as less. READ_PORT is the same shape on the way in: bits read, plus at least 1. The floor is still there. Zero-cost certification is still impossible. What this layer adds is that the cost tracks the actual bit content, not just the fact of the step.

TENSOR_SET and TENSOR_GET manage per-module metric tensor entries, storing the 4×4 geometric weight matrix used by the morphism algebra. The kernel tensor is a flattened 4×4 vector. The hardware has dedicated tensor state. Hardware and Coq agree step-for-step on both success and error paths: bad tensor indices latch error in both semantics; the success-path bisimulation requires $i, j < 4$ (and, for TENSOR_GET, the module to exist in the graph), both of which are direct argument-validity checks. Covered in `GraphReconstructionBridge.v` and recorded in `RTLGapRegistry.v`.

10.6 Group 6: Witness and certification (3 opcodes)

These opcodes accumulate binary game outcome statistics into hardware witness counters and manage the top-level certification flag.

CHSH_TRIAL(x, y, a, b, δ): record one CHSH trial.
Records one trial of the CHSH game. $x \in \{0, 1\}$ is Alice's measurement setting. $y \in \{0, 1\}$ is Bob's measurement setting. $a \in \{0, 1\}$ is Alice's outcome. $b \in \{0, 1\}$ is

Bob's outcome. The Coq constructor order is settings first (x, y), then outcomes (a, b). The instruction increments exactly one of the 8 WitnessCount counters: if $a = b$ (same outcome), it increments `wc_same_xy`; if $a \neq b$ (different outcome), it increments `wc_diff_xy`. There are 4 setting pairs $\times$ 2 same/different buckets = 8 counters. The CHSH score S is computed later from these counters. Charges δ to μ.

This is a Tier 0 operation: it records data but does not activate any certification channel. It is provably free in the sense that CHSH_TRIAL does not and cannot trigger a cert-insight event (proven in `WitnessInsightGeneral.v`, `chsh_trial_is_tier0`). One validation: if any of x, y, a, b is not in {0, 1}, for example, passing x=5, the machine fires a bad-bits error step: the error flag latches, no counter is incremented, but the cost is still charged. The machine does not silently count invalid inputs as valid trials.

CERTIFY(δ): top-level proof-carrying certification.
Sets `vm_certified = true`, charges $S(\delta) \geq 1$ to μ, advances PC. This is the simplest cert-channel activation: you are saying "I, the program, certify that this execution has reached an attested state." The flag is set. The cost is charged. There is no way to set `vm_certified = true` without this opcode, and this opcode always charges ≥ 1.

REVEAL(*m*, *bits*, *cert*, δ): disclose a value.
Reveals *bits* bits from module *m* with certificate string *cert*. A bit-commitment protocol is a two-phase scheme: first you commit to a value (seal it in an envelope), later you reveal it. The reveal step proves you didn't change your answer after seeing the other side's response. REVEAL is the disclosure half of that scheme. REVEAL charges bits $+ S(\delta)$: the number of bits being disclosed, plus at least 1. The cost is also recorded in the μ-tensor at the direction index encoded in the instruction, marking *where* in the morphism geometry the disclosure cost lands. Revealing 0 bits still costs at least 1. Revealing 32 bits costs at least 33. You cannot disclose more for the same price as less. This is what makes the revelation requirement concrete: obtaining attested cross-correlations from binary game trials requires an explicit disclosure instruction, and that disclosure is priced by how much you reveal.

MORPH_ASSERT (see Group 7): it activates the cert channel and belongs conceptually here too, but is listed in the morphism group because it operates on a morphism ID.

10.7 Group 7: Categorical morphism operations (7 opcodes)

These are the opcodes that operate on the categorical morphism layer of the partition graph. Six of them (MORPH, COMPOSE, MORPH_ID, MORPH_DELETE, MORPH_ASSERT, MORPH_TENSOR) modify graph state and `is_classical_opcode` returns false on them. MORPH_GET only reads graph state into a register and is classified as classical by that predicate, even though a Turing Machine still has to simulate the graph on its tape to evaluate it. A Turing Machine can simulate programs using these opcodes by encoding the full Thiele state on its tape.

The simulation can be honest (maintaining the μ-ledger cost semantics in the encoding) or dishonest (omitting that accounting). The TM substrate's transition relation is indifferent to which, which is exactly the substrate-versus-program distinction Section 3 names: A2 lives in the simulator program when the substrate runs the simulation, not in the substrate itself.

A *morphism* is a record saying: "module A is related to module B in this specific, named way." Not just "A and B exist." Not just "A and B are connected by an edge." A morphism has a source module ID, a target module ID, coupling data (a list of (source-cell, target-cell) pairs plus a label string), and an is-identity flag.

The coupling data is actual relational content. MORPH creates a morphism with an empty coupling list initially. COMPOSE computes the relational composition of two morphisms' coupling lists: pair (a, c) is included if and only if there exists b such that (a, b) is in the first coupling and (b, c) is in the second. This is proven correct in `CategoryBridge.v`. MORPH_TENSOR concatenates two coupling lists. The coupling is not a programmer tag. It is data the machine builds and proves properties about. You can read back the number of coupling pairs via MORPH_GET (selector 2).

Why does this matter? Because two programs can have identical classical state (same registers, same memory, same μ) while having completely different morphism maps. A classical machine cannot distinguish them. The Thiele Machine can, via a single morphism probe. This is the Categorical Separation Theorem (Section 12).

Opcode	Effect	Cost
`MORPH`(r_d, s, t, c, δ)	Create morphism $s \to t$, write ID into r_d. Empty coupling. (c carried but unused.)	δ
`COMPOSE`(r_d, m_1, m_2, δ)	Compose $m_1 : A \to B$ with $m_2 : B \to C$ (requires target of m_1 = source of m_2). Coupling = relational composition.	δ
`MORPH_ID`(r_d, m, δ)	Identity morphism on module m.	δ
`MORPH_DELETE`(id, δ)	Delete morphism id. Error if not found.	δ
`MORPH_ASSERT`(id, $prop$, $cert$, δ)	Assert morphism id exists with non-empty property $prop$. Sets `csr_cert_addr` = checksum($prop$) on success. Always charges $S(\delta) \geq 1$.	$S(\delta)$
`MORPH_TENSOR`(r_d, m_1, m_2, δ)	Tensor $m_1 \otimes m_2$: requires disjoint source/target regions. New morphism from union-source to union-target. Coupling = concatenation.	δ
`MORPH_GET`(r, id, sel, δ)	Read field (0=source, 1=target, 2=#coupling-pairs, 3=is-identity) of morphism id into r.	δ

MORPH_ASSERT deserves special attention. It charges $S(\delta) \geq 1$ regardless of whether

the assertion succeeds. This is the sharpest instance of No Free Insight: the *attempt to claim structural knowledge costs something*, even when the claim is false. A failed assertion is not free. A machine without a structural ledger has no way to express this constraint. In the Thiele Machine it is enforced by the step semantics and proven in `AbstractNoFI.v`.

10.8 Why 47 opcodes?

The number 47 comes from what the machine needs to do:

- Compute arithmetic and data: 14 ops (LOAD_IMM, XFER, ADD, SUB, MUL, AND, OR, SHL, SHR, LUI, XOR_LOAD, XOR_ADD, XOR_SWAP, XOR_RANK)
- Memory and control flow: 8 ops (LOAD, STORE, JUMP, JNEZ, CALL, RET, HALT, CHECKPOINT)
- I/O, heap, and tensor: 7 ops (READ_PORT, WRITE_PORT, EMIT, HEAP_LOAD, HEAP_STORE, TENSOR_SET, TENSOR_GET)
- Partition structure: 4 ops (PNEW, PSPLIT, PMERGE, PDISCOVER)
- Logic and certification: 3 ops (LASSERT, LJOIN, MDLACC)
- Witness and certification flags: 3 ops (CHSH_TRIAL, CERTIFY, REVEAL)
- Categorical morphisms: 7 ops (MORPH, COMPOSE, MORPH_ID, MORPH_DELETE, MORPH_ASSERT, MORPH_TENSOR, MORPH_GET)
- CHSH-aware certification: 1 op (CHSH_LASSERT)

$14 + 8 + 7 + 4 + 3 + 3 + 7 + 1 = 47$.

The 47th opcode: CHSH_LASSERT. `CHSH_LASSERT mu_delta` reads the eight buckets of `vm_witness` directly, computes the integer-arithmetic check `column_contractive_check_witness` (defined in `coq/kernel/foundation/VMStep.v` using only Z arithmetic on the same/diff counts), and either advances the program counter on success or traps to `LASSERT_TRAP_PC` and latches `vm_err` on failure. The cost is $S(\textit{mu\_delta}) \geq 1$ regardless of outcome (cert-setter discipline; you pay to check, even to fail). The bridge theorem `chsh_lassert_no_trap_implies_quantum_realizable` (`coq/kernel/quantum/QuantumPartitionPSD.v`) proves that any no-trap, no-err-flip execution of CHSH_LASSERT implies the witness-derived NPA moment matrix is PSD (i.e., quantum-realizable). The chain in plain order: the integer check on the witness counters passes; that fact lifts to real-valued column-contractivity by `column_contractive_check_witness_sound` in `coq/kernel/nfi/MuLedgerQuantumBridge.v`; column-contractivity is biconditional with NPA-PSD by `column_contractive_iff_quantum_realizable`. Three theorems. No physics input. The opcode is the place in the kernel where the algebra of Section 15 becomes something the machine actually refuses to do.

Every opcode exists for a reason. None are redundant. If you removed CERTIFY, you could not activate `vm_certified`. If you removed MORPH, you could not build morphism relationships. If you removed CHSH_TRIAL, you could not track 2-player binary game statistics. If you removed CHSH_LASSERT, the kernel would record CHSH trials but could not enforce that certified outcomes lie inside the NPA-PSD boundary. The ISA is designed, not grown.

11 What a categorical morphism is: the category theory, from scratch

Category theory from scratch. I didn't know what it was when I started this project.

11.1 The idea of a category

A category is a collection of two things: objects and arrows between them.

The objects can be anything. In abstract math, they might be sets, or vector spaces, or topological spaces. In the Thiele Machine, the objects are modules (partitions of state).

The arrows are also called morphisms. An arrow goes from one object to another: $f : A \to B$ means "there is an arrow f from object A to object B." The arrow does not have to be a function. In the Thiele Machine, a morphism just says "there is a relationship of this type between these two modules."

Two laws must hold:

1. **Composition**: If $f : A \to B$ and $g : B \to C$, there must exist a composite $g \circ f : A \to C$. This is what COMPOSE implements.
2. **Identity**: For every object A, there must exist an identity arrow $\mathrm{id}_A : A \to A$. This is what MORPH_ID implements.

And composition must be associative: $(h \circ g) \circ f = h \circ (g \circ f)$.

Why composition? Because you want to chain relationships. If A relates to B and B relates to C, you want to say A therefore relates to C, transitively, without having to name every possible A-to-C path individually. Without composition, the structure collapses into a flat list of unconnected pairs. It's entirely useless for multi-hop reasoning.

Why identity? Because every object needs a minimal self-relationship. An isolated module with no connections to anything else still needs to be expressible in the framework. MORPH_ID is how you say "module A relates to itself with no coupling content." Without it, isolated objects fall outside the framework entirely, and any module with zero external relationships becomes unrepresentable.

Why associativity? Because the parenthesization of a chain should not matter. Whether you write $(h \circ g) \circ f$ or $h \circ (g \circ f)$, both mean the same three-hop path $A \to B \to$

$C \to D$. If associativity failed, two programs that build the same chain in different groupings would produce different morphisms. That is a bug: the chain is the thing, not how you assembled it.

These laws are proven in Coq in `CategoryLaws.v`. Concretely: morphisms carry coupling data, a list of (source-cell, target-cell) pairs plus a label. A "cell" is a natural number indexing into a module's memory region. A coupling pair (a, c) says: cell a in the source module is related to cell c in the target module. The coupling is the *relational content* of the morphism, the actual evidence of which parts of one module correspond to which parts of another. COMPOSE produces a new morphism whose coupling pairs are the relational composition of the two inputs: (a, c) is included when there exists an intermediate cell b such that (a, b) is in the first coupling and (b, c) is in the second. This is the standard relational composition (like matrix multiplication but for sets of pairs). This is proven in `CategoryBridge.v` (`graph_compose_morphisms_coupling`). The Thiele Machine's categorical layer is not just called a category; it satisfies the actual axioms on the actual coupling data.

11.2 Why modules as objects

In the Thiele Machine, the objects of the category are the modules of the partition graph. A module is a named piece of the computation: it owns a region of memory or state, it may carry certified axioms about that region, and it contributes a local μ cost.

A morphism between two modules says: "there is a named relationship between module A and module B, with explicit coupling data." The coupling data is a list of (source-cell, target-cell) pairs. COMPOSE computes the relational composition of two couplings: pair (a, c) is in the composed coupling exactly when there exists b such that (a, b) is in the first coupling and (b, c) is in the second. This is proven correct. The machine tracks it. Via MORPH_GET (selector 2), you can read back the number of coupling pairs in any morphism.

What the coupling pairs represent is up to the programmer. They could encode:

- Module B refines module A (which cells in A map to which cells in B)
- Module A entangles with module B (which cell-pairs are correlated)
- Module A simulates module B (which transitions/outputs correspond)

The machine enforces the relational composition correctly. The interpretation is yours.

11.3 Why this is not just "extra state"

Here is the question I asked myself for a long time: if you can just add extra registers to a classical machine to store morphism IDs, why is the categorical layer special?

The answer is in the MORPH_ASSERT opcode and in the Categorical Separation Theorem.

Extra registers do not have provenance. You can write anything into a register. You can write 42 into register 5 and claim it is a morphism ID. But the Thiele Machine checks: did a MORPH instruction actually run that created morphism 42 with the source and target you claim? If not, MORPH_ASSERT fails. And MORPH_ASSERT costs $S(\delta) \geq 1$ even if it fails.

Building the morphism chain by calling MORPH, COMPOSE, and MORPH_ID is free when those instructions carry $\delta = 0$. Those ops don't have a mandatory positive floor. What costs is MORPH_ASSERT: the act of asserting that the morphism exists and activating the cert channel. That always costs $S(\delta) \geq 1$. You can build the chain for free, but you cannot certify it for free.

This is why the morphism layer is not "extra state." It is *provenance-bearing state*. The machine knows where it came from. A classical machine cannot fake this at the same cost. That is the Categorical Separation Theorem.

12 Categorical separation: the machine is strictly richer

Section 3 established the substrate distinction: Thiele's step relation enforces A2 by construction; the Turing machine's transition table has no slot for it. That is the load-bearing result. This section gives the within-model illustration of that distinction: a pair of Thiele states whose classical projection collides but whose morphism graphs differ. The witness alone is not load-bearing on its own. Any computational model with a non-identity projection produces an analogous within-model witness; the README at *Why It Is Not Just A Toy Counter* flags this. What anchors the witness in the rest of the argument is the substrate-level claim from Section 3 plus the cost-floor theorem `universal_nfi_any_substrate` and the uniqueness theorem `mu_is_initial_monotone`. Read the construction below as: here is what the substrate distinction looks like inside the model, concrete enough that you can object to a specific piece. Read Section 3 first if you skipped it; the witness is downstream of that section, not a replacement for it.

Theorem 12.1 (Categorical Separation)**.** *There exist states s_1 and s_2 such that:*

- $s_1.$`vm_regs` $= s_2.$`vm_regs` *(identical registers)*
- $s_1.$`vm_mem` $= s_2.$`vm_mem` *(identical memory)*
- $s_1.$`vm_mu` $= s_2.$`vm_mu` *(identical μ)*
- $s_1.$`vm_err` $= s_2.$`vm_err` *(identical error flag)*
- $s_1.$`vm_pc` $= s_2.$`vm_pc` *(identical program counter)*
- $s_1.$`vm_certified` $= s_2.$`vm_certified` *(identical top-level certification flag)*

yet their morphism graphs differ. In the concrete witness, one state has an identity morphism in `pg_morphisms`*; the other has no morphisms.*

Construction. Build s_1 with `pg_morphisms = [(0, id)]`. Build s_2 with `pg_morphisms = []`. Set every classical field the same: registers, memory, μ, PC, error flag, and `vm_certified`. The equality proof is by reflexivity on those fields. The graph distinction is by list constructor mismatch: a one-element morphism list is not the empty list. □

The two witness states side by side. What the classical observer sees, and what the structural layer actually contains.

	State s_1	State s_2
`vm_regs`	[]	[]
`vm_mem`	[]	[]
`vm_mu`	0	0
`vm_pc`	0	0
`vm_err`	false	false
`vm_certified`	false	false
`pg_morphisms`	[(0, id)]	[]

What this witness shows. The two states above are not contrived. They are the simplest possible classical-shadow collision under the Thiele projection: identical registers, identical memory, identical PC, identical error flag, identical μ, identical `vm_certified`. Every observer restricted to the six-field projection says they are the same state. The morphism graph says they are not. This is a witness inside the model that the chosen projection drops information. It is not, on its own, a substrate-comparison theorem: an analogous projection on any other substrate reproduces the same shape. What anchors this in the rest of the argument is the cost floor and uniqueness theorems Section 3 subsection 2 lists.

What the witness does not prove on its own. A reader who tries to escape this witness by adding a register, a tape region, or any auxiliary slot to a Turing machine reproduces the witness rather than escaping it. The augmented machine still has a non-identity projection (drop the added slot from the projected fields) and the same collision shape on the augmented states. The shadow lemma is collapsible by construction.

The deeper question (whether you can extend a Turing machine with a cost-tracking instruction so the substrate itself enforces A2) is the substrate-versus-program distinction. Section 3's subsection "The structural axis is canonical, not orthogonal" makes this precise once and explicitly. The short version: a Turing machine with extra tape and a longer instruction set is still a Turing machine. A buggy program on the augmented TM can still flip the cert bit without writing the cost increment. The kernel theorems require the cost-on-certification rule to live in the substrate's step function, not in the running program; the Turing substrate does not satisfy that requirement no matter how the program above it is written.

Connecting to the universal NoFI. Combine this within-model witness with `universal_nfi_any_substrate` and `thiele_represents_simulating_cert_system` (both in `coq/kernel/nfi/UniversalCertificationCost.v`) and you get the substrate-level statement: any certification system that satisfies A2 admits a faithful embedding into the Thiele VM, under which its certifications become Thiele certifications and its cost lower-bounds the embedded Thiele cost. The witness theorem shows what the Thiele projection drops within the model. The substrate-independent theorem shows that any honest substrate inherits the μ-price on certification. Together they say: the cost floor is forced wherever A2 is accepted, and Thiele is the canonical place where the dropped information sits as primitive state.

Falsifier. If you produce a function on the classical fields (`vm_regs`, `vm_mem`, `vm_mu`, `vm_pc`, `vm_err`, `vm_certified`) that returns different values on s_1 and s_2 above, `shadow_strictly_lossy` is wrong and the Coq proof in `coq/kernel/witness/ShadowProjection.v` won't compile. That is what falsifies this section.

Theorem 12.2 (Shadow Separation)**.** *There exist two states with the same classical shadow but different morphism graphs, and a legitimate graph-preserving probe after which the morphism-level distinction remains.*

Proof. Take s_1, s_2 to be the Categorical-Separation witness pair: s_1 has `pg_morphisms` $= [(0, \mathrm{id})]$, s_2 has `pg_morphisms` $= []$, and the six classical-shadow fields agree. So `shadow_proj`(s_1) = `shadow_proj`(s_2) but s_1.`vm_graph` $\neq$ s_2.`vm_graph`.

The probe is the single-instruction program [`instr_add 0 0 0 0`] (`ADD` with $\delta = 0$). This instruction is classical: `is_classical_opcode(instr_add 0 0 0 0)` = true. By D3 conservativity, the probe leaves `vm_graph` untouched on both s_1 and s_2. After the probe, the six classical-shadow fields agree (because they agreed before and the probe is deterministic on the shadow) and the morphism graphs still differ. □ □

Plain reading. Same witness pair as Categorical Separation. The probe is a classical opcode that touches only the classical shadow. D3 says it leaves the morphism graph alone. So the graph-level difference survives the probe. The point: the difference between s_1 and s_2 is not an artifact of the initial state; it persists through ordinary classical computation.

The probe used in `ShadowProjection.v` is deliberately boring: `ADD 0 0 0 0`. It is not a morphism operation. That is the point. The theorem is not relying on a failing probe to manufacture a difference. The difference is already in the structural state, and ordinary classical projection cannot see it.

Corollary 12.3 (Classical observer is blind, `ShadowProjection.v`)**.** *No function of (registers, memory, μ, PC, error, certified) can separate all Thiele states. The classical shadow is strictly lossy.*

Corollary 12.4 (Thiele strictly extends classical, `TuringStrictness.v`)**.** *Every program in the classical fragment, programs restricted to `is_classical_opcode`, meaning no graph mutations, no cert-channel ops, no witness ops, leaves `pg_next_id` unchanged. There exist Thiele programs that change `pg_next_id` (via PNEW). Therefore Thiele strictly extends the classical fragment: it can represent and probe states that no program in that fragment can reach. A Universal Turing Machine can simulate the full Thiele state by encoding it on tape; what it cannot do is produce that state without expending the equivalent structural work, the μ-ledger cost is preserved under any faithful simulation.*

The classical machine simulates inside the Thiele Machine faithfully (every classical program runs correctly, no side effects on the structural layer). But the Thiele Machine has programs that escape the classical fragment. This is proven, not claimed.

13 The knowledge receipt: what makes certification unforgeable

Four concrete facts about why the μ ledger is unforgeable.

Act 1: The attempt costs, even when it fails.

Run MORPH_ASSERT(k, "test", "", 0) where morphism k does not exist. What happens:

- `vm_err` becomes true (the assertion failed, morphism k not found).
- `csr_cert_addr` stays 0, the cert channel is NOT activated on failure.
- μ increases by $S(0) = 1$ regardless (the attempt cost something).

The machine paid for a failed assertion. A machine without a structural ledger has no way to express this. In the Thiele Machine, the cost of claiming knowledge is charged whether or not the claim is true.

Act 2: Build the chain.

Run this sequence:

$$\text{PNEW}(A),\ \text{PNEW}(B),\ \text{PNEW}(C)$$
$$\text{MORPH}(f, A, B),\ \text{MORPH}(g, B, C)$$
$$\text{COMPOSE}(h, f, g)$$

Then MORPH_GET(r_0, h, source) and MORPH_GET(r_1, h, target). The resulting state has $r_0 = A$, $r_1 = C$, and morphism $h : A \to C$ exists in the morphism map. With $\delta = 0$ on each build step, $\mu = 0$. The morphism exists, but no structural cost has been paid yet.

Act 3: Certify.

Run MORPH_ASSERT(h, "composed", "", 0). Morphism h exists and the property string "composed" is non-empty. The assertion succeeds. `csr_cert_addr` goes to the ASCII checksum of "composed" (nonzero). Tier 1 cert channel activated. μ goes up by $S(0) = 1$.

(Why must the property string be non-empty? Because csr_cert_addr stores the ASCII checksum of the property string. If property = "", checksum = 0, and the channel stays at 0, not activated. Any non-empty string works; "composed" is just an example label.)

The machine now holds a μ-ledger entry: at this point in the trace, a morphism assertion was made and paid for. The receipt is in the ledger. It cannot be removed. It cannot be backdated. μ never goes down.

Act 4: Classical programs cannot forge it.

Snapshot both programs at the end of Act 2, before Act 3's MORPH_ASSERT. Program B takes a shortcut: it loads $r_0 = A$ and $r_1 = C$ via LOAD_IMM. Classical state is identical: same registers, $\mu = 0$ on both sides (Program A hasn't asserted yet either). No morphism h was ever created in Program B.

Apply MORPH_DELETE(h): Program A (Act 2) succeeds. Program B fails and sets err.

The μ ledger is not a counter that measures how much work was done. It is a record of which structural commitments were made. Program B did the same amount of "work" (in the μ sense) but did different work. The machine knows the difference.

14 How the machine is built: three layers

The Thiele Machine exists in three forms. The three layers are tied together by formal bisimulation proofs at the Coq-kernel-to-Kami-RTL boundary (Section 2 defines bisimulation: two systems produce the same output on the same input, field by field) and by an audited parity-test boundary at the Coq-kernel-to-OCaml-extracted-runner boundary (Section 2 defines a parity test: running both implementations on the same input and checking the results match, on a battery of test inputs). The kernel-to-RTL bridge is proof; the kernel-to-OCaml bridge is tested correspondence. Both boundaries are documented explicitly.

14.1 Layer 1: The Coq kernel

This is the mathematical ground truth. The Coq kernel defines:

- The VMState record (all 12 fields, as in Section 9).
- The `vm_instruction` type: an inductive type (Section 2) with 47 constructors, one for each opcode, each carrying the right argument types.

- The `vm_apply` function: takes a state and an instruction, returns a state. It is total (Section 2): every input pair produces a defined output, including the error case.
- All the theorems: No Free Insight, μ-monotonicity, μ-initiality, categorical separation, Turing strictness, and the rest.

The Coq kernel operates mostly over natural numbers, but the machine state is not an unbounded mathematical fantasy. It fixes `REG_COUNT = 16` registers and `MEM_SIZE = 128` memory words in the synthesised RTL, and its accessors wrap indices through those sizes (the wrap is modular arithmetic from Section 2). Both constants are parametric in the proofs (Section 2 again: the same proofs work for any value of the constant), so nothing in the formal chain depends on the specific numbers. Register and memory writes are truncated through the kernel's 64-bit word helpers to keep the math clean. The μ counter is a plain natural number with no fixed upper bound in the kernel. The RTL target instantiates a finite 32-bit datapath (the part of the chip that moves and operates on data, as opposed to the control logic) and bounded tables.

Zero Admitted means: no step in any theorem proof was asserted without derivation. The machine checked everything. I did not.

14.2 Layer 2: The OCaml extracted runner

Coq can automatically extract executable code from a Coq development. The language it targets here is OCaml, a general-purpose programming language. "Extraction" means: Coq keeps the computational content of the definitions and throws away the proof terms. A function like `vm_apply` (which defines what every opcode does) extracts to an OCaml function that computes the same thing. The extracted result is not a translation of the proofs, the proofs are erased. What remains is the algorithm. The extracted runner executes all 47 opcodes from the Coq semantics, including the morphism operations and CHSH_LASSERT.

The trust boundary at this layer is named explicitly. `coq/kernel/hardware_bridge/OCamlExtractionBridge.v` proves formal facts about the Coq-side observable (`shadow_to_eo (vm_apply s i)`, which strips a state down to its observable fields after one instruction): exact μ accounting, μ never decreases, and the function is total. The file also names the external runner boundary. The Coq lemma called `ocaml_runner_agrees` is a Coq-side reflexive witness (a proof that one Coq expression equals itself, useful as a placeholder), not an unproved `Hypothesis` (Section 2). The actual cross-language claim, that the built OCaml binary returns the same observable as the Coq semantics on every input, is checked by the parity test suite (Section 2: run both implementations on the same battery of inputs and verify they agree). I do not count that as a kernel theorem. It is an audited boundary backed by tests.

14.3 Layer 3: The Kami RTL hardware

The Kami framework lets you write hardware specifications in Coq and extract them to Bluespec SystemVerilog. Bluespec SystemVerilog is a high-level hardware description language: you describe what the hardware should compute, including the registers, the rules, and the conditions under which state transitions fire. The Bluespec compiler generates standard Verilog RTL. Verilog RTL is the low-level gate-and-register description that synthesis tools can compile into actual logic gates on an FPGA (a reconfigurable chip you can program after manufacturing) or an ASIC (a custom chip manufactured for a specific design, not reprogrammable). The Thiele Machine hardware is defined in `ThieleCPUCore.v` (Kami module) and extracted through this pipeline.

The hardware has:

- 16 registers, each 32 bits wide (`RegCount = 16`; kernel proofs parametric)
- 128-bit instruction encoding (`InstrSz = 128`)
- 128 words of data memory and 128 instruction words (`MemSize = 128`; kernel proofs parametric in `MEM_SIZE`)
- bounded instruction transport and execution state
- 64 partition module slots (`PTableSz = 64`)
- 16 slots in each of the morphism, coupling, formula, certificate, descriptor-metadata, and coupling-pair tables
- An on-chip LASSERT FSM for formula verification (with 64-entry inlined backing buffers)
- The μ ledger updated atomically with each executed instruction (no separate parallel datapath; the cost arithmetic happens inside the same `step` rule)

The Verilog is the design. The FPGA part is whatever I could put it on for free. I picked the Xilinx Artix-7 `xc7a35tcsg324-1` on the Arty A7-35T because that is what the fully open-source toolchain supported out of the box. Section 2 explains the synthesis pipeline I use: yosys does the gate-level synthesis, nextpnr-xilinx does the place-and-route, and prjxray's `xc7frames2bit` produces the bitstream the chip reads to configure itself.

The CI workflow at `.github/workflows/ci-full.yml` runs both halves on every relevant push. One job produces a JSON netlist plus a simulation VCD waveform from iverilog (all three terms in Section 2); the other job produces a real `.bit` file loadable on the FPGA. Running yosys against the committed RTL today gives a resource breakdown: a few thousand LUTs, a few thousand flip-flops, a small number of carry chains, two block RAMs (the FPGA-primitive vocabulary is in Section 2; the exact numbers are 14,643 LUTs, 3,582 flip-flops, 173 carry chains, 64 distributed RAMs,

2 block RAMs of 36 Kbit each, 0 DSP slices). The design fits the part with comfortable margin.

The kernel proofs are parametric in `REG_COUNT` and `MEM_SIZE`, so the same proofs apply at any size constants the design picks. This part was a proof of existence on hardware, not a design constraint. The size constants (16 registers, 128 data words, 128 instruction words) were chosen small enough to fit on a free-toolchain FPGA. The design scales to whatever size you actually want.

All 47 opcodes are implemented in the RTL design: `ThieleCPUCore.v` has hardware logic for all of them. The question is not which opcodes are in hardware, but which opcodes have a completed formal *bisimulation proof*.

A bisimulation proof says: given the same starting state and the same instruction, the hardware and the Coq model produce the same output state. Not "similar" or "equivalent in some abstract sense." Literally: the hardware's answer equals the Coq model's answer, field by field. It is a step-by-step correspondence, verified in Coq.

Formal bisimulation proofs exist for all 47 opcodes (zero Admitted, all Qed). Here is the precise breakdown:

- **31 opcodes are truly unconditional.** These are the `SupportedOpcode` group: `True` in Coq, no conditions whatsoever. They cover all standard compute, load/store, jump, I/O, and certification operations that don't involve morphism tables or formula checking, plus CHSH_LASSERT (whose snapshot-level check reads the same witness counters and runs the same `column_contractive_check_witness` function as `vm_apply` via `abs_phase1`). Verified in `EmbedStep.v` (`embed_step_compute`).
- **6 opcodes require valid instruction arguments.** The hardware and Coq model AGREE on the error path for invalid arguments, so no semantic disagreement exists, but the success-path proof requires the argument to be in range:
  - CALL: stack pointer must be in bounds (`WellFormedSnapshot`), and `pc` $<$ `MEM_SIZE`.
  - RET: stack pointer must be in bounds (`WellFormedSnapshot`).
  - CHSH_TRIAL: all four values x, y, a, b must be in $\{0, 1\}$. Passing 5 as an outcome value triggers the bad-bits error path, on which both hardware and VM agree.
  - TENSOR_SET: indices i, j must both be < 4 (valid for a 4×4 tensor).
  - TENSOR_GET: indices valid AND the module must exist in the graph.
  - LASSERT: the declared formula-unit count *flen* must match the actual in-memory formula header. (Mismatch causes a trap to 0xF00, on which both sides agree; the precondition is needed for the success-path proof.)

Covered in `EmbedStep_WF.v` (CALL, RET, CHSH_TRIAL) and `GraphReconstructionBridge.v` (TENSOR_SET, TENSOR_GET, LASSERT).

- **10 opcodes carry Qed proofs under structural invariants that hold inductively from hardware reset.** Seven from the morphism group (MORPH, MORPH_ID, MORPH_DELETE, MORPH_ASSERT, MORPH_GET, COMPOSE, MORPH_TENSOR) and three graph-shaping opcodes (PNEW, PSPLIT, PMERGE). The preconditions are `coupling_wf` (= `coupling_desc_bounded` ∧ `coupling_pairs_in_range` ∧ `coupling_pairs_fully_populated`) and `morph_table_wf`. Both invariants are proved to hold at hardware reset and to be preserved by every `kami_step` operation: `coupling_wf_kami_step_preserved` and `morph_table_wf_kami_step_preserved`. So the precondition is discharged in practice for any state reachable by machine execution; no unreachable-state caveat applies. PNEW/PSPLIT/PMERGE additionally need `pt_well_formed` and arithmetic side conditions that are likewise inductive.

The official classification (`coq/kami_hw/RTLGapRegistry.v`, theorem `rtl_coverage_partition`) records the partition $37 + 10 + 0 = 47$, all Qed, zero structural gaps. `rtl_gap_count = 0` witnesses the empty gap registry.

The 47/47 proof coverage (all Qed) is the correct relationship between an abstract ISA and its physical instantiation. The abstract model defines what is correct; the hardware implements all of it; all 47 opcodes have formal bisimulation proofs. This is not a deficiency. It is honest proof accounting.

The abstract Thiele Machine is the Coq kernel. The hardware is a finite physical instantiation with explicit table sizes and word widths. The proofs state the bridge between them under the finite representation invariants where those invariants are needed. This is the only honest relationship physical hardware can have with an abstract mathematical model.

Substrate-level vs program-level enforcement. In the Coq kernel, A2 is part of the step relation. A2 says: flipping certification false-to-true costs at least 1, at the step level. Look at the cases for CERTIFY and MORPH_ASSERT in `vm_apply`: the cost is added before the flag flips; the step that mutates the cert channel and the step that charges μ are the same step. That is what A2 means in the kernel.

A Turing machine simulating the Thiele Machine has A2 only as a property of its simulator program. A buggy simulator that skipped the cost increment would not satisfy A2; the Coq proof does not cover such a simulator. The proof covers the step relation in the kernel and, under the BSC compiler trust boundary, the Kami RTL extracted from it. This is not a metaphysical claim that substrate-level enforcement is deeper than program-level enforcement. It is a precise statement of what the proof covers: the kernel definition and the synthesised RTL, with documented trust

boundaries at the OCaml extraction and the BSC compiler.

Substrate-level property enforcement is not unique to Thiele as a category of design. Trusted Platform Modules enforce key isolation in hardware; CHERI enforces capability discipline in the ISA itself. Both put the property the system cares about into the substrate's primitive operation rather than relying on program-level discipline to maintain it. What Thiele enforces is different from either: cost-on-certification, μ as an unforgeable receipt. The shared structural feature with TPMs and CHERI is that the enforcement lives in the substrate's step, not in the data the step operates on. If a TPM is the right analog for one piece (a hardware-rooted invariant a program cannot forge) and CHERI is the right analog for another (an ISA-level discipline programs cannot escape), the analogy is structural, not detailed; I'm not claiming the underlying mechanisms match, only the substrate-level shape of what is being enforced.

15 Binary Game Statistics: where the μ-Ledger Meets the NPA Boundary

The Thiele Machine has an opcode called CHSH_TRIAL that records outcomes of a 2-player binary game into hardware witness counters, and a kernel-level enforcement opcode CHSH_LASSERT that refuses to certify the trace unless those counters satisfy a Z-arithmetic check. I used the pair to stress-test the μ-ledger's cost algebra against a known algebraic boundary: the NPA moment matrix conditions for bounding what correlations are achievable under polynomial constraints.

I did not set out to find what I found. The chain is concrete. The Z-arithmetic check the kernel runs on the witness counters (decidable, integer-only) is sound with respect to the real-valued column-contractive predicate by `column_contractive_check_witness_sound` (188 lines, closing with `nra` after IZR distribution and field simplification, in `coq/kernel/nfi/MuLedgerQuantumBridge.v`). That real-valued predicate is biconditional with PSD of the zero-marginal NPA moment matrix by `column_contractive_iff_quantum_realizable` (in `coq/kernel/quantum/QuantumPartitionPSD.v`). End-to-end: a successful CHSH_LASSERT step entails NPA-PSD on the witness-derived correlators, by `chsh_lassert_no_trap_implies_quantum_realizable` in the same file. Zero physics axioms. The bridge is a constructed chain of Coq proofs, not algebraic coincidence.

This section builds the CHSH game from scratch so that result makes sense.

15.1 The Bell test setup, from scratch

Imagine two players, Alice and Bob, in separate rooms. They cannot communicate once the game starts. A referee sends Alice one of two questions ($x \in \{0,1\}$). The referee sends Bob one of two questions ($y \in \{0,1\}$). Alice answers $a \in \{0,1\}$. Bob answers $b \in \{0,1\}$. (Settings (x,y), outcomes (a,b) — same convention as Section 9 and the Coq.)

The game rule: they want $a \oplus b = x \wedge y$. The symbol $\oplus$ is XOR (exclusive OR): $a \oplus b = 1$ when $a \neq b$, and 0 when $a = b$. The symbol $\wedge$ is AND: $x \wedge y = 1$ only when both $x = 1$ and $y = 1$, otherwise 0. In plain language: their answers should be different if and only if both questions were 1. For all other question combinations (both 0, or one each), they should match.

Why 75% is the classical ceiling. Before the game, Alice and Bob can agree on any deterministic strategy: a function $a(x, \lambda)$ for Alice and $b(y, \lambda)$ for Bob, where λ is any shared randomness they agree on in advance. Once the game starts, no communication.

Fix any deterministic strategy and freeze λ. Then Alice has two predetermined answers, a_0 and a_1, and Bob has two predetermined answers, b_0 and b_1. Winning all four question combinations would require:

$$a_0 \oplus b_0 = 0, \quad a_0 \oplus b_1 = 0, \quad a_1 \oplus b_0 = 0, \quad a_1 \oplus b_1 = 1.$$

Now XOR the four left sides. Every variable appears twice, so the total is 0. XOR the four right sides and the total is 1. That is impossible. So every deterministic local strategy loses at least one of the four cases, and the best deterministic strategy wins at most 3 out of 4 question combinations.

Randomized strategies cannot do better: they are just mixtures of deterministic strategies, and averaging strategies that each win 75% at most gives you 75% at most. This is not a vague claim. It is a direct consequence of linearity of expectation.

So: maximum classical win rate = 3/4 = 75%. Proven by exhaustion in `coq/kernel/foundation/ClassicalBound.v:165` (zero Admitted): an executable trace using only PNEW, PSPLIT, and CHSH_TRIAL with $\mu = 0$ achieves exactly this, and `coq/kernel/quantum/MinorConstraints.v:822` (`local_box_CHSH_bound`) proves no $\mu = 0$ program can exceed it.

15.2 The CHSH inequality: where it comes from

Now here is what took me a while to understand: why is there a specific algebraic combination S that detects whether correlations can be explained classically?

The key move is this. For any fixed shared randomness λ, Alice's output $A_x(\lambda) \in \{-1, +1\}$ (I'm using ± 1 encoding now, not 0/1, indexed by Alice's setting x) and Bob's $B_y(\lambda) \in \{-1, +1\}$ (indexed by Bob's setting y). Consider the expression:

$$A_0(\lambda)B_0(\lambda) + A_0(\lambda)B_1(\lambda) + A_1(\lambda)B_0(\lambda) - A_1(\lambda)B_1(\lambda)$$

Factor it:

$$= A_0(\lambda)(B_0(\lambda) + B_1(\lambda)) + A_1(\lambda)(B_0(\lambda) - B_1(\lambda))$$

Since B_0 and B_1 are each ± 1, either $B_0 + B_1 = \pm 2$ and $B_0 - B_1 = 0$, or $B_0 + B_1 = 0$ and $B_0 - B_1 = \pm 2$. The two terms cannot both be large. In either case, $|B_0 + B_1| + |B_0 - B_1| = 2$, and since $|A_0|, |A_1| \leq 1$, the entire expression is at most 2 in absolute value for any fixed λ.

Average over λ:

$$S = \langle A_0 B_0 \rangle + \langle A_0 B_1 \rangle + \langle A_1 B_0 \rangle - \langle A_1 B_1 \rangle$$

where $\langle A_x B_y \rangle$ is the correlation (expected value of the product) when settings are x and y. Since the expression before averaging was bounded by 2 for every λ, the average satisfies $|S| \leq 2$.

That is the argument. The combination with the minus sign is chosen exactly because the factoring trick works: one of the B-terms goes to zero and the other to ± 2, forcing the whole thing below 2. The inequality $|S| \leq 2$ is not a coincidence. It is a pure algebraic consequence of the outputs being ± 1 and being determined locally.

This is proven in Coq in `coq/kernel/quantum/MinorConstraints.v:822` (zero Admitted, `local_box_CHSH_bound`). The formal statement: for any factorizable box B, $|S(B)| \leq 2$. `coq/kernel/foundation/ClassicalBound.v:165` (`classical_bound_achieved`) gives the matching constructive witness: an executable trace at $\mu = 0$ reaching $|S| = 2$. Combined, the two say the classical bound is tight: $\max\{|S| : \mu = 0\} = 2$ exactly.

15.3 The quantum bound: why $2\sqrt{2}$?

Quantum mechanics allows $|S|$ to reach $2\sqrt{2} \approx 2.828$. That number is the Tsirelson bound, after Boris Tsirelson, who first wrote down the inequality $|S| \leq 2\sqrt{2}$ for quantum correlations. The physicists' derivation of *why* that specific number bounds quantum strategies involves Hilbert-space operators, anticommutators, the Pauli matrices, and the Bloch sphere. I will not reproduce that derivation here: it is a physics derivation, and the formal Coq content of this section does not depend on it.

What the formal content does depend on, and what I will derive in the next subsections, is the algebra. The Coq theorem about quantum realizability (`TsirelsonQuantumModel.v`) takes "the correlation is quantum-realizable in the zero-marginal NPA framework" as a premise (Section 2 builds the term) and concludes $|S| \leq 2\sqrt{2}$. The premise is the physics input; the conclusion is pure algebra. Section 15's last subsection then builds the same bound from polynomial inequalities on the NPA moment matrix without any quantum-physics premise at all. That is the path I follow, and that is the part I am claiming.

The gap between the classical bound 2 and the Tsirelson bound $2\sqrt{2}$ is real, observed in physical experiments, and excludes every classical local strategy. The physics derivation of *why* the number is $2\sqrt{2}$ lives in the quantum-mechanics literature. The question this section answers is why the Thiele Machine respects the bound, and the polynomial-arithmetic derivation below gives the answer: the same number falls out of the algebra of the cost ledger, with no quantum input.

A note on the rational approximation in the Coq files. The file `TsirelsonUpperBound.v` names the rational threshold $5657/2000 \approx 2.8285$ for exact rational comparisons (you can verify $(5657/2000)^2 = 32{,}001{,}649/4{,}000{,}000 > 8$, so $5657/2000 > \sqrt{8} = 2\sqrt{2}$). The real-valued quantum-model bridge uses $\sqrt{8}$ directly. Those are different proof surfaces, and they should not be conflated.

15.4 What the cost algebra proves

Theorem 15.1 (Classical CHSH Bound)**.** *For any locally factorizable correlation (Alice's answer depends only on x and shared randomness λ; Bob's answer depends only on y and λ), $|S| \leq 2$. And $|S| = 2$ is achievable at $\mu = 0$, so the bound is tight.*

Proof. $|S| \leq 2$: the XOR-factoring argument from Section 15's "Why 75% is the classical ceiling." Fix any local deterministic strategy and any λ. Alice produces $A_x(\lambda) \in \{-1, +1\}$; Bob produces $B_y(\lambda) \in \{-1, +1\}$. The quantity

$$A_0(\lambda)B_0(\lambda) + A_0(\lambda)B_1(\lambda) + A_1(\lambda)B_0(\lambda) - A_1(\lambda)B_1(\lambda) \;=\; A_0(\lambda)(B_0(\lambda) + B_1(\lambda)) + A_1(\lambda)(B_0(\lambda) - B_1(\lambda$$

satisfies $|\cdot| \leq |B_0 + B_1| + |B_0 - B_1| = 2$ (because for $B_0, B_1 \in \{\pm 1\}$, one of $|B_0 + B_1|$ and $|B_0 - B_1|$ is 2 and the other is 0, and $|A_0|, |A_1| \leq 1$). Average over λ to get $|S| \leq 2$. Mixtures of deterministic strategies are convex combinations, so the same bound holds by linearity of expectation.

Tightness: the strategy $A_x \equiv 1$, $B_y \equiv 1$ gives $\langle A_0 B_0 \rangle = \langle A_0 B_1 \rangle = \langle A_1 B_0 \rangle = \langle A_1 B_1 \rangle = 1$, so $S = 1 + 1 + 1 - 1 = 2$. This is achievable at $\mu = 0$ by a program with all $\delta = 0$ instructions accumulating one trial per setting pair in `vm_witness`. The Coq witness is `classical_bound_achieved` at `coq/kernel/foundation/ClassicalBound.v:165`. □ □

Plain reading. Factor the expression. Two terms. One has $B_0 + B_1$, the other has $B_0 - B_1$. With $B_0, B_1 \in \{\pm 1\}$, only one can be nonzero at a time, and the nonzero one is ± 2. Multiply by $|A| \leq 1$. The whole expression caps at 2. Average over λ, you get $|S| \leq 2$. To make it 2, just have both players always answer 1.

Theorem 15.2 (Non-locality of CHSH violation)**.** *WitnessCount configurations with $S > 2$ cannot be produced by any local hidden variable model (the term is built in Section 2: a classical-physics-style theory where outcomes come from shared randomness λ plus local settings).*

Proof. Contrapositive of the Classical CHSH Bound. Any LHV model produces a locally factorizable correlation by definition: each player's outcome is a function of their setting and the shared λ. The theorem above says all such correlations satisfy $|S| \leq 2$. So a configuration with $S > 2$ cannot have arisen from any LHV model. □
□

Plain reading. Direct contrapositive. If $|S| \leq 2$ for every LHV, then $S > 2$ rules out every LHV. The Bell-test contradiction is in this form.

Theorem 15.3 (Tsirelson upper bound for the zero-marginal NPA model)**.** *If a trace is quantum-realizable in the zero-marginal NPA model, then its CHSH value satisfies* $|S| \leq \sqrt{8} = 2\sqrt{2}$.

Proof. Quantum realizability in the zero-marginal NPA model means the four CHSH correlators E_{xy} arise as $\mathrm{Tr}(\rho\, A_x B_y)$ for some quantum state ρ and self-adjoint ± 1-valued observables A_x, B_y with $\rho_{A_x A_x} = \rho_{B_y B_y} = I$ and vanishing marginals. Equivalently (by Section 15's biconditional), the correlators satisfy the three column-contractivity conditions. From column-contractivity, $S^2 \leq 8$: this is the row-bounds factorization (Cauchy-Schwarz applied to each row of the correlator matrix viewed as inner products of unit vectors). Specifically, the row norms of E satisfy $\|E_{0,\cdot}\|_2 \leq 1$ and $\|E_{1,\cdot}\|_2 \leq 1$, and the CHSH expression S is an inner product of bounded vectors with the row vectors, yielding $|S|^2 \leq 4(\|E_{0,\cdot}\|_2^2 + \|E_{1,\cdot}\|_2^2)$ via the Cauchy-Schwarz argument in `tsirelson_from_row_bounds` at `coq/kernel/quantum/TsirelsonGeneral.v`. Combined with row-norms ≤ 1: $|S|^2 \leq 8$, so $|S| \leq 2\sqrt{2}$. □ □

Plain reading. Quantum-realizable means column-contractive (by the biconditional). Column-contractive bounds the row norms of the correlator matrix by 1. Cauchy-Schwarz on the CHSH expression viewed as an inner product gives $|S| \leq 2\sqrt{2}$. No quantum-mechanical detail enters past the biconditional; the bound is then pure algebra.

Theorem 15.4 (General algebraic Tsirelson bound)**.** *For every algebraically coherent correlator (one satisfying* $|E_{xy}| \leq 1$ *along with four constraints derived from* 3×3 *submatrices of the NPA moment matrix; the submatrix determinants, called minors, must be non-negative, which is part of the requirement that the full matrix be positive-semidefinite, see Section 2 for both terms), and given some witness parameters* t *and* s *that the Coq file builds explicitly, the CHSH value satisfies* $S^2 \leq 8$*, that is,* $|S| \leq 2\sqrt{2}$*. This is the general case: every algebraically coherent correlator, not just the symmetric configuration. The Coq proof uses* `psatz` *(described in Section 2: a Coq tactic that proves polynomial inequalities by exhibiting a sum-of-squares decomposition). The rational approximation corollary uses* $5657/2000$ *as an exact rational overestimate of* $2\sqrt{2}$ *(you can verify* $(5657/2000)^2 = 32{,}001{,}649/4{,}000{,}000 > 8$*), allowing the bound to be checked in exact rational arithmetic without floating-point. No physics premises. Pure polynomial arithmetic. The Tsirelson bound* $2\sqrt{2}$ *is derived from algebra, not assumed.*

Proof. The four minor-positivity conditions on 3×3 submatrices of the NPA moment matrix are: each principal 3×3 minor's determinant is non-negative. Working through the determinant formulas with the moment-matrix structure of Section 15, the four constraints become four polynomial inequalities of the form $1 - E_{xy}^2 - E_{x'y'}^2 \geq 0$ for the appropriate index pairs (these reduce to the column-contractivity conditions 1 and 2 plus their transposes).

Given these constraints, $S^2 \leq 8$ is a polynomial inequality in $(E_{00}, E_{01}, E_{10}, E_{11})$ subject to four polynomial constraints. The Positivstellensatz (the algebraic certificate for polynomial-positivity) gives a sum-of-squares decomposition: the polynomial $8 - S^2$ minus a non-negative combination of the constraint polynomials is itself a sum of squares. Coq's `psatz` tactic finds and verifies such a decomposition; the explicit witness parameters t, s are coefficients in the SOS combination, computed once and stored in the Coq file.

The rational approximation corollary uses $5657/2000$ in place of $\sqrt{8}$ to keep the comparison in $\mathbb{Q}$: $(5657/2000)^2 = 32{,}001{,}649/4{,}000{,}000 > 8 = 32{,}000{,}000/4{,}000{,}000$, so $5657/2000 > \sqrt{8}$. Combined with $|S|^2 \leq 8$, this gives $|S| < 5657/2000$, a strict rational bound checkable without floating-point evaluation. □ □

Plain reading. Four polynomial constraints on the correlators (the 3×3 minor positivity conditions). Goal: show $S^2 \leq 8$ subject to those constraints. Positivstellensatz says: if a polynomial is non-negative on the feasible set, you can write it as a non-negative combination of the constraint polynomials plus a sum of squares. `psatz` finds the combination. The Coq file stores the witness coefficients. The Tsirelson bound is the consequence; no physics input goes into the proof.

There is also a coupling-language version of Bell's theorem, proved in `CHSHCouplingBridge.v`. A WitnessCount-consistent locally deterministic strategy (the strategy is described by four bits: Alice's answer for each of her two settings, Bob's answer for each of his two settings) produces a separable coupling, by which I mean a coupling that maps each setting to exactly one outcome type. Any locally factorizable morphism coupling satisfying the WCLocallyConsistent predicate has $|S| \leq 2$. Taking the contrapositive (the standard logical move: $P \Rightarrow Q$ has the same content as $\neg Q \Rightarrow \neg P$), if $S > 2$ then no locally factorizable morphism coupling is possible. The no-go result is stated in the coupling language, not just in the statistical layer.

What does this have to do with computation? The connection is through the witness counters and the certification layer. If a Thiele Machine program accumulates CHSH trial results (via CHSH_TRIAL) and then certifies that $S > 2$ (via CERTIFY or LASSERT), the certification is non-free. Certifying binary game statistics requires paying $\mu \geq 1$. The statistics themselves (raw CHSH trials) are free. Claiming you know what they mean is not. The witness counters are indexed by Alice's setting x

and Bob's setting y throughout, matching Section 9 and the Coq. This is the witness insight taxonomy (Section 6).

15.5 The bit-commitment requirement

Obtaining attested cross-correlations above the classical bound requires a disclosure instruction. Here is why that is a structural fact, not a policy choice.

Each CHSH_TRIAL records setting bits and outcome bits into witness counters. Those raw counts are free. But certifying what they mean, attesting that the cross-correlations $\langle A_x B_y \rangle$ justify a particular claim, requires explicit certification, and certification requires an explicit cost.

Proved in `RevelationRequirement.v`. The structural fact: the only instruction that can set `csr_cert_addr` nonzero is MORPH_ASSERT (when the morphism exists and the property string is non-empty). Any trace that ends with supra_cert must include a valid MORPH_ASSERT step. The raw data (CHSH_TRIAL counts) is free. The certified claim about what the data means is not.

15.6 Algebraic characterization of the μ-ledger honesty condition

If you only read one subsection of Section 15, this is the one. The rest of the section is the derivation. This is the result.

Here is the result of stress-testing the cost algebra against the NPA boundary. The column-contractivity conditions arise from the natural polynomial conditions that make the NPA moment matrix well-formed; they were not imported from physics, and the biconditional with NPA-PSD is proved in pure polynomial arithmetic. The following biconditional holds.

Positive semidefinite (PSD). A matrix M is positive semidefinite if for every vector v you can multiply with it, the quantity $v^T M v$ is at least 0. (Here v^T is the transpose of v from Section 2: turning a column of numbers on its side; $v^T M v$ is the matrix product of v^T, M, and v, which is just one number.) The intuition: PSD means the matrix never maps a vector to something pointing "backwards" relative to the vector. For a 2×2 matrix $\begin{pmatrix} a & b \\ b & c \end{pmatrix}$, PSD reduces to three simple conditions: $a \geq 0$, $c \geq 0$, and $ac - b^2 \geq 0$. The first two say the diagonal entries are non-negative. The third says the determinant (Section 2) is non-negative. For larger matrices, the PSD condition generalises: every principal submatrix (every square subblock you get by picking some rows-and-columns at the same indices, see Section 2) must also be PSD.

PSD is the mathematical condition that a matrix can represent genuine inner products, covariances, or correlations from a quantum state. The NPA framework uses PSD of a specific 5×5 matrix (the "moment matrix") as the condition for quantum realizability. You build that 5×5 matrix from the four CHSH correlators, and if it is PSD, the correlators are consistent with some quantum state.

15.6.1 The matrix, written out

Here is the actual 5×5 NPA moment matrix for the zero-marginal CHSH scenario. The five operators are: I (identity), A_0 (Alice's measurement for setting 0), A_1 (Alice's measurement for setting 1), B_0 (Bob's for setting 0), B_1 (Bob's for setting 1). The (i, j) entry is the expected value of $O_i \cdot O_j$. Zero-marginal means we set $\langle A_0 \rangle = \langle A_1 \rangle = \langle B_0 \rangle = \langle B_1 \rangle = 0$ and the cross-correlation terms $\langle A_0 A_1 \rangle = \langle B_0 B_1 \rangle = 0$.

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{pmatrix}$$

Rows and columns are indexed 0 through 4, in the order $\{I, A_0, A_1, B_0, B_1\}$. The diagonal is all 1s because each operator O_i satisfies $O_i^2 = I$ when the outcomes are ± 1-valued: squaring a ± 1-valued operator gives the identity, so the diagonal entries are all $\langle I \rangle = 1$. The off-diagonal entries E_{xy} are the CHSH correlators with Alice's setting x and Bob's setting y (Section 2). Everything else is zero because the marginals vanish (Section 2: a marginal is the average of one variable with the others integrated out; zero-marginal means $\langle A_x \rangle = \langle B_y \rangle = 0$).

15.6.2 Where the three conditions come from

The three column-contractivity conditions come directly from PSD of this matrix. Here is the derivation.

Condition 1: $1 - E_{00}^2 - E_{10}^2 \geq 0$

Take the 3×3 submatrix from rows and columns $\{1, 2, 3\}$ of the 5×5 moment matrix (the rows and columns indexed by A_0, A_1, B_0):

$$\begin{pmatrix} 1 & 0 & E_{00} \\ 0 & 1 & E_{10} \\ E_{00} & E_{10} & 1 \end{pmatrix}$$

Compute the determinant of this 3×3 submatrix by the cofactor expansion along the first row (Section 2: pick a row, multiply each entry by the determinant of the 2×2 matrix you get by deleting that entry's row and column, with alternating signs, then sum):

$$1 \cdot (1 \cdot 1 - E_{10}^2) - 0 \cdot (0 \cdot 1 - E_{10} \cdot E_{00}) + E_{00} \cdot (0 \cdot E_{10} - 1 \cdot E_{00}) = 1 - E_{10}^2 - E_{00}^2.$$

The principal-submatrix rule for PSD says this determinant must be ≥ 0. That is condition 1.

Condition 2: $1 - E_{01}^2 - E_{11}^2 \geq 0$

Take the 3×3 submatrix from rows and columns $\{1, 2, 4\}$ (the A_0, A_1, B_1 block):

$$\begin{pmatrix} 1 & 0 & E_{01} \\ 0 & 1 & E_{11} \\ E_{01} & E_{11} & 1 \end{pmatrix}$$

Same calculation: determinant $= 1 - E_{01}^2 - E_{11}^2 \geq 0$. That is condition 2.

Condition 3: $(1 - E_{00}^2 - E_{10}^2)(1 - E_{01}^2 - E_{11}^2) \geq (E_{00}E_{01} + E_{10}E_{11})^2$

This comes from the 4×4 submatrix at rows and columns $\{1, 2, 3, 4\}$. That submatrix has a block structure (an arrangement where the entries are themselves 2×2 matrices):

$$\begin{pmatrix} I_2 & E \\ E^T & I_2 \end{pmatrix}$$

where I_2 is the 2×2 identity matrix (Section 2) and E is the 2×2 correlator matrix

$$E = \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix}.$$

E^T is the transpose of E from Section 2. A standard linear-algebra fact (the Schur complement criterion, also Section 2) says that a block matrix of this shape is positive-semidefinite if and only if the Schur complement of the upper-left I_2 is positive-semidefinite. The Schur complement here works out to $I_2 - E^T E$. The order of the transpose matters: $E^T E$ puts the squared norms (sums of squares) of the columns of E on its diagonal, while EE^T would put the squared norms of the rows. The right one for this argument is $E^T E$.

Computing $E^T E$ entry by entry:

$$E^T E = \begin{pmatrix} E_{00} & E_{10} \\ E_{01} & E_{11} \end{pmatrix} \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix} = \begin{pmatrix} E_{00}^2 + E_{10}^2 & E_{00}E_{01} + E_{10}E_{11} \\ E_{00}E_{01} + E_{10}E_{11} & E_{01}^2 + E_{11}^2 \end{pmatrix}.$$

The Schur complement is therefore

$$I_2 - E^T E = \begin{pmatrix} 1 - E_{00}^2 - E_{10}^2 & -(E_{00}E_{01} + E_{10}E_{11}) \\ -(E_{00}E_{01} + E_{10}E_{11}) & 1 - E_{01}^2 - E_{11}^2 \end{pmatrix} = \begin{pmatrix} A & -B \\ -B & C \end{pmatrix}$$

where $A = 1 - E_{00}^2 - E_{10}^2$ (the left side of condition 1), $B = E_{00}E_{01} + E_{10}E_{11}$, and $C = 1 - E_{01}^2 - E_{11}^2$ (the left side of condition 2). For this 2×2 Schur complement to be PSD, two things must hold: the diagonal entries must be non-negative (which are conditions 1 and 2 again) and the determinant must be non-negative, $AC - B^2 \geq 0$. That last inequality is condition 3.

That is the complete derivation. Three conditions. Two from 3×3 submatrices. One from the Schur complement of the 4×4 block. All of it falls out of the single requirement that M is positive semidefinite.

Why this equals quantum realizability. The forward direction is the derivation above: if the correlators are column-contractive (the three conditions hold), the matrix M is PSD by construction. The reverse direction (proved as `npa_psd_implies_column_contractive` in `coq/kernel/quantum/QuantumPartitionPSD.v`) is that if M is PSD, the correlators must be column-contractive. Here is the argument. Take any PSD matrix M and feed it the specific 5-dimensional vector $v = (0,\ -(E_{00}y_0 + E_{01}y_1),\ -(E_{10}y_0 + E_{11}y_1),\ y_0,\ y_1)$, where y_0 and y_1 are free real numbers. The PSD condition $v^T M v \geq 0$, after the matrix algebra, gives a quadratic expression in y_0 and y_1: $Ay_0^2 - 2By_0y_1 + Cy_1^2 \geq 0$ for all real y_0 and y_1. A quadratic of the form $ay^2 + 2by + c$ is non-negative for every real y if and only if $a \geq 0$ and $ac \geq b^2$ (the non-negative-discriminant rule from intermediate algebra: the parabola does not dip below the axis exactly when its discriminant is non-positive, which after sign-flipping reads $b^2 \leq ac$). That closes condition 3. The diagonal conditions (1 and 2) come from setting $y_1 = 0$ and $y_0 = 0$ separately.

A correlation $(E_{00}, E_{01}, E_{10}, E_{11})$ is *Thiele-honest* if and only if it is zero-marginal NPA realizable. Exactly. Not approximately. Not "consistent with some physics axiom." The biconditional is pure polynomial arithmetic, zero axioms:

Theorem 15.5 (Thiele honesty $\iff$ NPA-PSD)**.**

$$\textit{honest}(\mathbf{E}) \iff \textit{psd-npa}(\mathbf{E})$$

where honest *means the three column-contractivity conditions hold, and* psd-npa *means the* 5×5 *zero-marginal NPA matrix is positive semidefinite.*

Proof. The full math is in the two derivations of Section 15 preceding this theorem. Recapitulated for the proof block:

Forward direction (column-contractive ⇒ PSD). Assume the three column-contractivity conditions hold. The Section 15 derivation showed: condition 1 is exactly the principal-submatrix determinant on rows/columns $\{1, 2, 3\}$; condition 2 is exactly the principal-submatrix determinant on rows/columns $\{1, 2, 4\}$; condition 3 is exactly the Schur-complement determinant of the 4×4 block. Combined with the $\{0\}$ principal submatrix (just the entry $1 \geq 0$) and the diagonal entries (all $1 \geq 0$), every principal submatrix has non-negative determinant and non-negative diagonal, so by the principal-submatrix PSD criterion (Section 2), the full 5×5 matrix is PSD. The Coq witness is `zero_marginal_npa_column_contractive_implies_psd`.

Reverse direction (PSD ⇒ column-contractive). Assume the moment matrix M is PSD.

Specialize $v^T M v \geq 0$ at the vector

$$v(y_0, y_1) := \big(0,\ -(E_{00}y_0 + E_{01}y_1),\ -(E_{10}y_0 + E_{11}y_1),\ y_0,\ y_1\big)$$

for free $y_0, y_1 \in \mathbb{R}$. Computing $v^T M v$ using the explicit moment-matrix entries and collecting terms gives the quadratic form

$$A y_0^2 - 2B y_0 y_1 + C y_1^2 \ \geq\ 0 \quad \text{for all } y_0, y_1 \in \mathbb{R},$$

where $A = 1 - E_{00}^2 - E_{10}^2$, $B = E_{00}E_{01} + E_{10}E_{11}$, $C = 1 - E_{01}^2 - E_{11}^2$. By the quadratic-non-negative-discriminant lemma (`quadratic_nonneg_discriminant` at `coq/kernel/quantum/ConstructivePSD.v:335`), this is equivalent to: $A \geq 0$, $C \geq 0$, and $B^2 \leq AC$. The three conditions are exactly the three column-contractivity conditions. Specializing further at $(y_0, y_1) = (1, 0)$ and $(0, 1)$ recovers conditions 1 and 2 directly; the general (y_0, y_1) case yields condition 3. The Coq witness is `npa_psd_implies_column_contractive`. □ □

Plain reading. Both directions are pure algebra on the 5×5 matrix. Forward: each of the three column-contractivity conditions is a principal-submatrix or Schur-complement determinant being non-negative. PSD reduces to all principal-submatrix determinants ≥ 0, which is exactly what column-contractivity gives you. Reverse: feed the PSD matrix a specific test vector parameterized by (y_0, y_1). The result $v^T M v$ comes out as a quadratic in (y_0, y_1). Non-negative-for-all-(y_0, y_1) means the quadratic's discriminant is non-positive, i.e., $b^2 \leq ac$. That's condition 3. The diagonal conditions fall out at $(1, 0)$ and $(0, 1)$. No physics anywhere; both directions are polynomial-arithmetic identities.

The left side is three polynomial inequalities (column contractivity of the zero-marginal NPA matrix):

$$1 - E_{00}^2 - E_{10}^2 \geq 0 \quad \text{(condition 1)}$$

$$1 - E_{01}^2 - E_{11}^2 \geq 0 \quad \text{(condition 2)}$$

$$AC \geq B^2 \quad \text{(condition 3)}$$

where $A = 1 - E_{00}^2 - E_{10}^2$, $B = E_{00}E_{01} + E_{10}E_{11}$, $C = 1 - E_{01}^2 - E_{11}^2$. These are the three conditions of `zero_marginal_column_contractive` (defined in `coq/kernel/nfi/MuLedgerQuantumBridge.v`): column 0 norm ≤ 1, column 1 norm ≤ 1, determinant $AC - B^2 \geq 0$. The right side says the NPA moment matrix is positive semidefinite, the standard mathematical characterization of quantum realizability.

The forward direction (`zero_marginal_npa_column_contractive_implies_psd` in `coq/kernel/nfi/MuLedgerQuantumBridge.v`) was proved first. The reverse direction: given a PSD NPA matrix, specialize it at the vector $(0, -(E_{00}y_0 + E_{01}y_1), -(E_{10}y_0 + E_{11}y_1), y_0, y_1)$. The PSD condition gives a quadratic-in-

y_0/y_1 that is everywhere non-negative. Then `quadratic_nonneg_discriminant` ($\forall t, a + 2bt + ct^2 \geq 0 \Rightarrow b^2 \leq ac$) closes the determinant condition. The diagonal conditions come from $y_0 = 1, y_1 = 0$ and $y_0 = 0, y_1 = 1$. Both directions are packaged as the biconditional `column_contractive_iff_quantum_realizable` in `coq/kernel/quantum/QuantumPartitionPSD.v`.

What this means for the hierarchy of correlations:

- Classical local hidden variables: $|S| \leq 2$.
- Thiele-honest = zero-marginal NPA realizable (biconditional, both directions, zero axioms). Every Thiele-honest correlation satisfies $|S| \leq 2\sqrt{2}$, but NOT every correlation with $|S| \leq 2\sqrt{2}$ is Thiele-honest: $(1, 0, 1, 0)$ has $|S| = 2$ but fails condition 1.
- No-signaling: no constraint beyond $|E_{ab}| \leq 1$.
- PR box ($E_{00} = E_{01} = E_{10} = 1$, $E_{11} = -1$): condition 1 gives $1 - E_{00}^2 - E_{10}^2 = 1 - 1 - 1 = -1 < 0$. Fails immediately by direct arithmetic.
- Tsirelson bound: `state_column_contractive_implies_tsirelson` (`coq/kernel/nfi/MuLedgerQuantumBridge.v`) proves Thiele-honest $\Rightarrow$ $|S|^2 \leq 8$ (i.e., $|S| \leq 2\sqrt{2}$) directly from column-contractivity. The chain factors through PSD: column-contractive $\Rightarrow$ NPA-PSD $\Rightarrow$ row bounds $\Rightarrow$ Tsirelson, and the row-bounds step is `tsirelson_from_row_bounds` in `coq/kernel/quantum/TsirelsonGeneral.v`. The contrapositive ($|S| > 2\sqrt{2} \Rightarrow$ not Thiele-honest) follows immediately. The converse is false and not proved.

A Turing machine can output the correlations $(1, 0, 1, 0)$. These are not Thiele-honest: condition 1 gives $1 - 1^2 - 1^2 = -1 < 0$. The arithmetic closes by `lra` once `column_contractive_iff_quantum_realizable` is destructed on condition 1. The μ-ledger's column-contractivity conditions are what make the quantum boundary real for computation: there exist correlations a Turing machine certifies but no Thiele Machine certifies as honest.

Kernel-level enforcement (the CHSH_LASSERT opcode). The opcode `CHSH_LASSERT` (in `coq/kernel/foundation/VMStep.v`) is the kernel-level enforcement that ties "Thiele-honest" as column-contractivity to the certification channel: a generic `CERTIFY` can set `vm_certified := true` without inspecting the witness counters, so the certification channel needs an opcode that does inspect them before declaring honesty. That opcode reads the WitnessCounts buckets, computes the three integer-arithmetic conditions `column_contractive_check_witness` via Z-arithmetic with no real or rational evaluation at runtime, and either advances the program counter (on success) or traps to `LASSERT_TRAP_PC` and latches `vm_err` (on failure). Two theorems close the chain to NPA-PSD; both are stated and proved below.

Theorem 15.6 (Integer-check soundness, `column_contractive_check_witness_sound` in `coq/kernel/nfi/MuLedgerQuantumBridge.v`). *Let w be the WitnessCount record with eight non-negative-integer fields $\texttt{wc\_same}_{xy}, \texttt{wc\_diff}_{xy}$ for $(x,y) \in \{0,1\}^2$. Define the integer-only check as the conjunction of three $\mathbb{Z}$-arithmetic inequalities on these counts (the boolean function `column_contractive_check_witness` written in $\mathbb{Z}$, with no division). If the integer check returns `true`, then defining*

$$N_{xy} = \texttt{wc\_same}_{xy} + \texttt{wc\_diff}_{xy}, \qquad E_{xy} = \frac{\texttt{wc\_same}_{xy} - \texttt{wc\_diff}_{xy}}{N_{xy}}$$

(with the convention $E_{xy} = 0$ when $N_{xy} = 0$) yields a real-valued correlator $\mathbf{E} = (E_{00}, E_{01}, E_{10}, E_{11}) \in [-1,1]^4$ that satisfies the three column-contractivity conditions of Section 15.

Proof. The integer-only check is written in $\mathbb{Z}$ as three inequalities that, after multiplying through by N_{xy}^2 to clear all denominators, are polynomial inequalities in the non-negative integer counts. For each condition:

- Condition 1 ($1 - E_{00}^2 - E_{10}^2 \geq 0$) becomes the inequality $N_{00}^2 \cdot N_{10}^2 - (\texttt{wc\_same}_{00} - \texttt{wc\_diff}_{00})^2 N_{10}^2 - (\texttt{wc\_same}_{10} - \texttt{wc\_diff}_{10})^2 N_{00}^2 \geq 0$ after substituting E_{xy} and clearing denominators.
- Condition 2 is the analogous inequality with the E_{01}, E_{11} correlators.
- Condition 3 ($AC \geq B^2$ in the notation of Section 15) becomes a polynomial inequality of degree 4 in the integer counts, after clearing the N_{xy}^4 denominator.

The integer check is exactly these three polynomial inequalities tested on the integer counts (using $\mathbb{Z}$, no rationals). To prove the conclusion: assume the check returns `true`. Cast the integer counts to $\mathbb{R}$ by $\texttt{IZR} : \mathbb{Z} \to \mathbb{R}$, which preserves the inequalities. Divide through by N_{xy}^2 (or N_{xy}^4, for condition 3) using the fact that $N_{xy} \geq 0$ as a natural number; in the $N_{xy} = 0$ case, the convention $E_{xy} = 0$ makes the inequality trivial. After dividing, the three inequalities are exactly conditions 1, 2, 3 of column-contractivity in $\mathbb{R}$. The Coq proof closes the post-division algebra by `nra` (nonlinear real arithmetic) acting on a sum-of-squares-shaped goal. □ □

Plain reading. Integer arithmetic upstairs, real arithmetic downstairs, IZR is the elevator. You take the integer counts of same and different outcomes, write down the three column-contractivity conditions in terms of those counts (cleared of denominators), and you have polynomial inequalities in $\mathbb{Z}$. Cast to $\mathbb{R}$ via IZR; this preserves ≥ 0. Divide back through by the (non-negative) total-trial counts; this gives the real-valued column-contractivity conditions. The algebra after the division is just a sum-of-squares-shaped polynomial inequality; `nra` closes it. The whole route avoids any floating-point evaluation at any step: integers in, reals out, division at the boundary.

Theorem 15.7 (CHSH_LASSERT ⇒ NPA-PSD, `chsh_lassert_no_trap_implies_quantum_realiz` in `coq/kernel/quantum/QuantumPartitionPSD.v`)**.** *Let s be a state and let $s' =$ `vm_apply s (instr_chsh_lassert δ)`. If s'.`vm_err` = false and s'.`vm_pc` $= s$.`vm_pc` $+ 1$ (i.e., the CHSH_LASSERT step advanced normally rather than trapping), then the zero-marginal NPA moment matrix M built from the witness-derived correlators E_{xy} of s.`vm_witness` is positive semidefinite.*

Proof. By the apply rule for `instr_chsh_lassert`: the step branches on `column_contractive_check_witness` run on s.vm_witness. If the check returns `false`, the step traps to `LASSERT_TRAP_PC` and sets `vm_err = true`. If the check returns `true`, the step advances PC by 1 and leaves `vm_err = false`. By hypothesis, s' exhibits the second behavior, so the integer check returned `true` on s.vm_witness.

Compose with the preceding theorem (integer-check soundness): the correlators E_{xy} derived from s.vm_witness satisfy the three real-valued column-contractivity conditions.

Compose with the forward direction of `column_contractive_iff_quantum_realizable` (the biconditional proved in Section 15): column-contractivity implies the NPA moment matrix M is positive semidefinite. □ □

Plain reading. Three theorems compose: the kernel step rule says no-trap means the integer check passed; the soundness lemma says the integer check passing implies real-valued column-contractivity; the biconditional says column-contractivity implies NPA-PSD. Stack them. A successful CHSH_LASSERT step is a proof that the witness-derived correlators land inside the NPA-PSD set. No physics input anywhere in the stack. The (1,0,1,0) correlation fails the integer check (condition 1 fails immediately: $1 - 1 - 1 = -1 < 0$), so any program that accumulated those statistics traps on CHSH_LASSERT.

The (1,0,1,0) trace executed under `CHSH_LASSERT` traps with `vm_err = true`; only column-contractive traces pass without trap.

To falsify: exhibit $E_{00}, E_{01}, E_{10}, E_{11}$ satisfying the three polynomial inequalities but with a non-PSD NPA matrix. No such example exists; the theorem proves it impossible. Or exhibit a quantum experiment producing a non-PSD NPA matrix. That would break quantum mechanics, not this theorem.

16 Irreversibility accounting and the μ-hierarchy

16.1 The Landauer association is motivation only

The cost-on-certification rule has the same shape as Landauer's 1961 argument that erasing one bit of information must release at least $kT \ln 2$ of heat into the environment

(k is the Boltzmann constant; T is absolute temperature; $\ln 2$ is the natural log of 2). The Coq proofs below do not depend on Landauer's claim being physically correct. They formalise irreversibility at the instruction level only: a cost-positive instruction is tagged as irreversible, and the count of such firings is bounded by total μ.

Theorem 16.1 (μ-Landauer step bound)**.** *Let* $\texttt{irreversible\_bits}(i)$ *be the indicator function that returns* 1 *if* $\texttt{instruction\_cost}(i) > 0$ *and* 0 *otherwise. Let* $\texttt{total\_irreversible\_bits}(\mathit{trace}) := \sum_{i \in \mathit{trace}} \texttt{irreversible\_bits}(i)$*. Then for every trace,*

$$\texttt{total\_irreversible\_bits}(\mathit{trace}) \leq \texttt{trace\_total\_cost}(\mathit{trace}).$$

The Coq witness is `total_irreversible_bits_le_cost` *in* `coq/kernel/nfi/LandauerDerivation.v:241`.

Proof. Per-instruction inequality: for every i, $\texttt{irreversible\_bits}(i) \leq \texttt{instruction\_cost}(i)$. Two cases.

Case $\texttt{instruction\_cost}(i) = 0$. Then $\texttt{irreversible\_bits}(i) = 0$ by definition, and $0 \leq 0$ holds.

Case $\texttt{instruction\_cost}(i) > 0$. Then $\texttt{irreversible\_bits}(i) = 1$, and the cost schedule (Section 7) gives $\texttt{instruction\_cost}(i) \geq S(\delta) \geq 1$ for cert-setters or $\texttt{instruction\_cost}(i) \geq \delta \geq 1$ otherwise. So $1 \leq \texttt{instruction\_cost}(i)$.

Summing the per-instruction inequality over the trace:

$$\sum_{i \in \mathit{trace}} \texttt{irreversible\_bits}(i) \leq \sum_{i \in \mathit{trace}} \texttt{instruction\_cost}(i) = \texttt{trace\_total\_cost}(\mathit{trace}).$$

The right equality is by definition of `trace_total_cost` (or via the Latent-μ theorem applied to the trace from $\mu_0 = 0$). □ □

Plain reading. Each cost-positive instruction in the trace contributes at least 1 to the total cost and at most 1 to the irreversible-bit count. So summing across the trace, the count is bounded by the cost. The bound is shape-Landauer (irreversible operations are charged) but doesn't invoke any physics — it's just observing that a 0/1 indicator is bounded by what it indicates. Landauer-shaped in motivation only: no microscopic erasure model, no physical entropy, no Boltzmann constant. The conversion of μ counts to joules is the calibration hypothesis (`mu_landauer_unruh_calibrated`, listed in Section 23); none of the kernel theorems use it.

16.2 The μ-hierarchy

The proof:

The μ-cost hierarchy theorem is shaped like the time hierarchy theorem from classical complexity theory, but the content is narrower. It says every cost level $k \geq 1$ is

reachable by some certified trace, and no cost level k is reachable for less than k units of μ. That is cost-level reachability plus conservation, not problem-class separation. The class-separation question is open and is listed in Section 23.

Theorem 16.2 (μ-Hierarchy (`mu_hierarchy_theorem` in `coq/kernel/mu_calculus/MuHierarchyTheorem.v:162`)). *For every $k \geq 1$:*

1. *There exists a trace reaching `vm_certified = true` with μ-cost exactly k.*
2. *Any trace with `vm_mu` $\geq k$ has `trace_mu_cost` $\geq k$.*

Proof. Part (1): cost-k certified trace exists. Construct the trace

$$prog_k := \underbrace{[\texttt{instr\_load\_imm}\,0\,0\,0,\ \ldots,\ \texttt{instr\_load\_imm}\,0\,0\,0]}_{k-1 \text{ copies}},\ [\texttt{instr\_certify}\,0].$$

The $k-1$ `LOAD_IMM` instructions each have $\delta = 0$ and so cost 0 each. The final `CERTIFY 0` has cost $S(0) = 1$ and sets `vm_certified` to `true`. So `trace_total_cost`$(prog_k) = 0 \cdot (k-1) + 1 = 1$, not k. To get cost exactly k, use $prog_k :=$ [`instr_load_imm 0 0` $(k-1)$, `instr_certify 0`]: the `LOAD_IMM` with $\delta = k-1$ costs $k-1$ (ordinary compute pays δ directly), the `CERTIFY 0` costs 1, total k. After running, `vm_certified = true`.

Part (2): cost lower bound. By Latent μ applied to any trace from $\mu_{\text{initial}} = 0$: $\mu_{\text{final}} =$ `trace_total_cost`$(trace)$. So `vm_mu` $\geq k$ implies `trace_total_cost`$(trace) \geq k$ directly.

Each $k \geq 1$ is therefore realizable, and each k-realizing trace pays at least k. □ □

Plain reading. Two parts. Part (1) is by construction: pick a single `LOAD_IMM` with $\delta = k-1$ (costs $k-1$) and follow with `CERTIFY 0` (costs 1). Total cost k, ends certified. Part (2) is just Latent μ: total trace cost equals total μ accumulated from 0, so $\mu \geq k$ means cost $\geq k$. No shortcuts.

Corollary 16.3 (`mu_hierarchy_no_upper_bound`). *No fixed μ-budget suffices for all certification levels.*

Proof. Suppose for contradiction that some fixed budget $B \geq 0$ suffices for all certification levels — meaning every certified trace satisfies `vm_mu` $\leq B$. Apply the μ-Hierarchy theorem at $k = B + 1$: there exists a certified trace with `vm_mu` $= B + 1 > B$, contradicting the budget bound. □ □

Plain reading. For any budget you propose, the hierarchy theorem produces a certified trace that exceeds it by 1. So no finite budget caps all certifications.

The point: at the ledger, there is no shortcut. Every integer cost level is reachable. No level is reachable for less than its stated price. That is what the theorem proves and

what the Coq checks. "Strictly more powerful" would be the problem-class reading, which this theorem does not address.

The classical time hierarchy theorem has the same shape: for any time bound $T(n)$, there are problems solvable in time $T(n) \log T(n)$ that are not solvable in time $T(n)$. More time buys strictly more computation. The μ-hierarchy says more μ buys strictly more cost levels — narrower content, same shape. The analogy is exposition; the proof above is the load-bearing piece.

To falsify: find $k \geq 1$ and a trace reaching `vm_certified = true` and `vm_mu` $\geq k$ with total trace cost $< k$. The theorem says this cannot happen. If it does, the cost accounting is wrong.

17 Discrete Geometry over Partition Graphs

The partition graph is also a discrete geometric object. Here is one result that falls out, no physics required.

17.1 Discrete triangulation of the partition graph

Each module in the partition graph carries structural data, and several files at the bridge between the kernel proofs and the physics-inspired analogies use μ-derived quantities as geometry-like weights. The discrete Gauss-Bonnet file is more specific than that: it reads the partition graph as a triangulated combinatorial object (a graph cut up into triangular pieces) and uses an equilateral angle model where every triangle corner contributes $\pi/3$ radians (60 degrees, since an equilateral triangle has all three angles equal at 60 degrees and there are 2π radians in a full turn, or 360 degrees). It does not derive those angles directly from μ costs.

A discrete triangulation defines a curvature-like angle defect. As I understand it (I had to research this for the project, and the usual caveat applies that I do not trust my own reading of geometry sources): on a flat surface, if you draw a triangle, the interior angles sum to 180°. On a curved surface, the angles can sum to more or less than 180°. Curvature is the amount by which triangles deviate from flat. In the equilateral discrete model, curvature at a vertex is the defect left after subtracting the angles of the incident triangles (the triangles meeting at that vertex) from 2π (a full turn).

I went looking for the right invariant and found the Euler characteristic χ, a number that captures the overall shape of a triangulated surface, independent of how you draw the triangles. For any triangulation with V vertices, E edges, and F triangular faces: $\chi = V - E + F$. As I read it, no matter how you triangulate the same surface, you get the same number. For a sphere, $\chi = 2$. For a torus (donut shape), $\chi = 0$. The continuum Gauss-Bonnet theorem (which I am citing, not certifying) says the total curvature of a closed surface equals $2\pi\chi$: the local geometric information (curvature) integrates to a global topological fact (shape). The discrete analogue is stated below. The continuum result is exposition.

17.2 The boundary-triangulated angle-defect identity

A heads-up before the theorem: this is not the closed-surface Gauss-Bonnet identity. The standard closed-surface result is total curvature equals $2\pi\chi$. The theorem here uses a different predicate (planar triangulations with a boundary) and a different constant (5π instead of 2π). A reader who imports the closed-surface invariants will conclude the number is wrong; it is the right number for the class of graphs this predicate describes. The full derivation is in the paragraphs after the theorem; the theorem statement itself includes the predicate's invariants so the reader can see the scope before judging the constant.

Theorem 17.1 (Boundary-triangulated angle-defect identity (`discrete_gauss_bonnet` in `coq/kernel/curvature/DiscreteGaussBonnet.v:295-297`)). *Let g be a partition graph satisfying `well_formed_triangulated` (`coq/kernel/curvature/DiscreteTopology.v:311`, a* definition*, not a theorem; the structural identities it bundles are packaged into the named theorem `triangulation_combinatorial_identity` at line 651): a planar triangulation whose interior edges I and boundary edges B obey $E = I + B$ and $3F = 2I + B$, with an Euler-relation field forcing $B = 3\chi$ where $\chi = V - E + F$ is the Euler characteristic. In the equilateral angle model (every triangle corner contributes $\pi/3$), the total angle defect satisfies*

$$\text{angle\_defect}(g) = 5\pi \cdot \chi(g).$$

The constant is 5π, not 2π, because `well_formed_triangulated` is a planar-region predicate with a triangular boundary; a closed-surface predicate would force $B = 0$ and the algebra would yield $2\pi\chi$ instead. The canonical instance the predicate accepts is a triangulated planar disk ($\chi = 1$, $B = 3$).

Proof. The total angle defect is

$$\text{angle\_defect}(g) \;=\; \sum_v \big(2\pi - \deg(v) \cdot \tfrac{\pi}{3}\big) \;=\; 2\pi V - \tfrac{\pi}{3}\sum_v \deg(v),$$

where $\deg(v)$ counts triangles meeting at vertex v (triangle-incidence, not edge-incidence). The triangle-incidence sum is

$$\sum_v \deg(v) \;=\; 3F,$$

because each triangle has three corners, each contributing once to the corner-count of its vertex. So

$$\text{angle\_defect}(g) \;=\; 2\pi V - \pi F.$$

From `well_formed_triangulated`: the structural invariants $E = I + B$ and $3F = 2I + B$ combine to give $3V = 5E - 6F$ (the triangulation-combinatorial identity at `DiscreteTopology.v:651`). Solving for V: $V = (5E - 6F)/3$. Substituting:

$$\text{angle\_defect}(g) \;=\; 2\pi \cdot \tfrac{5E-6F}{3} - \pi F \;=\; \tfrac{10\pi E - 12\pi F - 3\pi F}{3} \;=\; \tfrac{10\pi E - 15\pi F}{3} \;=\; \tfrac{5\pi(2E-3F)}{3}.$$

The Euler characteristic in this triangulation satisfies $\chi(g) = (2E - 3F)/3$ (from the same identity, by elimination). Therefore angle_defect$(g) = 5\pi \cdot \chi(g)$. □ □

Plain reading. Total angle defect is $2\pi V - \pi F$ once you use the triangle-corner count $\sum \deg = 3F$. The `well_formed_triangulated` invariants give V in terms of E and F. Substitute and simplify. The 5π falls out as the coefficient of χ. Not chosen, not picked, not aimed at; just what the algebra returns when you solve the system the predicate hands you.

In my own words first, because I expect this to trip people up. I did not author any of the math in this section. I do not write code or proofs by hand. I directed LLMs and refused everything that did not compile. When I went looking for what discrete Gauss-Bonnet should look like, the version I kept finding was the closed-surface one, with constant 2π. The graphs the predicate I ended up with accepts are not closed surfaces. They have a boundary, like a flat sheet of paper cut into triangles. The constant that fell out of the proof for that case was 5π, not 2π. I did not pick 5π. I do not pick numbers. The algebra picked it and the checker confirmed it. So the warning to a reader who has the closed-surface version cached: the predicate named `well_formed_triangulated` is not the closed-surface case. If you assume it is, you will assume identities the predicate does not assume, and you will conclude the number is wrong. The number is right for the case the proof covers. If it is wrong, the proof should not have compiled, and breaking the proof is the way to break the result.

Now the same thing in the formal voice. `well_formed_triangulated` is a planar-triangulation predicate defined in `coq/kernel/curvature/DiscreteTopology.v:311`, with its combinatorial invariants packaged into the theorem `triangulation_combinatorial_identity` at line 651 of the same file. Its structural invariants are $E = I + B$ and $3F = 2I + B$, where I count the edges in two flavours: I for interior edges (edges shared by two triangles) and B for boundary edges (edges with only one triangle on either side). These invariants give $\chi(g) = (2E - 3F)/3$ and $3V = 5E - 6F$. In a closed surface like a sphere the corresponding identity is $2E = 3F$, but this predicate is for planar regions with a boundary, not closed surfaces.

In this discrete model the symbol $\deg(v)$ stands for the number of triangles meeting at vertex v (triangle-incidence), not the number of edges meeting at v (edge-incidence, which is the usual graph-theory definition). The triangle-incidence sum is $\sum_v \deg(v) = 3F$ because each triangle contributes once per corner and a triangle has three corners. Substituting that into the total angle defect $\sum_v (2\pi - \deg(v) \cdot \pi/3) = 2\pi V - \pi F$ yields $5\pi\chi$ exactly. (A reader who plugs in the edge-incidence identity $\sum_v \deg(v) = 2E$ will get a different answer, because that is a different quantity than the one this proof tracks.) The factor differs from 2π because the predicate differs from a closed surface, not because the algebra is wrong. A tetrahedron (a four-faced closed solid: V=4, E=6, F=4) violates $3V = 5E - 6F$ and is correctly excluded from the predicate.

A planar disk with V=7, E=15, F=9 gives $\chi = 1$ and $B = 3$ (a single triangular outer boundary), and the arithmetic of every invariant the predicate requires checks out for those values. The predicate's invariants accept this disk under the same checks the kernel applies to its packaged witnesses.

This is a fully proven theorem about the discrete triangulation model used here. It is not the continuum Gauss-Bonnet theorem and it is not a derivation of angles from μ cost. It is a combinatorics result: any well-formed triangulated partition graph (in the planar-triangulation sense above) satisfies the angle-defect formula. That is the full extent of the geometry section.

18 The substrate and its halting-shaped wall

The Thiele Machine is not the 47 opcodes. The 47 opcodes are scaffolding. They make the substrate concrete enough to verify in Coq, run as OCaml, and synthesize to silicon, but they are not the substrate itself. The substrate is the step relation together with the cost-ledger monotonicity rule: every atomic transition either preserves μ or charges it. PNEW, MORPH, CHSH_LASSERT, the whole instruction set — those are the experimental apparatus that gives the substrate a concrete realization to probe. Strip the opcodes and the substrate is still there. Add different opcodes that respect the substrate's monotonicity discipline and the substrate is still the same substrate.

The Turing machine has the same shape. Tape, head, and transition table are scaffolding for one realization. Register machines, lambda calculus, counter machines, RAMs, and partial recursive functions all instantiate the same step-relation primitive with different scaffolding, and the equivalence between them is the content of the Church-Turing thesis. They are all the Turing machine in the sense that matters — they share the same notion of a step.

The result that follows is a fact about the substrate, not about the 47 opcodes. The opcodes are one realization. Other substrates that supply the same typeclass fields inherit the same theorem the moment they are presented.

18.1 The Substrate typeclass

The file `coq/kernel/foundation/Substrate.v` defines a Coq typeclass capturing exactly what an A2-respecting computational substrate is, divorced from any particular instruction set. The typeclass exposes:

- A type `state` of substrate states.
- A type `Program` of substrate programs.
- A bounded-execution function `run : Program → state → option state`, returning `Some r` on convergence and `None` on divergence.
- A cost function `mu : state → nat`.
- An atomic step relation `step : Program → state → state → Prop`.

- A cost-ledger monotonicity field `mu_monotone`: `forall p s s', step p s s' → mu s ≤ mu s'`. This is a strictly weaker general-purpose monotonicity property than the kernel's A2 (which specifically asserts that a step flipping certification false-to-true costs $\geq$ 1, and which is carried by the `CertificationSystem` record in `coq/kernel/nfi/UniversalCertificationCost.v`, not by this typeclass). The two coincide on the 47-opcode VM where all instruction costs are non-negative and the cert transition costs $\geq$ 1. See Section 2.8 for the A2 statement and the related axioms A3 through A6.
- An internal Gödel coding: `encode : Program → state` and `decode_safe : state → Program` with the round-trip property `decode_safe (encode p) = p`.
- An internal-representability predicate `Representable : (Program → Program) → Prop`, scoping which Coq function transformers the substrate can express as programs of its own kind. Concrete substrates instantiate this with a precise meaning; the minimal nat-coded substrate (`NatSubstrateInstance.v`) defines `Representable f` as "`f` is the diagonal flip transformer for the substrate's own decide function," which is exactly the shape the diagonalization needs.
- A recursion-theorem field, internal form: for every internally representable transformer `f : Program → Program`, there is a program `p` extensionally equivalent to `f p`.

The recursion-theorem field is the load-bearing piece. It is the substrate's Kleene 1938 recursion theorem, restricted to transformers the substrate can actually express. The restriction is not a hedge; it is the substrate's own scope. Turing 1936 did not prove "no decider exists in any conceivable mathematical universe"; he proved "no Turing machine decides halting," because Turing machines were the substrate Turing was studying. The restriction to internally representable transformers is the same scope move at the substrate level: the substrate's limitative theorem is about what the substrate itself can express, not about every Coq function that exists in the meta-theory.

The recursion theorem is not derivable from `mu_is_initial_monotone` (the cost-functional uniqueness result in `MuInitiality.v`). Initiality of the cost functional is the statement that any monotone cost measure satisfying the axioms equals μ on reachable states; that is a uniqueness theorem, collapsing a type to a single inhabitant. The recursion theorem is a fixed-point statement, asserting that a non-trivial type admits self-reference. They are different categorical statements. The recursion theorem is supplied by the typeclass field above and discharged unconditionally for the minimal nat-coded substrate by exhibiting a self-aware program that is its own diagonal flip's fixed point.

18.2 The shortcut-admittance predicate

The substrate becomes interesting for the limitative result when it is equipped with a non-trivial *shortcut-admittance predicate*: a Prop-valued predicate `AdmitsShortcut` on programs, witnessed by at least one program that admits and at least one that does not, and depending only on extensional behavior (programs with the same `run` behavior on every state admit shortcuts iff each other).

The typeclass extension `WithShortcutPredicate` packages this: an `AdmitsShortcut` field, two canonical witnesses `yes_program` and `no_program` with their respective inhabitedness or non-inhabitedness lemmas, and the extensionality field. These are the conditions the diagonalization needs. A substrate that supplies them earns the limitative theorem.

Two concrete substrate instances are constructed in the kernel.

The minimal nat-coded substrate `nat_substrate` (in `coq/kernel/foundation/NatSubstrateInstance.v`) discharges every typeclass field unconditionally, no `Hypothesis` and no `Axiom`. Its programs are natural numbers; three program codes have meaning (`0` = yes, `1` = no, `2` = the self-aware flip fixed point); `run` is a Coq `Fixpoint`; `Representable f` is the predicate "`f` is the diagonal flip transformer for the substrate's own decide function"; `recursion_theorem` is discharged by exhibiting code `2` as the fixed point of any such transformer. `Print Assumptions nat_recursion_theorem` is `Closed under the global context`. The substrate-level limitative theorem fires unconditionally for this substrate.

The 47-opcode VM substrate `vm_substrate` (in `coq/kernel/foundation/VMSubstrateInstance.v`) is parameterized over the VM-specific Gödel encoding (`vm_encode`, `vm_decode_safe`, the round-trip), the VM's internal-representability predicate (`vm_representable`), and the VM's internal recursion theorem restricted to representable transformers. The Coq Section discipline is identical to `BekensteinBound.v`'s use for physical-constant positivity: the parameters are not global axioms; closing the surrounding Section discharges them as explicit `forall` premises on every consumer. The encoding piece is a closed lemma (`coq/kernel/foundation/VMInstructionEncoding.v` provides `program_to_nat`, `nat_to_program`, and the round-trip with no Hypothesis). The VM's internal-representability predicate and internal recursion theorem are the VM-instance-specific Section parameters; the substrate-level result does not depend on them, because `nat_substrate` discharges the limitative theorem unconditionally at the substrate level.

The VM-side `AdmitsShortcut` is the extensional predicate "this VM program's bounded run from `init_state` coincides with the bounded run of `simple_morph_trace`." `yes_program` is `simple_morph_trace` itself (defined at `coq/kernel/nfi/SimpleMorphShortcut.v:32`); `no_program` is

the empty trace (which produces `init_state`, demonstrably distinct from `simple_morph_final` at `coq/kernel/nfi/SimpleMorphShortcut.v:38` since the latter has fired the supra-cert channel and the former has not). The extensionality field is the standard "equal `run` on every state implies the predicates agree." The instantiation that assembles these into the `WithShortcutPredicate` interface lives in `coq/kernel/nfi/StructuralUndecidability.v`.

18.3 The substrate-level limitative theorem

Theorem 18.1 (Structural-axis undecidability, substrate-level (`coq/kernel/nfi/StructuralUndecidability.v:127-164`)). *For every Substrate-instance equipped with a WithShortcutPredicate extension, there is no Coq-definable function `decide : Program → bool` whose corresponding diagonal flip transformer is internally representable in the substrate and that for all programs `p` satisfies: `decide p = true` iff `AdmitsShortcut p`.*

Proof. Assume for contradiction such a `decide` exists. Define the diagonal flip transformer

$$\texttt{flip}(p) \;=\; \begin{cases} \texttt{no\_program} & \text{if } \texttt{decide}\, p = \texttt{true} \\ \texttt{yes\_program} & \text{if } \texttt{decide}\, p = \texttt{false} \end{cases}$$

By hypothesis, `flip` is internally representable in the substrate. Apply the substrate's typeclass field `recursion_theorem` to `flip`: there exists `p` such that `prog_equiv` p (`flip` p), where `prog_equiv` is the extensional equivalence "`run` agrees on every initial state."

Case-split on `decide p`. *Case A* (`decide p = true`): by the bi-implication, `AdmitsShortcut p`; by definition of `flip`, `flip` p = `no_program`; by the fixed-point witness, `prog_equiv` p `no_program`; by extensionality of `AdmitsShortcut` (typeclass field), `AdmitsShortcut p` iff `AdmitsShortcut no_program`. The latter is false (typeclass field `no_program_no_shortcut`). Contradiction.

Case B (`decide p = false`): symmetrically, by the bi-implication ¬ `AdmitsShortcut p`; `flip` p = `yes_program`; `prog_equiv` p `yes_program`; extensionality yields `AdmitsShortcut p` iff `AdmitsShortcut yes_program`. The latter is true (typeclass field `yes_program_admits`). Contradiction.

Both cases close. No such `decide` exists. `Print Assumptions structural_shortcut_undecidable` returns `Closed under the global context`. □

Plain reading. Kleene 1938's diagonal, transplanted from time to structure. Same shape: assume a decider, build a flipper, find a fixed point, watch the program disagree with itself in both cases. The work is in the typeclass — once the substrate has its own recursion theorem, two canonical inhabitants on opposite sides of the predicate, and

extensionality, the diagonal lands itself. The structural axis has its own halting wall, on its own axis, internal to its own machinery. Not transported. Not assumed. Built.

Theorem 18.2 (Structural-axis undecidability for the nat-coded substrate (`coq/kernel/foundation/NatSubstrateInstance.v`)). *For every Coq function `d : nat → bool`, the substrate `nat_substrate d` cannot internally decide its own structural-shortcut membership predicate. In particular, `d` itself is not a correct decider for the predicate of the substrate it parameterizes; the corollary `nat_self_undecidable` reads off this self-referential conclusion directly.*

Proof. The substrate `nat_substrate` discharges every typeclass field constructively: `Program := nat`; `yes_program := 0`; `no_program := 1`; `run` is a Coq `Fixpoint`; `prog_equiv` is pointwise equality of `run`; `Representable f` is the precise predicate "`f` is the diagonal flip transformer for the substrate's own decide function"; `recursion_theorem` is discharged by exhibiting code `2` as the fixed point of any such transformer (its `run` is defined to mirror whatever the transformer outputs); `AdmitsShortcut` is extensional by construction; `yes_program_admits` and `no_program_no_shortcut` hold by definitional unfolding of `run` at `0` and `1`. Apply the substrate-level theorem above. The corollary `nat_self_undecidable` specializes the conclusion to `d` itself: `d` cannot be a correct decider for `nat_admits d`. `Print Assumptions` on both `nat_structural_shortcut_undecidable` and `nat_self_undecidable` returns `Closed under the global context.` □

Plain reading. The substrate-level theorem with every typeclass field hand-discharged. No `Hypothesis`, no `Axiom`, no Section parameter — a concrete substrate built out of three natural numbers and a fixpoint, where the limitative result fires unconditionally. The diagonal is not a thing that might exist for some substrates; here it exists, and it bites.

Theorem 18.3 (Structural-axis undecidability for the 47-opcode VM (`coq/kernel/nfi/StructuralUndecidability.v`)). *For the 47-opcode VM, there is no Coq function `decide : list vm_instruction → bool` whose diagonal flip transformer is internally representable in the VM language and that for all programs `p` satisfies: `decide p = true` iff `vm_admits_shortcut_extensional p`.*

Proof. Instantiate the substrate-level theorem at `vm_substrate`. The Gödel encoding piece (`vm_encode`, `vm_decode_safe`, round-trip) is supplied as a closed lemma by `coq/kernel/foundation/VMInstructionEncoding.v` (`program_to_nat`, `nat_to_program`, no Hypothesis). The VM's internal-representability predicate `vm_representable` and the VM's internal recursion theorem restricted to representable transformers are the VM-instance-specific Section parameters; closing the surrounding Coq Section discharges them as explicit `forall` premises on every consumer, the same Section-parameter discipline used by `BekensteinBound.v` for $\hbar > 0$, $c > 0$, $k_B > 0$. The VM-side `AdmitsShortcut` is the

extensional predicate "this VM program's bounded run from `init_state` coincides with the bounded run of `simple_morph_trace`"; `yes_program` is `simple_morph_trace` (`coq/kernel/nfi/SimpleMorphShortcut.v:32`); `no_program` is the empty trace, which produces `init_state` and so differs from `simple_morph_final` (`coq/kernel/nfi/SimpleMorphShortcut.v:38`, where the supra-cert channel has fired). The substrate-level diagonal applies; `vm_structural_shortcut_undecidable` is its specialization, carrying the VM-side parameters as premises. The substrate-level content is closed at `nat_substrate` (above); the VM corollary presents the same content at VM scope. □

Plain reading. The 47-opcode VM is one instance of the substrate. The wall exists at the substrate level — `nat_substrate` already proved that unconditionally. The VM corollary is the same wall, named at VM scope, with the VM-specific Gödel-encoding machinery written out as premises in the same style `BekensteinBound.v` writes out the physical constants. The kernel proves the limitative theorem once at the substrate; the VM inherits.

18.4 The proof, in prose

The proof is the standard Kleene-1938 diagonal, transported to the substrate.

Assume for contradiction that `decide : Program → bool` satisfies the bi-implication. Define the flipping transformer

$$\texttt{flip}(p) = \begin{cases} \texttt{no\_program} & \text{if } \texttt{decide p = true} \\ \texttt{yes\_program} & \text{if } \texttt{decide p = false} \end{cases}$$

This is a total Coq function from `Program` to `Program`. Apply the substrate's recursion-theorem field to `flip`: there exists a program `p` with `prog_equiv p (flip p)`.

Case-split on `decide p`.

Case A: `decide p = true`. By the bi-implication, `p` admits a shortcut. By definition of `flip`, `flip p = no_program`. By the recursion-theorem witness, `p` is extensionally equivalent to `no_program`. By extensionality of `AdmitsShortcut`, `p` admits iff `no_program` admits. But `no_program` does not admit. Contradiction.

Case B: `decide p = false`. By the contrapositive of the bi-implication, `p` does not admit. By definition of `flip`, `flip p = yes_program`. By the recursion-theorem witness, `p` is extensionally equivalent to `yes_program`. By extensionality, `p` admits iff `yes_program` admits. But `yes_program` admits. Contradiction.

Both cases close. No total decision procedure for `AdmitsShortcut` exists. □

The proof is short — the body runs about 38 lines

(`coq/kernel/nfi/StructuralUndecidability.v:127-164`), of which roughly 16 are actual tactic steps; the rest is the case-split structure and the inline narration — because the work is in the typeclass, not in the diagonal. Once the substrate has a recursion theorem, two canonical inhabitants, and an extensional predicate, the diagonal is mechanical. The substrate-level work was assembling the right typeclass; the limitative result is what falls out.

18.5 The two-axis picture this completes

Time axis	Structural axis
steps	μ
HALT (Turing 1936)	AdmitsShortcut (above)
no decision procedure for halting	no decision procedure for shortcut admittance

The substrate's own halting-shaped wall, on its own axis, proved by its own diagonal, internal to the substrate's typeclass machinery. Not transported from the time axis by reduction. Not axiomatic. Built.

18.6 What this closes

What the substrate-level theorem closes. Whether a uniform translation procedure exists from informal structural arguments into `SoundStructuralShortcut` witnesses reduces to whether the substrate can internally decide membership in its own structural-shortcut class. The theorem above answers that in the negative: no internally representable decision procedure for the membership predicate exists, so no uniform internally representable translation procedure exists either. The structural axis has its own undecidable predicate, on the same diagonal shape Turing used in 1936.

What the nat-substrate theorem closes. The minimal nat-coded substrate `nat_substrate` discharges every typeclass field constructively and the limitative theorem fires unconditionally. `Print Assumptions nat_structural_shortcut_undecidable` and `Print Assumptions nat_self_undecidable` both return `Closed under the global context`. The substrate's own halting-shaped wall is, at the substrate level, a closed Coq theorem.

What the VM corollary closes. The 47-opcode VM corollary `vm_structural_shortcut_undecidable` is the substrate-level theorem instantiated at `vm_substrate`. It carries the VM-side Section parameters as explicit `forall` premises in the Bekenstein-bound style. The substrate-level content does not depend on the VM-side parameters; it is closed at the `nat_substrate` level. The VM corollary is the substrate-level result presented at the VM scope, with the VM-side internal-representability work named in its premises.

How to falsify. Three routes.

1. Exhibit a Coq function `decide : Program → bool` whose diagonal flip transformer is internally representable in some Substrate plus WithShortcutPredicate instance and that satisfies the bi-implication. The substrate-level proof must fail at some step; identify which.
2. For the nat-coded substrate specifically: exhibit a Coq function `d : nat → bool` that correctly decides `nat_admits d`. The corollary `nat_self_undecidable` forbids this; finding such a `d` contradicts the corollary.
3. Exhibit a substrate that has both an internal recursion-theorem witness for a representable transformer and a non-trivial extensional predicate, but for which the diagonal construction does not produce a contradiction. The proof body is about 38 lines (`coq/kernel/nfi/StructuralUndecidability.v:127-164`), roughly 16 of them actual tactic steps; finding such a substrate means finding an error in those tactic steps.

19 What Coq is, and why I used it

Coq is a proof assistant. It is a programming language where the type system is so expressive that you can write proofs as programs and have the compiler check whether they are valid.

Here is the idea. In a normal programming language, a type says something about what kind of value a variable holds. In Python, `x: int` means x is an integer. In Coq, types are propositions. The type `n > 0` means "the proof that n is positive." A function from `n > 0` to `n + 1 > 0` is a proof that if n is positive, $n + 1$ is too.

The compiler checks whether the types line up. If your proof has a hole and you write "trust me," the compiler rejects it. There is no way to convince the compiler of a false theorem by being persuasive or authoritative. The only way through is to actually prove it.

I used Coq because I don't trust informal arguments. Including my own. I spent months being convinced by my own reasoning that various claims were true, only to find when I tried to push the Coq proof through that some key step was wrong. The machine caught it. That is the point.

19.1 The Inquisitor

Beyond Coq's type checker, the project has a custom proof auditor called the Inquisitor (`scripts/inquisitor.py`). It runs on every commit and emits 86 distinct rule codes (see the Rules section of the generated `INQUISITOR_REPORT.md`). The main categories:

- **No holes.** No `Admitted`, `admit`, or `give_up` anywhere in the active proof files. No unproved `Axiom` or `Parameter` declarations in project code.

- **No vacuous theorems.** No theorem whose conclusion is literally `True`, `0 = 0`, or any other tautology that proves nothing. No "theorem" of the form $P \to P$. No existence claims backed by constant witnesses. Coq's kernel already prevents circularly dependent proofs (it checks definitional well-foundedness); the Inquisitor catches separately the case where a proof is technically valid but epistemically empty, proving `True` by `trivial`, for instance.
- **No physics stubs.** No physics quantity defined as a constant placeholder. No physics claim without a corresponding invariance condition. No theorem stated about a physics concept if that concept is not genuinely formalized.
- **No trivial true proofs.** If a theorem's entire proof is `trivial` or `tauto` and the conclusion reduces to `True`, the theorem is rejected.
- **Scope enforcement.** The proof tree is split into tiers. Tier 1 is the kernel files: the core proof of the machine. Tier 1 files may not import from exploratory or research-side files. Theorem connectivity: every file in the active proof tree must reach the cost or semantics foundation through its import chain. If a file is disconnected, it does not get to claim the kernel's pedigree.
- **Comment hygiene.** No TODO/FIXME/WIP markers in active proof comments.

Current Inquisitor status (250 Coq files scanned): zero HIGH findings, zero MEDIUM, and exactly one LOW. The LOW is a vacuity-score warning on `coq/kami_hw/F4_VerilogEvaluator.v`, flagged because the file matches a heuristic for "constant function" patterns. That file is the F4 reference Verilog evaluator surface (a small implementation of the reference 4-element field F_4, deliberately defined in terms of small constant predicates and enumerated boolean functions, which is what triggers the heuristic). The LOW is documented and audited. It does not correspond to a missing proof step.

20 Zero Admitted: what it actually means

"Zero Admitted" means: every theorem in the proof tree was derived without any gaps. No step was asserted without a derivation. The machine verified the complete chain.

This is what it does NOT mean:

- It does not mean the theorems prove physical reality. The theorems prove things about the Thiele Machine model, which is a formal object in Coq. Whether the model correctly describes physical reality is a separate empirical question.
- It does not mean the Coq axioms themselves are consistent. Coq is built on a formal foundation called the Calculus of Inductive Constructions. The key idea underlying it is the Curry-Howard correspondence: proofs and programs are the same thing. A proof that P is true is literally a program of type P. Checking

the proof is the same as type-checking the program. The "inductive" part means you can define your own recursive types, natural numbers, lists, trees, inside the system instead of having them as magical primitives. This matters because it makes the foundation explicit and auditable.
Now here is the honest limitation. The sources I went and read on Coq's foundation say it is believed to be logically consistent, meaning you cannot derive a contradiction from it, but this has not been proven within Coq itself. The way I understand it (and I do not fully trust my own reading), that is not a weakness specific to Coq. The label I dug up is Gödel's incompleteness theorem from 1931: any formal system powerful enough to do arithmetic cannot prove its own consistency from within itself. If it could, it would be contradictory. So Coq's foundation is trusted, not proved-from-within. "Trusted" has substance here: the same foundation has been used to verify the CompCert C compiler and the Four Color Theorem (Gonthier 2005, the first published machine-checked proof of a major open problem in mathematics), among other artifacts. Large-scale inconsistencies in the foundation would have surfaced through these uses if they were there. I am relaying what I read; I have not personally audited CompCert or Gonthier's proof. "Zero Admitted" means every theorem follows from those axioms. It does not mean those axioms are themselves in-system guaranteed; it means they are the same axioms a substantial body of verified software has relied on without breaking.

- It does not mean the hardware implements exactly the Coq semantics in all cases. All 47 opcodes have Qed bisimulation proofs (37 unconditional, 10 under structural invariants `coupling_wf` and `morph_table_wf` that hold inductively from hardware reset, zero structural gaps); documented in `RTLGapRegistry.v` and `CloseoutVerification.v`.
- It does not mean the OCaml runner binary is proven inside Coq to be bitwise identical to the Coq semantics. `OCamlExtractionBridge.v` proves Coq-side observable facts and the parity suite checks the extracted binary empirically.

The kernel tier (Section 8) is the strongest tier. The theorems listed there have zero Admitted and zero named hypotheses. They are machine-checked derivations from the Coq type theory axioms and the Thiele Machine definitions. Nothing else.

The bridge tier involves named hypotheses and documented trust boundaries. Three of them: the BSC (Bluespec compiler) trust boundary for hardware (Section 14 introduces Bluespec; the BSC compiler is the tool that translates Bluespec into Verilog, and the proof assumes this tool's output faithfully implements its input), the A_QM physical assumption for the quantum connection (a named hypothesis I use sparingly, marking the place where the Coq proof would need a physics input it does not derive; see `coq/PhysicsConditionalClosure.v`), and the external OCaml parity boundary (described in Section 14: the OCaml binary is checked against the Coq semantics by

tests, not by a Coq theorem). These are explicitly labelled and auditable. They are not Admitted proofs. They are documented limits of the formal claim.

21 How to break this

Every claim in this document has a falsification condition. Here are the main ones.

Falsify No Free Insight.
Find a Thiele Machine program trace that:

- starts with `vm_certified = false` and `vm_mu = 0`
- ends with `vm_certified = true`
- ends with `vm_mu = 0`

If you find such a trace, the Coq proof of `certification_requires_positive_mu` is wrong and the compilation will fail when you add the counterexample as a theorem.

Falsify μ-monotonicity.
Find a state s and instruction i such that $(\texttt{vm\_apply}\ s\ i).\mu < s.\mu$. The Coq proof of `vm_apply_mu` will fail.

Falsify categorical separation.
Prove that for ALL states s_1, s_2 with identical classical shadow, ALL instructions produce the same error behavior. This would contradict `categorical_separation` in Coq, meaning the proof has an error.

Falsify Turing strictness.
Prove that every Thiele program can be simulated by a classical program with the same observable behavior (registers, memory, μ) without ever touching the partition graph or morphism map. This would contradict `D5_thiele_strictly_extends_classical`.

Falsify structural entitlement.
Find two Thiele states with the same classical shadow where no probe can distinguish their structural fields. That would contradict `shadow_strictly_lossy`. Or produce a strict feasible-set subset with a distinguishing observation that does not yield stronger membership predicates; that would contradict the feasible-subset theorem.

Falsify universal certification cost.
Build a `CertificationSystem` satisfying the one-step rule `cs_cert_costs`, then give a trace from uncertified to certified with total cost 0. That would contradict `universal_nfi_any_substrate`. If you instead drop the one-step rule, you have found a free-certification system, not a counterexample.

Falsify substrate-level structural-axis undecidability.
Three routes (Section 18 expands).

- Exhibit a Coq function `decide : Program → bool` whose diagonal flip transformer is internally representable in some Substrate plus WithShortcutPredicate instance and that satisfies the bi-implication. The substrate-level proof of `structural_shortcut_undecidable` must fail at some step; identify which.
- For the nat-coded substrate specifically: exhibit a Coq function `d : nat → bool` that correctly decides `nat_admits d`. The corollary `nat_self_undecidable` forbids this; finding such a `d` contradicts the corollary directly.
- Exhibit a substrate that has both an internal recursion-theorem witness for a representable transformer and a non-trivial extensional predicate, but for which the diagonal construction does not produce a contradiction. The proof body is about 38 lines in `coq/kernel/nfi/StructuralUndecidability.v:127-164`, roughly 16 of them actual tactic steps; finding such a substrate means finding an error in those tactic steps.

Falsify the classical CHSH bound.
Exhibit a locally factorizable strategy that achieves $|S| > 2$. This would contradict `chsh_stat_violation_not_local` and also violate known mathematics.

The Inquisitor standard.
Run `python scripts/inquisitor.py` on the repository. If it finds any HIGH or MEDIUM finding, or any new LOW beyond the single documented `F4_VerilogEvaluator.v` vacuity-score warning, the proof hygiene has degraded. Current status: 0 HIGH, 0 MEDIUM, 1 LOW (the documented F4 evaluator finding). If that changes, it is a bug.

Falsify the axiom audit.
If running `Print Assumptions` (Section 2: the Coq command that lists everything a theorem depends on) on any named theorem in the kernel tier shows a project-local `Axiom` or `Parameter` in the dependency tree, the audit has degraded. The current closure across all kernel theorems shows 2,539 named theorems that compile under the global Coq context with no project-local axioms. A separate companion set of 25 theorems depends on the excluded middle, which is the classical-logic axiom $P \vee \neg P$ ("either P is true or P is false, with no third option"); those uses come from the Coq standard library (specifically, the real-number theory needed for the CHSH algebra), not from project code. The two sets are disjoint: the 2,539 are closed under the global context with no project-local axioms appearing, and the 25 sit in the broader `depends_on_axioms` bucket. Any project-local axiom or parameter appearing in

either set's dependency trees is a bug.

22 The full claim ledger

These are the kernel-tier claims (machine-checked, no named hypotheses):

1. *no_free_certification*: csr_cert_addr going $0 \to$ nonzero in one step requires cost ≥ 1.
2. *no_free_certification_mu*: same, with μ increment ≥ 1.
3. *no_free_certification_trace_mu*: trace-level version.
4. *no_free_certification_certified*: vm_certified going false→true requires cost ≥ 1.
5. *certification_requires_positive_mu*: both channels covered.
6. *universal_nfi_any_substrate*: any abstract certification system satisfying the one-step cost rule has trace-level non-free certification.
7. *thiele_universal_nfi_cert_addr*: universal theorem instantiated for Thiele's `csr_cert_addr` channel.
8. *thiele_universal_nfi_certified*: universal theorem instantiated for Thiele's `vm_certified` channel.
9. *thiele_represents_simulating_cert_system*: any certification system with a simulation morphism into Thiele transfers certification into a Thiele certified execution.
10. *forget_kernel_is_eq_on_classical*: the classical snapshot quotient forgets exactly the non-classical fields.
11. *blindness_non_injective*: distinct Thiele states can have the same classical snapshot.
12. *certification_is_lost*: certification can be in the kernel of the classical forgetful map.
13. *shadow_strictly_lossy*: the classical shadow forgets morphism structure.
14. *mu_ledger_necessity*: no strict-classical oracle on $(\texttt{mem}, \texttt{regs}, \texttt{pc})$ recovers the pair $(\mu, \texttt{certified})$.
15. *vm_mu_not_classically_determined / vm_certified_not_classically_determined*: neither ledger balance nor certification status is a function of strict classical state.
16. *mu_ledger_mutual_independence*: μ and `vm_certified` are each independent of strict classical state and of each other under cost-only/cert-only projections.
17. *mu_ledger_minimality / P_full_is_minimal_complete_extension*: the full $(\texttt{mem}, \texttt{regs}, \texttt{pc}, \mu, \texttt{certified})$ projection is complete, and dropping either μ or certification destroys the corresponding completeness.
18. *thiele_nfi_pc_indexed*: PC-indexed execution version.
19. *mu_is_initial_monotone*: μ is the unique canonical cost measure.
20. *vm_apply_mu*: exact ledger identity $(\texttt{vm\_apply}\ s\ i).\mu = s.\mu + \texttt{instruction\_cost}(i)$. The cost is not just δ, it is δ for ordinary instructions, $S(\delta)$ for cert-setters, $|\text{payload}|_{\text{bits}} + S(\delta)$ for EMIT, etc.
21. *kernel_certified_implies_positive_mu*: from zero, reaching certified implies $\mu > 0$.
22. *feasible_strict_subset_implies_strict_predicates*: feasible-set narrowing plus a distinguishing observation yields a strictly stronger predicate.

23. *strengthening_requires_structure_addition*: certified predicate strengthening requires a structure-addition event.
24. *b4_information_reduction_derives_strict_predicates*: feasible-set reduction can generate the strict-predicate premise used by NoFI.
25. *info_priced_cert_executions_bound*: executed cert-setting events are bounded by $\Delta\mu$.
26. *observation_partition_reduction_implies_posterior_representative_reduction*: observation classes package into the posterior-representative witness for structural entitlement.
27. *info_priced_arbitrary_feasible_reduction_bound*: with a decision-tree/representative witness, feasible-set compression is bounded by $\Delta\mu$.
28. *info_priced_weighted_feasible_reduction_bound*: the $\Delta\mu$ lower bound extends to nat-weighted feasible sets.
29. *structural_entitlement_representation*: strict narrowing plus distinguishability, certified posterior admissibility, and an explicit posterior-representative reduction imply a strictly stronger predicate, a structure-addition event, and a $\Delta\mu$ lower bound.
30. *every_sound_structural_shortcut_lands_here*: every packaged `SoundStructuralShortcut` instantiates the structural-entitlement representation theorem.
31. *structural_shortcut_undecidable* (`coq/kernel/nfi/StructuralUndecidability.v`): for every Substrate plus WithShortcutPredicate instance, no Coq function whose diagonal flip transformer is internally representable in the substrate decides the structural-shortcut admittance predicate. The substrate's own halting-shaped wall, by Kleene 1938 diagonal restricted to the substrate's internal expressive power.
32. *nat_recursion_theorem / nat_structural_shortcut_undecidable / nat_self_undecidable* (`coq/kernel/foundation/NatSubstrateInstance.v`): the minimal nat-coded substrate discharges every `Substrate` typeclass field unconditionally (no Hypothesis, no Axiom). The recursion theorem is proved by exhibiting a self-aware fixed-point program (code `2`). The substrate-level limitative theorem fires unconditionally, and the corollary `nat_self_undecidable` reads off: the substrate cannot internally decide its own structural-shortcut predicate. `Print Assumptions` on all three returns `Closed under the global context`.
33. *vm_structural_shortcut_undecidable* (`coq/kernel/nfi/StructuralUndecidability.v`): substrate-level theorem instantiated at the 47-opcode VM. Carries the VM-side Section parameters (Gödel encoding, internal-representability predicate, internal recursion theorem) as explicit `forall` premises in the Bekenstein-bound style. The substrate-level content is unconditional at `nat_substrate`; the VM corollary presents the result at the VM scope.
34. *program_to_nat / nat_to_program / nat_to_program_program_to_nat* (`coq/kernel/foundation/VMInstructionEncoding.v`): closed Gödel encoding of `list vm_instruction` as a `nat` with a proven left inverse, all 47 opcode constructors handled, no Hypothesis. Composes the existing binary encoders from `VMEncoding.v` with a `list bool ↔ nat` bijection through `positive`.
35. *categorical_separation*: classical-identical states distinguishable by morphism probe.

36. *classical_observer_cannot_separate*: no classical function separates morphism-distinct states.
37. *shadow_proj / shadow_separation_theorem*: formal surjection, strictly lossy.
38. *D2_faithfulness*: classical programs embedded faithfully in Thiele.
39. *D3_conservativity*: classical programs leave structural layer untouched.
40. *D4_strictness*: there exist Thiele steps that escape the classical fragment.
41. *D5_thiele_strictly_extends_classical*: Thiele strictly exceeds classical semantics.
42. *degenerate_projection_theorem*: shadow projection is the classical equivalence quotient.
43. *no_classical_certification_decider*: no classical machine can decide morphism-certification.
44. *chsh_stat_violation_not_local*: CHSH-violating statistics are non-local.
45. *structural_trace_preserves_cert_addr*: traces with no cert-address setter preserve `csr_cert_addr`.
46. *partition_refinement_nonfree*: certified partition knowledge cannot be free.
47. *partition_free_but_certification_nonfree*: partition structure can be built at $\delta = 0$, but certified partition knowledge cannot be acquired for free.
48. *witness_insight_nonfree_general*: certified non-local witness insight requires $\mu \geq 1$.
49. *witness_insight_complete_taxonomy*: full three-tier taxonomy proven.
50. *separable_coupling / non_separable_coupling*: Functional (separable) and non-functional (non-separable) coupling predicates with concrete witnesses (`CategoryLaws.v`).
51. *CHSH coupling bridge* (`CHSHCouplingBridge.v`): Bell's theorem in morphism coupling language. A locally consistent strategy produces a separable coupling and has $|S| \leq 2$. $S > 2$ rules out every locally factorizable morphism coupling.
52. *graph_certify_morphism_lookup*: morphism certification cost utility, including lookup correctness.
53. *exists_covering_tree*: for any feasible-set reduction, a covering complete binary tree exists constructively.
54. *info_priced_reduction_no_tree_hypothesis*: entropy bound without requiring the tree as an explicit input.
55. *sound_shortcut_from_components*: assembly lemma for SoundStructuralShortcut from its component witnesses.
56. *mu_budget_decidable / mu_budget_verifiable / classical_mu_budget_decidable*: predicates classifying decision problems by their polynomial-μ envelope, plus the inclusion that every zero-μ classical solver is mu_budget_decidable. Not a class separation. See Section 23 for what these predicates do and do not say.
57. *column_contractive_iff_quantum_realizable* (`coq/kernel/quantum/QuantumPartitionPSD.v`): Thiele honesty $\iff$ zero-marginal NPA realizability. A zero-axiom biconditional. Forward (`zero_marginal_npa_column_contractive_implies_psd` in `coq/kernel/nfi/MuLedgerQuantumBridge.v`): column-contractive $\to$ NPA PSD. Reverse (`npa_psd_implies_column_contractive` in `coq/kernel/quantum/QuantumPartitionPSD.v`): NPA PSD $\to$

column-contractive, proved by vector specialization plus the `quadratic_nonneg_discriminant` lemma from `coq/kernel/quantum/ConstructivePSD.v`. Zero-marginal NPA realizability is the orthogonal-observables slice of the quantum elliptope ($\rho_{AA} = \rho_{BB} = 0$), not the full elliptope. Classical correlations like $(1, 0, 1, 0)$ have $|S| \leq 2$ but fail column contractivity. The classical polytope and the Thiele-honest set intersect but neither contains the other.

58. *state_column_contractive_implies_tsirelson* (`coq/kernel/nfi/MuLedgerQuantumBridge.v`): Thiele-honest $\Rightarrow |S|^2 \leq 8$, i.e., $|S| \leq 2\sqrt{2}$. The proof factors through quantum realizability: `state_column_contractive_implies_quantum_gram` composes with `quantum_realizable_implies_tsirelson_bound` (in the same file). The row-bounds lemma `tsirelson_from_row_bounds` in `coq/kernel/quantum/TsirelsonGeneral.v` is the algebraic Cauchy–Schwarz step inside this chain. The contrapositive ($|S| > 2\sqrt{2} \Rightarrow$ not Thiele-honest) follows immediately. The converse ($|S| \leq 2\sqrt{2} \Rightarrow$ Thiele-honest) is false and is not claimed.

59. *Concrete falsifications by direct arithmetic.* The PR box ($E_{00} = E_{01} = E_{10} = 1$, $E_{11} = -1$) fails the first column-contractivity condition: $1 - E_{00}^2 - E_{10}^2 = 1 - 1 - 1 = -1 < 0$. Any super-quantum value $|S| > 2\sqrt{2}$ falls outside the Thiele-honest set by the contrapositive of `state_column_contractive_implies_tsirelson`. The Turing-classical correlation $(1, 0, 1, 0)$ also fails condition 1. None of these require new theorem statements; they are immediate by `lra` once the biconditional is destructed.

60. *mu_hierarchy_theorem* (`coq/kernel/mu_calculus/MuHierarchyTheorem.v:162`): for every $k \geq 1$, there exists a certified trace with cost exactly k, and every certified trace with `vm_mu` $\geq k$ has cost $\geq k$. Infinite strict μ-complexity separation.

61. *mu_hierarchy_no_upper_bound*: no fixed μ-budget suffices for all levels.

62. *Structural-invariant Qed group* (`coq/kami_hw/RTLGapRegistry.v`, `coq/kami_hw/EmbedStep.v`, `coq/kami_hw/GraphReconstructionBridge.v`): the 10 opcodes PNEW, PSPLIT, PMERGE, MORPH, MORPH_ID, MORPH_DELETE, MORPH_ASSERT, MORPH_GET, COMPOSE, MORPH_TENSOR carry Qed bisimulation proofs under the inductive invariant `coupling_wf` (= `coupling_desc_bounded ∧ coupling_pairs_in_range ∧ coupling_pairs_fully_populated`) and `morph_table_wf`. The preservation theorems `coupling_wf_kami_step_preserved` and `morph_table_wf_kami_step_preserved` show these invariants hold inductively at hardware reset and through every `kami_step` operation, so the precondition is discharged in practice for any state reachable by machine execution.

63. *RTL trust boundary* (`coq/kernel/hardware_bridge/VerilogRTLCorrespondence.v`): the Section Variable `rtl_step_correct` states that the synthesised RTL implements the Kami step function. It is backed empirically by the cosim suite (`tests/test_verilog_cosim.py`, 38 tests including `test_all_47_opcodes_in_cosim` which checks coverage of every opcode) and the parametrized cross-layer fuzz suite (`tests/test_fuzz_random_programs.py`, `tests/test_cross_layer_adversarial_fuzz.py`). `coq/kami_hw/VerilogSemantics.v` proves `rtl_step_correct` for the

COQKAMI model and names `bsc_kami_compilation_trusted` as the residual trust boundary covering the BSC compiler / pretty-printer pipeline.

Bridge-tier claims (machine-checked, with named hypotheses or documented trust boundaries):

- **Hardware bisimulation**: 47/47 opcodes with Qed proofs (37 unconditional + 10 under structural invariants `coupling_wf` / `morph_table_wf`, 0 structural gaps); documented in `RTLGapRegistry.v` (`rtl_gap_count = 0` and `rtl_coverage_partition` witnessing $37 + 10 + 0 = 47$) and `CloseoutVerification.v`.
- **OCaml extraction**: `ocaml_bisimulation_closure` proves NoFI, μ-monotonicity, and totality for the Coq-side extracted observable. The external OCaml binary equivalence is a parity-test trust boundary, not a kernel theorem.
- **Structural advantage**: `StructuralAdvantage.v` proves exact μ costs and the arithmetic time-tax facts for the factored-search witness; executable tests check the measured VM behavior.
- **Cost formula forcing**: `cost_necessity` + `cost_uniqueness`, conditional on Landauer-Unruh calibration for physical interpretation.

23 What I don't know

Here is what I don't know.

Physical interpretation of μ. Whether the μ ledger actually corresponds to thermodynamic entropy in a physical Thiele Machine is an open empirical question. The formal development is consistent with Landauer's principle. I think it does. I cannot prove it.

Complexity implications. `MuComplexity.v` defines two predicates: `mu_budget_decidable` (a solver exists for the problem within polynomial fuel and polynomial μ-cost) and `mu_budget_verifiable` (every positive instance has a certificate verifiable within those budgets). The names are descriptive of what the predicates actually say; they do not evoke the classical P-versus-NP question, and the formalism delivers no claim about it. The Coq file proves a basic implication (every decidable problem is verifiable, by re-running the solver and certifying). It does not prove a class separation. There is no concrete problem placed in `mu_budget_verifiable` but provably not in `mu_budget_decidable`, and the factored-search witness ($\mu = 18$ pays for one structural shortcut) is a witness-level cost gap between a blind and a sighted solver, not a class separation.

Whether AdmitsShortcut is the privileged choice of extensional predicate. The diagonal in `structural_shortcut_undecidable` (`coq/kernel/nfi/StructuralUndecidability.v`) works for *any* non-trivial extensional predicate on programs that comes with two canonical witnesses

(`yes_program`, `no_program`) and an extensionality field. The substrate-level theorem covers the whole family. The VM instance picks one specific predicate — "this program's bounded run from `init_state` coincides with the bounded run of `simple_morph_trace`" — and proves the limitative result for that one instance. Whether `simple_morph_trace`-coincidence is the natural choice, versus a predicate built directly from `has_supra_cert` reachability, versus receipt-list equality at a specific fuel bound, versus something else, is an open question. The substrate-level result is the family statement. The VM corollary is one tile that satisfies it. I picked the tile that was nearest to hand. I do not know if it is the right one.

Shannon-style lower bound. I went and looked into information theory because the kind of bound I wanted seemed to belong there. The sources I dug up attribute to Claude Shannon a 1948 result that you need at least $\log_2 n$ yes-or-no questions to identify one item from a set of n items. As I read it, that is the information-theoretic lower bound on search. Anything that compresses a set of n possibilities to a smaller set must gather at least that much information. The Thiele Machine analog: any feasible-set reduction (narrowing Ω to Ω') requires at least $\log_2^{\uparrow}(|\Omega|/|\Omega'|)$ bits worth of certified structural work. `MuShannonBridge.v` proves that for any feasible-set reduction, a covering decision tree EXISTS constructively (`exists_covering_tree`). The `info_priced_reduction_no_tree_hypothesis` theorem proves the entropy bound without requiring the tree as an explicit input, the tree is chosen automatically from the reduction sizes. The remaining open step: derive the realization condition (`decision_tree_realized_by_trace`) from arbitrary VM trace behavior without requiring the user to verify the leaf count explicitly. The naive single-trace claim is false in general and the file records that explicitly.

Quantum mechanics connection. The mathematical equivalence between Thiele honesty and zero-marginal NPA realizability is a kernel-tier theorem (`column_contractive_iff_quantum_realizable` in `coq/kernel/quantum/QuantumPartitionPSD.v`, zero axioms). The zero-marginal NPA framework is the orthogonal-observables slice ($\rho_{AA} = \rho_{BB} = 0$): a proper subset of the full quantum elliptope. Whether this restriction maps to a physically privileged class of experiments, and whether a physical Thiele Machine can produce CHSH-violating statistics by exploiting quantum entanglement in hardware, are open empirical questions outside the scope of this project.

I could pretend these are answered. I could wave at them vaguely and say "future work." I don't know. These are real open questions.

24 Conclusion

Two pillars close this document, one at the instance level and one at the substrate level.

At the instance level, the strongest single result is `chsh_lassert_no_trap_implies_quantum_realizable` in

`coq/kernel/quantum/QuantumPartitionPSD.v`: a kernel-level Z-arithmetic check on CHSH outcome counters entails PSD of the NPA moment matrix, by a chain of Coq proofs that carries no physics premises. The algebraic bridge from a step-level certification check to a quantum realizability condition, with nothing in the dependency tree project-local. Run `Print Assumptions` on it; the answer is `Closed under the global context`.

At the substrate level, the strongest single result is `nat_structural_shortcut_undecidable` in `coq/kernel/foundation/NatSubstrateInstance.v`, together with the corollary `nat_self_undecidable`. A concrete substrate with every typeclass field discharged constructively, no Hypothesis and no Axiom, that cannot internally decide its own structural-shortcut predicate. The substrate's own halting-shaped wall on its own axis, by Kleene 1938 diagonal restricted to the substrate's internal expressive power, internal to the substrate's own typeclass machinery. `Print Assumptions` returns `Closed under the global context`.

CHSH-NPA at the instance level. The substrate-level structural-axis undecidability at the substrate level. The pair is what makes this document foundational rather than apparatus-with-results: an algebraic bridge with no physics premises on one side, and a limitative result internal to the substrate's own typeclass machinery on the other. The 47 opcodes, the OCaml runtime, the Bluespec hardware, the parity tests, the simulator harness are scaffolding for one realization of the substrate; the two theorems above are facts about substrates.

If you want to break this, the falsification conditions are explicit (Section 21). Find a trace from uncertified to certified at $\mu = 0$ and `certification_requires_positive_mu` falls. Exhibit a column-contractive correlator whose NPA matrix is not PSD and the CHSH-NPA bridge falls. Build a `CertificationSystem` that satisfies the one-step cost rule and gives a trace from uncertified to certified at total cost zero and `universal_nfi_any_substrate` falls. Exhibit a Coq function on `nat` that correctly decides `nat_admits` for the substrate parameterized over it, and `nat_self_undecidable` falls.

I built this. Read the proofs.